Machine Learning, Deep Learning & AI

Comprehensive Notes

A Conceptual and Mathematical Walkthrough

Linear Regression · Logistic Regression · Decision Trees · Random Forests · SVM · KNN

Naive Bayes · Gradient Boosting · ANN · Ridge/Lasso

CNN · RNN/LSTM · Transformers · GANs · VAEs · Diffusion Models

Reinforcement Learning · PCA/t-SNE/UMAP · AI Ethics · SHAP/LIME

Created by → **Sahil Raj**

Codeforces → **After_Dark**

September 3, 2026

Contents

Part I

The Story of Linear Regression

Introduction

Imagine you are trying to predict the price of a house based purely on its size (square footage). You plot 50 houses on a graph: *Size* is on the bottom (x-axis) and *Price* is on the left (y-axis).

You see a cluster of dots going up and to the right. Big houses cost more. Small houses cost less.

Key Idea

Linear Regression is simply the mathematical process of drawing the absolute "best" straight line through the center of that cloud of dots. Once you have that line, you can use it to guess the price of any house size, even ones you have never seen before.

But how does a computer know *where* to draw that line? And once it draws it, how do we know if it is actually a *good* line?

Here is exactly what is going on, step-by-step.

Step 1: The Equation (The Machine's Guess)

Every straight line in the universe is defined by a simple algebra equation:

$$y = mx + b \tag{1}$$

In Machine Learning, we change the letters to make it sound fancier:

$$\hat{y} = wx + b \tag{2}$$

- $\hat{y}$ **(y-hat)**: The computer's *Prediction* (e.g., predicted house price).
- x: The *Feature* (e.g., square footage).
- w: The *Weight* — the slope. How steep is the line? For every 1 extra square foot, how much does the price go up?
- b: The *Bias* — the y-intercept. If a house had 0 square feet, what is the starting base price of just the land?

Intuition

When you "train" a model, the computer starts entirely blind. It picks a random Weight and Bias and draws a terrible, random line on the graph.

Step 2: The Cost Function (Measuring the Mistakes)

To get better, the computer needs to know *how bad* its current line is.

It looks at every real house on your graph. It draws a vertical line straight down from the real house dot to the random line it just drew. That vertical distance is the **Error** (or **Residual**)—it is exactly how many dollars the computer's guess was off by.

The computer does this for all 50 houses, *squares* the errors (so negative and positive mistakes do not cancel each other out), and calculates the average. This final number is called the **Mean Squared Error (MSE)**.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2 \tag{3}$$

- If the MSE is **huge**, the line is terrible.
- If the MSE is **tiny**, the line is practically touching all the dots.

Step 3: Gradient Descent (Finding the Perfect Line)

Now the computer knows its MSE is huge. It needs to change the Weight (w) and Bias (b) to make the MSE smaller. But there are infinite numbers to guess. How does it know *which way* to adjust them?

Enter Gradient Descent.

Intuition

Imagine you are *blindfolded* on the side of a mountain, and your goal is to get to the very bottom of the valley (the lowest possible MSE).

1. You feel the ground with your foot to figure out which way is sloping downwards. (In math, this is taking the *derivative* or "gradient" of the cost function.)
2. You take a step in the steepest downward direction. (You adjust w and b slightly.)
3. You feel the ground again, take another step, and repeat.

The update rules are:

$$w \leftarrow w - \alpha \cdot \frac{\partial\,\text{MSE}}{\partial w} = w - \alpha \cdot \frac{2}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)\,x_i \tag{4}$$

$$b \leftarrow b - \alpha \cdot \frac{\partial\,\text{MSE}}{\partial b} = b - \alpha \cdot \frac{2}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i) \tag{5}$$

where α is the **learning rate**—how large a step you take.

The computer does this thousands of times per second. With every "step," it tweaks the line, recalculates the MSE, and checks the slope. When the ground becomes completely flat (the gradient is 0), you have reached the bottom of the valley. The algorithm stops. It has found the perfect Weight and Bias. The line is now officially "drawn."

Step 4: Grading the Model (The Statistics)

Just because the computer found the "best possible" line for your dots does not mean the line is actually *useful*. What if the dots are just random static, and house size actually has nothing to do with price?

We use statistics to *grade the model's homework*.

R-Squared (R^2): The "Score"

Imagine a *stupid model* that does not use square footage at all. Whenever you ask it for a house price, it just guesses the average price of all houses in the city. That forms a flat, horizontal line on the graph.

R^2 asks: *How much better is our trained line compared to that stupid, flat average line?*

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{6}$$

- $R^2 = 0$: Our model is *no better* than guessing the average. It learned nothing.
- $R^2 = 1$ (100%): Our model is *flawless*. Every single dot sits exactly on the line.
- $R^2 = 0.85$: The model explains *85% of the chaotic variance* in house prices. (This is a great score!)

The F-Value: The Signal-to-Noise Ratio

R^2 is just a percentage. The F-value is the raw ratio of **Signal** vs. **Noise**.

- **Signal (Explained Variance):** How much the prices change strictly because the houses get bigger (following our line).
- **Noise (Unexplained Variance):** How much the prices change for random reasons our line cannot explain (the errors/residuals).

$$F = \frac{\text{Signal}}{\text{Noise}} = \frac{\text{Mean Squared Regression}}{\text{Mean Squared Error}} \tag{7}$$

- If the F-value is a **big number** (e.g., 50, 100, 500), the Signal is drowning out the Noise. The model is definitively picking up on reality.
- If the F-value is **near 1**, your model is just listening to static.

The P-Value: The Proof

Even if you get a great F-value, statisticians are skeptical. They ask: *"What are the chances you got that F-value by pure, dumb luck?"*

The **p-value** is that probability.

Key Idea

If the overall model p-value is < 0.05 (less than 5%), we say the model is **Statistically Significant**. It means there is less than a 5% chance these results are a fluke. We can trust the model.

Individual P-Values (Checking the Features)

If we were trying to predict house prices using *Square Footage*, *Number of Bedrooms*, and *The Owner's Favourite Color*, we would get a p-value for each individual feature.

Feature	p-value	Decision
Square Footage	0.001	Highly significant — **keep it!**
Bedrooms	0.02	Significant — **keep it!**
Owner's Favourite Color	0.85	Not significant — **drop it.**

Table 1: Individual feature significance in a multi-feature model.

There is an 85% chance the last feature is just adding random noise. The algorithm will tell you to throw it out and run the model again.

Part II

Multiple Linear Regression

From Simple to Multiple

Simple linear regression draws a *line* through data with one predictor. **Multiple Linear Regression (MLR)** extends this to p predictors, fitting a **hyperplane** in a $(p+1)$-dimensional space:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i \quad i = 1, \ldots, n \tag{8}$$

where β_0 is the intercept, β_j are the partial regression coefficients, and $\varepsilon_i \overset{iid}{\sim} \mathcal{N}(0, \sigma^2)$ are the error terms.

Key Idea

The coefficient β_j measures the expected change in y for a one-unit increase in x_j, **holding all other predictors constant**. This is the *partial* or *ceteris paribus* interpretation — the key difference from simple regression.

Examples:

- House price $= \beta_0 + \beta_1(\text{size}) + \beta_2(\text{bedrooms}) + \beta_3(\text{age}) + \varepsilon$
- Salary $= \beta_0 + \beta_1(\text{years exp.}) + \beta_2(\text{education}) + \beta_3(\text{industry}) + \varepsilon$

The Matrix Formulation

Stack all n observations into matrix form. Define:

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad X = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & & & & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}, \quad \boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix} \tag{9}$$

where $X \in \mathbb{R}^{n \times (p+1)}$ is the **design matrix** (first column of all 1s for the intercept). The model is compactly:

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I_n) \tag{10}$$

Ordinary Least Squares (OLS) Solution

Minimise the Residual Sum of Squares (RSS):

$$\text{RSS}(\boldsymbol{\beta}) = \|\mathbf{y} - X\boldsymbol{\beta}\|^2 = (\mathbf{y} - X\boldsymbol{\beta})^\top (\mathbf{y} - X\boldsymbol{\beta}) \tag{11}$$

Taking the gradient and setting it to zero:

$$\frac{\partial \,\mathrm{RSS}}{\partial \boldsymbol{\beta}} = -2X^\top(\mathbf{y} - X\boldsymbol{\beta}) = \mathbf{0} \tag{12}$$

$$X^\top X\,\hat{\boldsymbol{\beta}} = X^\top \mathbf{y} \qquad \text{(Normal Equations)} \tag{13}$$

If $X^\top X$ is invertible (full column rank, i.e. no perfect multicollinearity):

$$\boxed{\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}} \tag{14}$$

The fitted values and residuals are:

$$\hat{\mathbf{y}} = X\hat{\boldsymbol{\beta}} = \underbrace{X(X^\top X)^{-1}X^\top}_{H}\,\mathbf{y} \qquad \hat{\boldsymbol{\varepsilon}} = \mathbf{y} - \hat{\mathbf{y}} = (I - H)\mathbf{y} \tag{15}$$

where $H = X(X^\top X)^{-1}X^\top$ is the **hat matrix** (or projection matrix). It is symmetric and idempotent ($H^2 = H$) and projects $\mathbf{y}$ orthogonally onto the column space of X.

Intuition

OLS finds the point $\hat{\mathbf{y}}$ in the column space of X that is *closest* (in Euclidean distance) to the observed $\mathbf{y}$. The residual vector $\hat{\boldsymbol{\varepsilon}}$ is always perpendicular to every column of X: $X^\top\hat{\boldsymbol{\varepsilon}} = \mathbf{0}$.

Properties of OLS Estimators

Property	Statement
Unbiasedness	$\mathbb{E}[\hat{\boldsymbol{\beta}}] = \boldsymbol{\beta}$ (holds under LINE assumptions).
Variance	$\mathrm{Cov}(\hat{\boldsymbol{\beta}}) = \sigma^2(X^\top X)^{-1}$.
Consistency	$\hat{\boldsymbol{\beta}} \xrightarrow{p} \boldsymbol{\beta}$ as $n \to \infty$.
Normality	Under normality of ε_i: $\hat{\boldsymbol{\beta}} \sim \mathcal{N}(\boldsymbol{\beta},\ \sigma^2(X^\top X)^{-1})$.

Table 2: Properties of OLS estimators.

The Gauss–Markov Theorem

Gauss–Markov Theorem (BLUE)

Under the assumptions:

1. Linearity: $\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}$.
2. X has full column rank (no perfect multicollinearity).
3. $\mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}$ (zero mean errors).
4. $\mathrm{Cov}(\boldsymbol{\varepsilon}) = \sigma^2 I$ (homoscedasticity + no autocorrelation).

OLS is the **B**est **L**inear **U**nbiased **E**stimator (BLUE) — among all linear unbiased estimators, OLS has the smallest variance for every β_j.

Note: "best" means minimum variance. If normality is additionally assumed, OLS is also the Maximum Likelihood Estimator (MLE).

Measuring Model Fit

R^2 and its Decomposition

The total variation in $\mathbf{y}$ decomposes as:

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{SST} = \underbrace{\sum_i (\hat{y}_i - \bar{y})^2}_{SSR} + \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{SSE} \tag{16}$$

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST} \quad \in [0, 1] \tag{17}$$

Problem with R^2: adding any predictor to the model — even a completely random one — *never decreases* R^2. It is trivially inflated by more predictors.

Adjusted R^2

Adjusted R^2 penalises unnecessary predictors:

$$\bar{R}^2 = 1 - \frac{SSE/(n-p-1)}{SST/(n-1)} = 1 - (1 - R^2)\frac{n-1}{n-p-1} \tag{18}$$

$\bar{R}^2$ increases only if a new predictor improves the model more than would be expected by chance. It can decrease when an unhelpful predictor is added. Use $\bar{R}^2$ (not R^2) for model comparison.

Information Criteria: AIC and BIC

For comparing models with different numbers of predictors, two penalised likelihood criteria are widely used:

$$AIC = n \ln\left(\frac{SSE}{n}\right) + 2(p+2) \qquad BIC = n \ln\left(\frac{SSE}{n}\right) + (p+2)\ln(n) \tag{19}$$

Lower is better. BIC penalises model complexity more heavily than AIC (by a factor of $\ln(n)$ vs. 2), so BIC tends to select more parsimonious models.

Criterion	Penalty	Prefers
R^2	None	Always adds predictors
$\bar{R}^2$	$(n-1)/(n-p-1)$	Parsimony, but inconsistent
AIC	$2(p+2)$	Predictive accuracy; slight overfit
BIC	$(p+2)\ln n$	True model (consistent selector)

Table 3: Model selection criteria compared.

Statistical Inference in MLR

Estimating σ^2

The error variance σ^2 is estimated by the **Mean Squared Error (MSE)**:

$$\hat{\sigma}^2 = s^2 = \frac{SSE}{n-p-1} = \frac{\hat{\boldsymbol{\varepsilon}}^\top \hat{\boldsymbol{\varepsilon}}}{n-p-1} \tag{20}$$

The denominator $n-p-1$ are the **residual degrees of freedom** (n observations minus $p+1$ estimated parameters).

t-Tests for Individual Coefficients

To test whether predictor j is significant given all others:

$H_0 : \beta_j = 0$ vs. $H_1 : \beta_j \neq 0$

$$t_j = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)} = \frac{\hat{\beta}_j}{\hat{\sigma}\sqrt{[(X^\top X)^{-1}]_{jj}}} \quad \sim \quad t_{n-p-1} \text{ under } H_0 \tag{21}$$

Reject H_0 if $|t_j| > t_{\alpha/2,\, n-p-1}$. The p-value of each coefficient in a regression summary table is exactly this test.

Caution: Individual vs. Joint Significance

A predictor may be *individually* insignificant (large p-value) yet *jointly* significant with others due to correlations. Always use the F-test to assess overall model significance first.

The F-Test for Overall Significance

Tests whether *any* predictor explains y:

$H_0 : \beta_1 = \beta_2 = \cdots = \beta_p = 0$

$$F = \frac{SSR/p}{SSE/(n-p-1)} = \frac{MSR}{MSE} \quad \sim \quad F_{p,\, n-p-1} \text{ under } H_0 \tag{22}$$

Note that $F = \frac{R^2/p}{(1-R^2)/(n-p-1)}$.

Confidence Intervals for Predictions

Two types of intervals for a new point $\mathbf{x}^*$:

$$\hat{y}^* = \mathbf{x}^{*\top} \hat{\boldsymbol{\beta}} \tag{23}$$

Interval	Formula
Confidence interval (for the mean response $\mathbb{E}[y\|\mathbf{x}^*]$)	$\hat{y}^* \pm t_{\alpha/2} \cdot \hat{\sigma}\sqrt{\mathbf{x}^{*\top}(X^\top X)^{-1}\mathbf{x}^*}$
Prediction interval (for a single new observation y^*)	$\hat{y}^* \pm t_{\alpha/2} \cdot \hat{\sigma}\sqrt{1 + \mathbf{x}^{*\top}(X^\top X)^{-1}\mathbf{x}^*}$

Table 4: The prediction interval is always wider than the CI because it accounts for both uncertainty in the mean *and* individual observation noise.

Multicollinearity

Multicollinearity occurs when two or more predictors are highly linearly correlated. It does not violate any Gauss–Markov assumption (OLS is still BLUE), but it has serious *practical* consequences:

- Individual coefficient estimates become **unstable** (high variance): a tiny change in data can flip the sign of $\hat{\beta}_j$.
- **Standard errors** of $\hat{\beta}_j$ inflate (the matrix $(X^\top X)^{-1}$ becomes ill-conditioned).
- **Interpretation** breaks down: it is impossible to isolate the effect of x_j "holding all others constant" when predictors move together.

Detection: Variance Inflation Factor (VIF)

Regress x_j on all other predictors; let R_j^2 be the resulting R^2. The VIF of $\hat{\beta}_j$ is:

$$\text{VIF}_j = \frac{1}{1 - R_j^2} \tag{24}$$

VIF_j	Severity	Action
1	None	No collinearity
1–5	Moderate	Acceptable
5–10	High	Investigate
> 10	Severe	Drop or combine predictors, use Ridge

Table 5: VIF thresholds.

Remedies for Multicollinearity

1. **Drop one of the correlated predictors.**
2. **PCA:** replace correlated predictors with uncorrelated principal components.
3. **Ridge regression:** adds λI to $(X^\top X)$, making the matrix invertible even under perfect collinearity: $\hat{\boldsymbol{\beta}}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$.
4. **Collect more data** to better separate the collinear effects.

Extensions of the Linear Model

Polynomial Regression

When the relationship between x and y is non-linear, add polynomial terms as new features:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \cdots + \beta_d x^d + \varepsilon \tag{25}$$

This is still *linear regression* (linear in the parameters $\boldsymbol{\beta}$) but fits non-linear curves. The design matrix simply includes columns for $x, x^2, \ldots, x^d$.

Degree	Shape	When to use
$d = 1$	Straight line	Linear relationship
$d = 2$	Parabola	One bend (U-shape or hump)
$d = 3$	S-curve	Two bends; growth then plateau
$d > 5$	Wiggly	Usually overfitting; use splines instead

Table 6: Polynomial regression degrees and their uses.

Caution: high-degree polynomials overfit badly. Always choose d by cross-validation, not by maximising R^2.

Interaction Terms

An **interaction** between x_1 and x_2 means the effect of x_1 on y *depends on the value of* x_2:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \underbrace{\beta_3 (x_1 \cdot x_2)}_{\text{interaction term}} + \varepsilon \tag{26}$$

The marginal effect of x_1 is: $\frac{\partial y}{\partial x_1} = \beta_1 + \beta_3 x_2$, which now depends on x_2.

Intuition

Example: Fertiliser (F) and water (W) interact in promoting plant growth. At high water levels, extra fertiliser may not help; at low water levels, fertiliser has a large effect. The model needs an $F \times W$ term to capture this dependency.

Hierarchical principle: if an interaction $x_1 x_2$ is included, always include the main effects x_1 and x_2 as well.

Categorical Predictors (Dummy Coding)

Categorical variables with k levels are encoded as $k - 1$ binary **dummy variables** (one category becomes the reference):

$$\text{Region: North, South, East} \longrightarrow d_{\text{South}} = \begin{cases} 1 & \text{if South} \\ 0 & \text{else} \end{cases}, \quad d_{\text{East}} = \begin{cases} 1 & \text{if East} \\ 0 & \text{else} \end{cases} \tag{27}$$

The coefficient β_{South} is the *expected difference* in y between South and North (the reference group), holding all other predictors constant. Including all k dummies would cause perfect multicollinearity (the **dummy variable trap**).

Weighted Least Squares (WLS)

When the homoscedasticity assumption fails ($\text{Var}[\varepsilon_i] = \sigma^2/w_i$ varies by observation), Weighted Least Squares minimises:

$$\text{WRSS} = \sum_{i=1}^{n} w_i \left(y_i - \mathbf{x}_i^\top \boldsymbol{\beta}\right)^2 = (\mathbf{y} - X\boldsymbol{\beta})^\top W (\mathbf{y} - X\boldsymbol{\beta}) \tag{28}$$

where $W = \text{diag}(w_1, \ldots, w_n)$. The WLS estimator is:

$$\hat{\boldsymbol{\beta}}_{WLS} = (X^\top W X)^{-1} X^\top W \mathbf{y} \tag{29}$$

WLS is BLUE under heteroscedasticity. Typical use: give more weight to more precise (lower-variance) observations.

Feature Selection in MLR

With p predictors, there are 2^p possible subsets. Exhaustive search is infeasible for large p. Common strategies:

Stepwise Selection

Method	Procedure
Forward	Start with intercept only. At each step, add the predictor that most improves the model (lowest p-value / highest $\bar{R}^2$ / lowest AIC). Stop when no improvement.
Backward	Start with all p predictors. At each step, remove the least significant predictor. Stop when all remaining are significant.
Stepwise (bidirectional)	Combine forward and backward: at each step, consider both adding and removing predictors.

Table 7: Stepwise feature selection methods.

Caveats:

- Stepwise methods do not guarantee the globally best subset.
- They inflate Type I errors (multiple testing problem).
- Modern preferred alternatives: cross-validated LASSO (automatic, continuous shrinkage) or best-subset selection via branch-and-bound (feasible up to $p \approx 30$).

LASSO as Continuous Feature Selection

LASSO (Least Absolute Shrinkage and Selection Operator) minimises:

$$\text{RSS}(\boldsymbol{\beta}) + \lambda\|\boldsymbol{\beta}\|_1 = \sum_{i=1}^{n}(y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 + \lambda \sum_{j=1}^{p} |\beta_j| \tag{30}$$

The L1 penalty induces exact zeros at the optimal solution, giving a **sparse** $\hat{\boldsymbol{\beta}}$ and performing feature selection automatically. Choose λ via cross-validation. (See Part 10 for the full treatment of Ridge and Lasso.)

Standardised Coefficients

Raw coefficients $\hat{\beta}_j$ are not directly comparable when predictors are measured on different scales (e.g. income in dollars vs. age in years). **Standardising** both response and predictors to zero mean and unit variance gives **standardised coefficients** (beta weights):

$$\hat{\beta}_j^* = \hat{\beta}_j \cdot \frac{s_{x_j}}{s_y} \tag{31}$$

A one-*standard-deviation* increase in x_j is associated with a $\hat{\beta}_j^*$-standard-deviation change in y. This makes the relative importance of predictors directly comparable.

Multiple Regression vs. Simple Regression Coefficients

Running simple regression of y on x_j gives a *different* coefficient than the MLR coefficient for x_j. The MLR coefficient $\hat{\beta}_j$ adjusts for all other predictors; the simple regression coefficient does not. They agree only when predictors are mutually uncorrelated (orthogonal design).

Practical Checklist for Multiple Regression

1. Check VIFs drop or combine highly correlated predictors.
2. Check residual plots (vs. fitted values, vs. each x_j) for non-linearity or heteroscedasticity.
3. Use Shapiro-Wilk / Q-Q plot to check normality of residuals.
4. Run Durbin-Watson test if data may be time-ordered.
5. Use $\bar{R}^2$, AIC, or BIC (not raw R^2) for model comparison.
6. Use the F-test before reporting individual t-tests.
7. Validate on a held-out test set or via cross-validation.

Part III

The Story of Logistic Regression

Introduction

Imagine you are no longer predicting the price of a house (a continuous number). Instead, you are a doctor trying to predict a *binary category*: Does this patient have a disease? (Yes = 1, No = 0) based on the size of a tumor.

You plot your patients on a graph. Tumor size is on the bottom (x-axis). The y-axis only has two possible levels: 0 (Healthy) and 1 (Sick).

You try to use your trusty Linear Regression line to fit the data. But immediately, things break.

- **The straight line shoots off into infinity.** It will eventually predict a "2.5" or "-1.2" chance of being sick, which makes no sense. Probabilities must live strictly between 0 and 1.
- **Outliers ruin the line.** A patient with an extreme tumor size tilts the line wildly, messing up the predictions for everyone else.

We need a way to bend that rigid straight line into a flexible **S-curve** that flatlines at 0 and flatlines at 1.

Key Idea

Logistic Regression uses the Sigmoid function to squish any raw score into a valid probability between 0 and 1, producing a smooth S-curve that respects the boundaries of binary classification.

Step 1: The Equation (The Sigmoid Squish)

Under the hood, Logistic Regression starts exactly the same way as Linear Regression. It calculates a raw score:

$$z = wx + b \tag{32}$$

But instead of outputting z directly, it feeds z into the **Sigmoid Function** σ:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{33}$$

The Sigmoid function has one simple job: *squish any number in the universe into a probability between 0 and 1.*

- If z is a large positive number (e.g., 100), $\sigma(z) \approx 0.9999$ (99.99% probability).
- If z is a large negative number (e.g., -100), $\sigma(z) \approx 0.0001$ (0.01% probability).
- If $z = 0$, $\sigma(z) = 0.5$ (50% probability).

Your computer's prediction $\hat{y}$ is now a clean, valid percentage.

Step 2: The Decision Boundary

The computer outputs a probability such as 0.85 (85% chance of being sick). But as a doctor, you must make a final binary call: Yes or No.

We establish a **Decision Boundary** (typically at 0.5):

$$\begin{aligned} \hat{y} \geq 0.5 \quad &\Rightarrow \quad \text{Classify as 1 (Sick)} \\ \hat{y} < 0.5 \quad &\Rightarrow \quad \text{Classify as 0 (Healthy)} \end{aligned}$$

Step 3: The Cost Function (Log Loss)

In Linear Regression we used MSE. But applying MSE to the wavy Sigmoid curve produces a *non-convex* loss landscape full of local minima that trap Gradient Descent.

Instead we use **Log Loss** (Binary Cross-Entropy):

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\Big] \tag{34}$$

Intuition

Log Loss evaluates *confidence*.

- Actual: Sick (1), predicted 0.99 — penalty is nearly zero. Good job.
- Actual: Sick (1), predicted 0.51 — moderate penalty. Barely right.
- Actual: Sick (1), predicted 0.01 — penalty shoots to *infinity*.

Log Loss **tortures the algorithm for being confidently wrong**, restoring a clean convex bowl that Gradient Descent can navigate perfectly.

Step 4: Gradient Descent (Learning the Weights)

Identical in spirit to Linear Regression:

1. Calculate the derivative (slope) of the Log Loss.
2. Take a step downhill by tweaking w and b.
3. Recalculate the Log Loss.
4. Repeat until the bottom of the bowl is reached.

When it converges, the algorithm has found the S-curve that *maximises the probability of correctly classifying all patients.*

Step 5: Grading the Model (The New Report Card)

Because we predict categories, not continuous numbers, R^2, F-values, and MSE are irrelevant. We care about *how many patients we misdiagnosed*.

Our report card is built on the **Confusion Matrix**:

Bucket	Abbrev.	Meaning
True Positives	TP	They were sick; we predicted sick. **(Great!)**
True Negatives	TN	They were healthy; we predicted healthy. **(Great!)**
False Positives	FP	They were healthy; we predicted sick. (Type I Error — False Alarm)
False Negatives	FN	They were sick; we predicted healthy. (Type II Error — worst in medicine)

Table 8: The four buckets of the Confusion Matrix.

Accuracy: The Overall Score

Out of all patients, how many did we get right?

$$\text{Accuracy} = \frac{TP + TN}{\text{Total Patients}} \tag{35}$$

The Flaw of Accuracy

If 99 out of 100 patients are healthy, a model that blindly guesses "Healthy" every time achieves 99% Accuracy—yet it completely fails to detect the one sick patient.

Recall (Sensitivity): The Net

Out of all *actually sick* patients, how many did we catch?

$$\text{Recall} = \frac{TP}{TP + FN} \tag{36}$$

In medicine, Recall should be near 100%. Sending a sick person home (False Negative) is catastrophic. To boost Recall, you can lower the decision boundary (e.g., to 0.3), becoming hyper-paranoid.

Precision: The Boy Who Cried Wolf

Out of all patients *we flagged as sick*, how many were actually sick?

$$\text{Precision} = \frac{TP}{TP + FP} \tag{37}$$

Low Precision means too many False Alarms—healthy patients being told they are sick.

The F1-Score: The Balancing Act

Perfect Recall is trivial: just call everyone sick (zero False Negatives, but infinite False Positives). Perfect Precision is trivial: only diagnose obvious cases (zero False Positives, but tons of False Negatives).

The **F1-Score** is the *harmonic mean* of Precision and Recall, forcing a balance:

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{38}$$

If the model blindly skews in one direction, the F1-Score tanks. A high F1-Score (close to 1.0) means you have built a genuinely intelligent classifier.

The ROC Curve and AUC: The Full Picture

Precision, Recall, and F1-Score all depend on a *fixed* decision boundary (e.g. 0.5). But what if we want to evaluate the model across **every possible threshold** at once?

Enter the **ROC Curve** (Receiver Operating Characteristic).

The Two Axes.

- **X-axis — False Positive Rate (FPR):** Out of all *actually healthy* patients, how many did we *wrongly* flag as sick?
$$\text{FPR} = \frac{FP}{FP + TN}$$
- **Y-axis — True Positive Rate (TPR):** Out of all *actually sick* patients, how many did we correctly catch?
$$\text{TPR} = \frac{TP}{TP + FN} \quad (= \text{Recall})$$

How the curve is drawn. As you sweep the decision threshold from 1.0 down to 0.0, the model goes from being hyper-conservative (almost nothing flagged) to hyper-paranoid (everything flagged). At each threshold you get one (FPR, TPR) point. Connecting all those points traces the ROC curve.

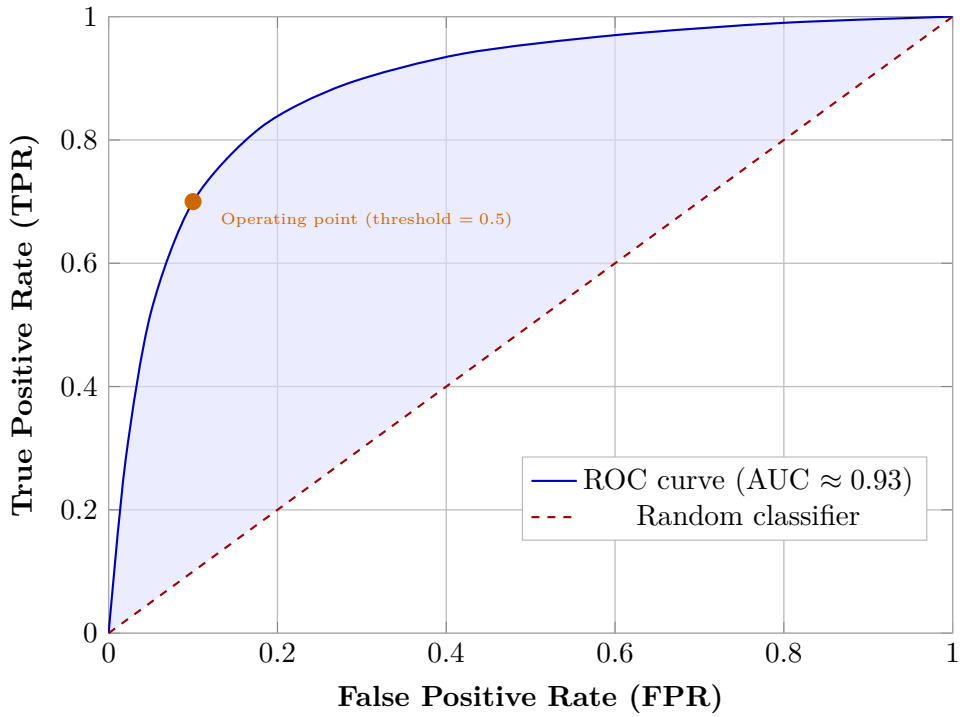

Figure 1: A typical ROC curve for a good binary classifier. The shaded region is the AUC. The dashed diagonal is a random coin-flip model.

Reading the graph.

- The **dashed diagonal** (FPR = TPR) represents a model that is no better than a random coin flip. AUC = 0.5.
- A **perfect model** would jump straight to the top-left corner (TPR = 1, FPR = 0) and hug the left and top walls. AUC = 1.0.
- The **shaded area** between the ROC curve and the diagonal is the **AUC** (Area Under the Curve).

AUC — Area Under the Curve. The AUC collapses the entire ROC curve into a single number:

$$\text{AUC} = \int_0^1 \text{TPR}\Big(\text{FPR}^{-1}(t)\Big)\, dt$$

AUC	Grade	Meaning
1.00	Perfect	Every sick patient ranked above every healthy one.
0.90–0.99	Excellent	Strongly separates classes.
0.80–0.89	Good	Solid real-world performance.
0.70–0.79	Fair	Some discrimination ability.
0.50–0.69	Poor	Barely better than random.
0.50	Useless	Pure coin flip.

Table 9: AUC interpretation guide.

Why AUC beats Accuracy

AUC is **threshold-independent** and **class-imbalance robust**. Even if 99% of patients are healthy, a model that *perfectly ranks* sick patients above healthy ones will score AUC = 1.0, while its Accuracy might look mediocre at certain thresholds.

Part IV

The Story of Decision Trees

Introduction

Forget everything you just learned about straight lines, S-curves, and slopes.

Linear and Logistic Regression are driven by algebra. They assume the world is somewhat smooth and continuous. But sometimes, human decisions are not smooth. Sometimes they are rigid, step-by-step logic gates.

Intuition

Imagine you are a bank manager deciding: *Should we give this person a mortgage?* You do not plug numbers into a formula. Instead, you play a game of 20 Questions.

- Is their credit score above 700?
- If below 700, is their income over $80,000?
- If income is over $80,000, do they have existing debt?

Key Idea

A **Decision Tree** is an algorithm that learns how to chop your data into smaller and smaller groups by asking a sequence of Yes/No questions—and it figures out *which* questions to ask entirely on its own.

Step 1: The Root Node (The Messy Room)

At the very top of the tree is the **Root Node**. All 1,000 of your past loan applicants live here: a chaotic 50/50 mix of *Good* (repaid) and *Bad* (defaulted) loans.

The algorithm's only goal is to **purify this room**. It wants to split it into two **Child Nodes** so that one room is predominantly "Good" loans and the other predominantly "Bad" loans. It repeats this splitting process recursively until the tree decides to stop.

Step 2: The Math of Splitting

The computer tests every feature (Age, Income, Credit Score) at every possible threshold and grades each candidate split with a *purity metric*.

Gini Impurity (Classification)

Gini Impurity measures how mixed a node is. For a node containing K classes, where p_k is the proportion of samples belonging to class k:

$$\text{Gini}(t) = 1 - \sum_{k=1}^{K} p_k^2 \tag{39}$$

- Gini = 0.0: The node is **perfectly pure** (e.g. 100% Good loans). This is the goal.
- Gini = 0.5: The node is **perfectly mixed** (50% Good, 50% Bad). This is the worst case for binary classification.

Example. Suppose a candidate split sends 400 people to the Left node (380 Good, 20 Bad) and 600 people to the Right node (120 Good, 480 Bad).

$$\text{Gini}_{\text{Left}} = 1 - \left(\tfrac{380}{400}\right)^2 - \left(\tfrac{20}{400}\right)^2 = 1 - 0.9025 - 0.0025 = 0.095 \tag{40}$$

$$\text{Gini}_{\text{Right}} = 1 - \left(\tfrac{120}{600}\right)^2 - \left(\tfrac{480}{600}\right)^2 = 1 - 0.04 - 0.64 = 0.32 \tag{41}$$

The **Weighted Gini** of the split is:

$$\text{Gini}_{\text{split}} = \frac{400}{1000} \cdot 0.095 + \frac{600}{1000} \cdot 0.32 = 0.038 + 0.192 = 0.230 \tag{42}$$

The algorithm computes this number for *every* possible question and picks the one that produces the **lowest** weighted Gini. The drop in impurity from the parent is called the **Gini Gain**:

$$\Delta\text{Gini} = \text{Gini}(\text{parent}) - \text{Gini}_{\text{split}} \tag{43}$$

Variance Reduction (Regression)

When the tree predicts a *continuous number* (e.g. house prices), purity is measured by **Variance**—how spread out the values are around the mean:

$$\text{Var}(t) = \frac{1}{n_t} \sum_{i \in t} (y_i - \bar{y}_t)^2 \tag{44}$$

where n_t is the number of samples in node t and $\bar{y}_t$ is their mean value. The algorithm chooses the split that maximises **Variance Reduction**:

$$\Delta V = \text{Var}(\text{parent}) - \left(\frac{n_L}{n} \text{Var}(L) + \frac{n_R}{n} \text{Var}(R) \right) \tag{45}$$

A high ΔV means the split separated a tightly-packed group of *cheap* houses cleanly from a tightly-packed group of *expensive* houses—exactly what we want.

Task	Purity Metric	Best Split Criterion
Classification	Gini Impurity	Minimise weighted Gini
Regression	Variance	Maximise Variance Reduction

Table 10: Splitting criteria by task type.

Step 3: Growing the Tree

The algorithm is recursive. Once it finds the best root split (say, "Credit Score > 700"), it steps into each child node and repeats the search independently.

- **Room A (Left):** 400 people, 95% Good — very pure.
- **Room B (Right):** 600 people, 20% Good — still messy. ⇒ The algorithm asks again: best split is perhaps "Income > $50,000".

This branching continues until a stopping condition is met.

Step 4: Leaves and the Overfitting Trap

The final rooms at the bottom of the tree are called **Leaves**. If you let a Decision Tree run completely unchecked, it will keep splitting until every leaf contains *exactly one person*—making it 100% pure.

The Overfitting Disaster

A leaf with one person has not learned a general rule; it has memorised a specific individual. The tree might encode:

"If you are 32, earn $41,200, have a credit score of 682, and own a red Honda Civic... you will default."

This is textbook **High Variance / Overfitting**: perfect on training data, catastrophic on new data.

Step 5: Pruning (Knowing When to Stop)

To prevent memorisation, we constrain the tree with **Hyperparameters**—settings fixed before training begins:

Hyperparameter	Effect
`max_depth`	Maximum number of question layers allowed. e.g. = 5 means at most 5 Yes/No questions from root to leaf.
`min_samples_split`	A node is only split if it contains at least this many samples. Prevents splitting tiny rooms.
`min_samples_leaf`	Each resulting leaf must contain at least this many samples after a split.

Table 11: Common Decision Tree hyperparameters.

By forcing early stopping, leaves end up slightly impure (e.g. 18 Good, 2 Bad). When a new customer arrives, they fall through the flowchart to a leaf and the tree takes a **Majority Vote**: 18 vs. 2 ⇒ "Approve the loan."

Step 6: The Report Card

Decision Trees are *flexible*: they handle both Classification and Regression, so they borrow the appropriate report card:

- **Classification tree** → Accuracy, Precision, Recall, F1-Score (from Part II).
- **Regression tree** → MSE, MAE, R^2 (from Part I).

The White Box Advantage

Unlike deep neural networks ("Black Boxes" whose reasoning is opaque), Decision Trees are completely **transparent**. A bank can print out the flowchart and explain to a customer, step-by-step and legally, exactly why their application was denied.

From One Tree to a Forest

Single Decision Trees are highly prone to overfitting. In practice we rarely use just one. Instead, we train **hundreds of trees**, each on a random subset of the data and features, and let them *vote*. This ensemble method is called a **Random Forest**, and it is one of the most powerful and widely used algorithms in all of Machine Learning.

Part V

The Story of Random Forests

Introduction

We ended the Decision Tree chapter with a sobering observation: a single tree is dangerously prone to overfitting. Left unconstrained, it memorises its training data to the point of encoding irrelevant quirks of individual examples.

The solution is democracy.

Intuition

Instead of asking one expert for a verdict, you convene a panel of hundreds of experts, each trained on different evidence and examining different clues. No single panelist has seen the full picture — but their collective verdict is remarkably wise and robust. This is a **Random Forest**.

Key Idea

A **Random Forest** is an ensemble of Decision Trees, each trained on a different random bootstrap sample of the data and each considering only a random subset of features at every split. The trees vote (classification) or average (regression) to produce a final prediction. Two deliberate sources of randomness — *bootstrap sampling* and *feature sub-sampling* — ensure the trees are decorrelated, so their errors cancel out rather than reinforcing each other.

What Is It Used For?

- **Credit scoring:** Will this loan applicant default?
- **Medical diagnosis:** Predicting disease from clinical measurements (e.g. diabetes, cancer screening).
- **Feature selection:** The built-in feature importance scores reveal which variables truly matter.
- **Ecology and remote sensing:** Land-cover classification from satellite imagery.
- **General tabular data:** A strong, reliable baseline for virtually any structured dataset.

Step 1: Bootstrap Aggregating (Bagging)

The foundational idea is **Bagging** — short for **Bootstrap Aggregating**, introduced by Leo Breiman in 1996.

The Bootstrap Sample

Given a training set $\mathcal{D}$ of n examples, a **bootstrap sample** $\mathcal{D}^{(b)}$ is constructed by drawing n examples *with replacement* from $\mathcal{D}$.

Because we sample with replacement, on average each bootstrap sample contains approximately 63.2% of the unique original examples, while the remaining $\approx 36.8\%$ are never drawn.

Intuition

The probability that a specific example is *not* selected in any given draw is $\left(1 - \frac{1}{n}\right)^n \to e^{-1} \approx 0.368$ as $n \to \infty$. Those unselected examples form the **Out-of-Bag (OOB)** set for that tree — a free validation set you get at no extra cost.

The Bagging Procedure

1. Draw B bootstrap samples $\mathcal{D}^{(1)}, \ldots, \mathcal{D}^{(B)}$.
2. Train one full Decision Tree $T^{(b)}$ on each sample $\mathcal{D}^{(b)}$.
3. To predict a new point $\mathbf{x}$:
   - **Classification:** Let each tree vote for a class; the class with the most votes wins (majority vote).
   - **Regression:** Average the B tree outputs: $\hat{y} = \frac{1}{B} \sum_{b=1}^{B} T^{(b)}(\mathbf{x})$.

Why Averaging Reduces Variance

Suppose B trees each produce predictions with variance σ^2 and pairwise correlation ρ. The variance of the averaged prediction is:

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T^{(b)}\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2 \tag{46}$$

- As $B \to \infty$, the second term vanishes.
- The irreducible floor is $\rho\,\sigma^2$: the correlation between trees.
- **Lower correlation ρ $\Rightarrow$ lower variance.** This is the key motivation for adding extra randomness in Random Forests.

Step 2: The Extra Ingredient — Random Feature Sub-sampling

Bagging alone helps, but there is a problem: if one feature is a very strong predictor (say, credit score), *every* tree will put that feature at its root. The trees end up highly correlated ($\rho \approx 1$) and bagging delivers little benefit.

Random Forests (Breiman, 2001) add a second layer of randomness: at each node, instead of searching all p features for the best split, the algorithm considers only a random subset of $m_{\text{try}} < p$ features.

The Random Forest Rule of Thumb

$$\text{Classification:} \quad m_{\text{try}} \approx \lfloor \sqrt{p} \rfloor$$
$$\text{Regression:} \quad m_{\text{try}} \approx \lfloor p/3 \rfloor$$

These defaults work remarkably well across diverse real-world datasets.

By randomly blocking the dominant feature from consideration at some nodes, weaker (but still informative) features get their chance. The trees diverge in structure, reducing ρ and therefore reducing the ensemble variance.

Step 3: Out-of-Bag (OOB) Error Estimation

One of the most elegant properties of Random Forests is a built-in, free cross-validation estimate: the **OOB error**.

1. For each training example $(\mathbf{x}_i, y_i)$, identify the subset of trees $\mathcal{B}_i$ for which $(\mathbf{x}_i, y_i)$ was *not* in the bootstrap sample.
2. Aggregate only those trees' predictions for $\mathbf{x}_i$ (a true held-out prediction, since those trees never saw it).
3. The OOB estimate is the average error across all training examples:

$$\varepsilon_{\text{OOB}} = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\left(y_i, \; \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} T^{(b)}(\mathbf{x}_i) \right) \tag{47}$$

Intuition

This is essentially a free cross-validation: you do not need to set aside a validation set or run multiple folds. On large datasets, the OOB error is an almost unbiasedly accurate estimate of generalisation performance.

Step 4: Feature Importance

Random Forests produce a natural ranking of feature usefulness, making them a powerful *feature selection* tool.

Mean Decrease in Impurity (MDI)

At every split across all trees, the algorithm records how much that split reduced the weighted Gini/Variance. Feature j's importance is the total reduction it caused, averaged across all B trees:

$$\text{Importance}_j^{\text{MDI}} = \frac{1}{B} \sum_{b=1}^{B} \sum_{\substack{t \in T^{(b)} \\ \text{split on } j}} \Delta I(t) \tag{48}$$

where $\Delta I(t)$ is the weighted impurity decrease at node t.

Mean Decrease in Accuracy (MDA / Permutation Importance)

A more reliable (but slower) method: for each feature j, randomly *permute* its values in the OOB set and measure how much the OOB accuracy drops. A large drop means the model relied heavily on that feature.

$$\text{Importance}_j^{\text{MDA}} = \varepsilon_{\text{OOB}}^{\text{permuted}_j} - \varepsilon_{\text{OOB}} \tag{49}$$

Method	Pros	Cons
MDI	Fast (computed during training), no extra passes	Biased toward high-cardinality features; can be misleading
MDA	Unbiased, model-agnostic, reflects true predictive power	Requires extra OOB evaluation passes; slower

Table 12: Comparison of feature importance methods in Random Forests.

Step 5: Hyperparameters

Hyperparameter	Effect
`n_estimators` (B)	Number of trees. More trees ⇒ lower variance and more stable OOB estimates, but diminishing returns beyond ∼200–500.
`max_features` (m_{try})	Features considered per split. Smaller ⇒ lower tree correlation, lower variance; larger ⇒ stronger individual trees.
`max_depth`	Maximum depth of each tree. Deeper trees have lower bias; Random Forests typically use fully grown trees (no pruning) because averaging handles the resulting high variance.
`min_samples_leaf`	Minimum samples per leaf. Larger values smooth predictions and speed up training.
`bootstrap`	If `False`, each tree trains on the full dataset (Pasting rather than Bagging). OOB error is unavailable.

Table 13: Key Random Forest hyperparameters.

Tuning Priority

In practice, the most impactful knob is `n_estimators` (more trees is almost always better, up to compute budget). `max_features` is the second most important lever for controlling the bias-variance trade-off.

Step 6: The Report Card

Random Forests inherit the appropriate metrics from the task type:

- **Classification** → Accuracy, Precision, Recall, F1-Score, AUC-ROC (plus OOB error for free).
- **Regression** → MSE, MAE, R^2 (plus OOB MSE for free).

Random Forests vs. Decision Trees: A Comparison

Property	Single Decision Tree	Random Forest
Variance	High (very sensitive to training data)	Low (averaging decorrelated trees)
Bias	Low (fully grown tree)	Slightly higher than a single tree
Overfitting	Severe without pruning	Highly resistant
Interpretability	Fully transparent (printable flowchart)	Black box (but feature importances help)
Training time	Fast	$B\times$ slower (parallelisable)
Prediction time	Very fast	$B\times$ slower
Missing data	Cannot handle natively	Can use surrogate splits
Feature importance	Simple (single tree's splits)	Robust (averaged across all B trees)

Table 14: Decision Tree vs. Random Forest side-by-side.

Strengths and Weaknesses

Strengths: Excellent generalisation with minimal tuning. Handles mixed feature types, missing data (via surrogates), and high-dimensional problems. Provides reliable feature importances. Naturally resistant to overfitting. Training is trivially parallelisable (each tree is independent).

Weaknesses: Less interpretable than a single tree. Slower to predict than a single tree (must query B trees). On very high-dimensional sparse data (e.g. text), gradient-boosted methods often outperform. Cannot extrapolate beyond the range of training values (like all tree-based methods).

Part VI

The Story of Support Vector Machines (SVM)

Introduction

Imagine a battlefield: your troops (**Blue**) are on the left, the enemy (**Red**) on the right. You must build a wall—a **Decision Boundary**—in the middle.

Logistic Regression considers *every* soldier, far and near. **SVM disagrees.** SVM says: *"Only the soldiers on the front lines matter. If we protect them, everyone else is automatically safe."*

Key Idea

An SVM finds the hyperplane that **maximises the margin**—the thickest possible buffer zone between the two classes. Only the **Support Vectors** (the closest data points to the boundary) determine the solution. Delete every other point and the boundary does not move.

Step 1: Geometry of the Margin

A hyperplane in $\mathbb{R}^n$ is defined by:

$$\mathbf{w}^\top \mathbf{x} + b = 0 \tag{50}$$

where $\mathbf{w} \in \mathbb{R}^n$ is the **normal vector** (perpendicular to the boundary) and $b \in \mathbb{R}$ is the **bias**.

We canonically scale $\mathbf{w}$ and b so that the two *margin hyperplanes* (one touching each class's closest points) satisfy:

$$\mathbf{w}^\top \mathbf{x} + b = +1 \quad \text{(positive margin plane, touches Blue support vectors)} \tag{51}$$

$$\mathbf{w}^\top \mathbf{x} + b = -1 \quad \text{(negative margin plane, touches Red support vectors)} \tag{52}$$

The perpendicular distance between these two parallel planes is:

$$\text{Margin} = \frac{2}{\|\mathbf{w}\|} \tag{53}$$

Intuition

To *maximise* the margin $\frac{2}{\|\mathbf{w}\|}$, we equivalently *minimise* $\|\mathbf{w}\|$, or for mathematical convenience, $\frac{1}{2}\|\mathbf{w}\|^2$.

Step 2: The Primal Optimisation Problem

Each training point $(\mathbf{x}_i, y_i)$ with $y_i \in \{-1, +1\}$ must lie on the correct side of its margin plane, giving us the constraint:

$$y_i \left(\mathbf{w}^\top \mathbf{x}_i + b\right) \geq 1 \qquad \forall\, i = 1, \ldots, m \tag{54}$$

The **Hard-Margin SVM primal** is therefore:

$$\boxed{\min_{\mathbf{w}, b} \quad \frac{1}{2}\|\mathbf{w}\|^2 \qquad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1,\ \forall\, i} \tag{55}$$

This is a **convex quadratic programme**: one global minimum, no local traps, guaranteed to be solvable.

Step 3: The Lagrangian and the Dual Problem

To solve the constrained problem (55) we attach a **Lagrange multiplier** $\alpha_i \geq 0$ to each constraint and form the Lagrangian:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{m} \alpha_i \Big[y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1 \Big] \tag{56}$$

KKT Stationarity Conditions

At the optimum, partial derivatives of $\mathcal{L}$ vanish:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \quad \Longrightarrow \quad \mathbf{w}^* = \sum_{i=1}^{m} \alpha_i^* y_i \mathbf{x}_i \tag{57}$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \quad \Longrightarrow \quad \sum_{i=1}^{m} \alpha_i^* y_i = 0 \tag{58}$$

Additionally, the **complementary slackness** condition must hold:

$$\alpha_i^* \Big[y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*) - 1 \Big] = 0 \qquad \forall\, i \tag{59}$$

This means $\alpha_i^* > 0$ *only* for points that sit exactly on the margin planes — precisely the **Support Vectors**. All other points have $\alpha_i^* = 0$ and contribute nothing to $\mathbf{w}^*$.

Derivation of $\mathbf{w}^*$

Substituting Eq. (57) and Eq. (58) back into the Lagrangian (56) and simplifying yields the **Wolfe Dual**:

Maximise over $\boldsymbol{\alpha}$:

$$W(\boldsymbol{\alpha}) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j \, y_i y_j \, (\mathbf{x}_i^\top \mathbf{x}_j) \tag{60}$$

$$\text{subject to:} \quad \sum_{i=1}^{m} \alpha_i y_i = 0, \quad \alpha_i \geq 0$$

Once the dual is solved (via quadratic programming), the optimal weight vector follows directly from Eq. (57):

$$\mathbf{w}^* = \arg\max_{\mathbf{w}} W(\boldsymbol{\alpha}^*) = \sum_{i \in \mathcal{S}} \alpha_i^* \, y_i \, \mathbf{x}_i \tag{61}$$

where $\mathcal{S}$ is the (often tiny) index set of support vectors. The bias is recovered from any support vector $s \in \mathcal{S}$:

$$b^* = y_s - \mathbf{w}^{*\top} \mathbf{x}_s = y_s - \sum_{i \in \mathcal{S}} \alpha_i^* y_i \, (\mathbf{x}_i^\top \mathbf{x}_s) \tag{62}$$

The **decision function** is then:

$$f(\mathbf{x}) = \operatorname{sign}\Big(\mathbf{w}^{*\top} \mathbf{x} + b^*\Big) = \operatorname{sign}\left(\sum_{i \in \mathcal{S}} \alpha_i^* \, y_i \, (\mathbf{x}_i^\top \mathbf{x}) + b^*\right) \tag{63}$$

Step 4: Soft Margin (Allowing Mistakes)

Real data is rarely perfectly separable. We introduce **slack variables** $\xi_i \geq 0$ that measure how far each point violates the margin:

$$\boxed{\min_{\mathbf{w}, b, \boldsymbol{\xi}} \; \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{m} \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \; \xi_i \geq 0, \; \forall \, i} \tag{64}$$

$\xi_i = 0$ means point i is correctly classified outside the margin.
$0 < \xi_i \leq 1$ means it is inside the margin but still on the correct side.
$\xi_i > 1$ means it has crossed the boundary (misclassified).

The Soft-Margin dual becomes:

$$\text{Maximise:} \quad W(\boldsymbol{\alpha}) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i^\top \mathbf{x}_j) \tag{65}$$

$$\text{subject to:} \quad \sum_{i=1}^{m} \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C \tag{66}$$

The only change from the hard-margin dual is the upper bound C on each α_i:

C	Margin	Behaviour
Small ($\to 0$)	Wide	Many violations allowed (High Bias / Underfitting)
Large ($\to \infty$)	Narrow	Few violations (High Variance / Overfitting)

Table 15: Effect of the regularisation hyperparameter C.

Step 5: The Kernel Trick

What if no straight line can separate the classes? The data in its original space $\mathbb{R}^n$ may be completely entangled—no hyperplane can slice it cleanly. The Kernel Trick is the elegant escape: we never actually move the data to a higher-dimensional space; instead, we secretly compute the *geometry* of that space using only a cheap scalar function.

Motivation: Lifting to Higher Dimensions

A **feature map** $\phi : \mathbb{R}^n \to \mathcal{H}$ sends every input $\mathbf{x}$ into a (possibly infinite-dimensional) Hilbert space $\mathcal{H}$ where the data *is* linearly separable.

Intuition

Imagine a 2D battlefield where **Red** troops completely surround **Blue** troops in a circle. No straight line can divide them. Apply the feature map $\phi(x_1, x_2) = \left(x_1,\ x_2,\ x_1^2 + x_2^2\right)$ to lift every point into 3D. In 3D, Blue lives near the *bottom* of a paraboloid and Red lives near the *top*. A flat horizontal plane—a valid linear separator—cuts them apart perfectly. Project that plane back to 2D and it becomes a circle: the true decision boundary.

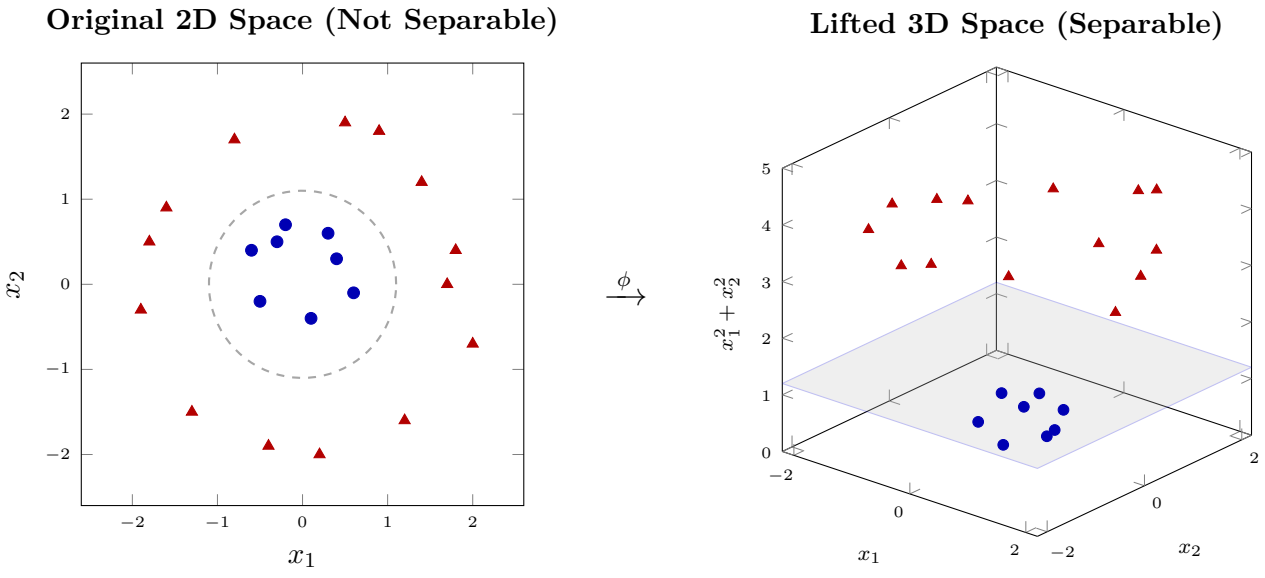

Figure 2: The Kernel Trick in action. Left: Red surrounds Blue in 2D—no line can separate them. Right: Lifting via $\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2)$ places Blue near $z \approx 0$ and Red at $z \approx 3$–5. The grey plane $z = 1.2$ is a valid linear separator. Projected back to 2D it becomes the dashed circle.

Why We Never Compute ϕ Explicitly — The Trick

Inspect the **dual objective** (60) after applying ϕ:

$$W(\boldsymbol{\alpha}) = \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \underbrace{\langle \phi(\mathbf{x}_i),\, \phi(\mathbf{x}_j) \rangle}_{K(\mathbf{x}_i, \mathbf{x}_j)} \tag{67}$$

Only *inner products* of ϕ-images appear—never ϕ itself. If we can find a scalar function K that directly computes $\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$ without ever constructing $\phi(\mathbf{x})$, we get the full power of the high-dimensional lift at the cost of a single scalar multiplication.

The Kernel Trick (One Sentence)

Replace every dot product $\mathbf{x}_i^\top \mathbf{x}_j$ in the SVM dual with a kernel evaluation $K(\mathbf{x}_i, \mathbf{x}_j)$ and the entire algorithm is implicitly operating in the feature space $\mathcal{H}$ *without ever computing* ϕ.

Worked Example: Polynomial Kernel $d = 2$, $c = 0$

Let $\mathbf{x} = (x_1, x_2)^\top$ and $\mathbf{z} = (z_1, z_2)^\top$. Then:

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z})^2 = (x_1 z_1 + x_2 z_2)^2 = x_1^2 z_1^2 + 2 x_1 x_2 z_1 z_2 + x_2^2 z_2^2 \tag{68}$$

This is identical to the inner product in $\mathbb{R}^3$ of the feature vectors:

$$\phi(\mathbf{x}) = \left(x_1^2,\ \sqrt{2}\, x_1 x_2,\ x_2^2 \right)^\top \qquad \Longrightarrow \qquad \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle = K(\mathbf{x}, \mathbf{z}) \tag{69}$$

We computed a 3-dimensional inner product using a *single squaring operation* on a scalar. For degree $d = 100$ this savings is astronomical.

Mercer's Theorem: When Is K Valid?

Not every function qualifies as a kernel. **Mercer's theorem** gives the exact condition:

Mercer's Condition

A symmetric function $K : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$ is a valid kernel (i.e. corresponds to an inner product in some Hilbert space) if and only if the **Gram matrix** $\mathbf{K}$ with entries $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ is **positive semi-definite** for every finite set of inputs.

This means: $\forall\, \mathbf{c} \in \mathbb{R}^m, \quad \mathbf{c}^\top \mathbf{K} \mathbf{c} \geq 0$.

All three standard kernels (Linear, Polynomial, RBF) satisfy Mercer's condition, which guarantees the dual remains convex and has a unique solution.

Standard Kernels

Kernel	Formula	Implicit feature space $\mathcal{H}$	When to use
Linear	$\mathbf{x}_i^\top \mathbf{x}_j$	$\mathbb{R}^n$ (same space)	Linearly separable
Polynomial	$(\mathbf{x}_i^\top \mathbf{x}_j + c)^d$	$\mathbb{R}^{\binom{n+d}{d}}$ (all monomials $\leq d$)	Polynomial boundaries
RBF / Gaussian	$\exp(-\gamma \, \lVert \mathbf{x}_i - \mathbf{x}_j \rVert^2)$	$\mathcal{H}$ is **infinite-dimensional**	General non-linear
Sigmoid	$\tanh(\kappa \, \mathbf{x}_i^\top \mathbf{x}_j + \theta)$	Approximates neural-network layer	Rare; not always valid

Table 16: Common SVM kernels and their implied feature spaces. c, d, γ, κ, θ are hyperparameters.

The RBF Kernel's Secret Infinite-Dimensional Space

Using the Taylor expansion of e^x one can show:

$$\exp\Big(-\gamma \|\mathbf{x} - \mathbf{z}\|^2\Big) = e^{-\gamma\|\mathbf{x}\|^2} \, e^{-\gamma\|\mathbf{z}\|^2} \sum_{k=0}^{\infty} \frac{(2\gamma)^k}{k!} (\mathbf{x}^\top \mathbf{z})^k \tag{70}$$

Each term $(\mathbf{x}^\top \mathbf{z})^k$ corresponds to a polynomial kernel of degree k. Summing to *infinity* means the RBF kernel implicitly works in an **infinite-dimensional** feature space where, theoretically, any smooth decision boundary is representable.

Effect of the γ Hyperparameter (RBF)

γ	Gaussian width	Decision boundary
Large ($\to \infty$)	Narrow bumps	Complex, wiggly — **Overfitting**
Small ($\to 0$)	Wide bumps	Smooth, round — **Underfitting**

Table 17: Effect of γ on the RBF kernel's decision boundary.

Intuition

Large γ: each support vector influences *only its immediate neighbourhood*. The boundary twists and turns to memorise every training point.
Small γ: each support vector influences the *entire space*. The boundary is very smooth but may miss complex structure.

The Kernelised Decision Function

After solving the dual with the chosen kernel, **every** occurrence of a dot product is replaced by a kernel evaluation. The final prediction function is:

$$\boxed{f(\mathbf{x}) = \operatorname{sign}\left(\sum_{i \in \mathcal{S}} \alpha_i^* \, y_i \, K(\mathbf{x}_i, \mathbf{x}) + b^* \right)} \tag{71}$$

where $\mathcal{S}$ is the support-vector index set. This is the complete, kernelised SVM classifier. No explicit ϕ is ever evaluated; K does all the work.

Why the Kernel Trick Is Remarkable

The SVM algorithm—gradient descent, the dual, and the decision function— is **completely unchanged**. Swapping $\mathbf{x}_i^\top \mathbf{x}_j$ for $K(\mathbf{x}_i, \mathbf{x}_j)$ is the *only* modification, yet it silently transports the entire computation to an infinite-dimensional feature space. This is considered one of the most elegant ideas in all of machine learning.

Step 6: The Report Card

As a classifier, SVM inherits the same report card as Logistic Regression: Accuracy, Precision, Recall, F1-Score, and AUC-ROC.

Strengths and Weaknesses

Strengths: Excellent on small/medium, high-dimensional datasets. Mathematically elegant; guaranteed global optimum.
Weaknesses: Computationally expensive on large datasets ($\mathcal{O}(m^2)$ to $\mathcal{O}(m^3)$ for the QP solver). Sensitive to the choice of C and Kernel hyperparameters. Has largely been superseded by Random Forests and gradient-boosted trees (XGBoost) on tabular data.

Part VII

The Story of K-Nearest Neighbours (KNN)

Introduction

Every algorithm we have studied so far has involved *training*: the computer spends time adjusting weights, drawing lines, or building trees before it is ready to make predictions.

KNN throws all of that away.

Key Idea

KNN is a lazy, non-parametric algorithm. It does *zero* training. Instead, it memorises the entire dataset and, when asked to classify a new point, it looks up the K training examples closest to it and lets them vote.

Intuition

Imagine you move to a new neighbourhood and want to know if a house will sell quickly. You do not build a mathematical model. You simply look at the $K = 5$ most similar houses nearby and see what happened to them. If 4 out of 5 sold quickly, you predict your house will too.

Step 1: Distance Metrics (Defining "Closeness")

Before KNN can find the "nearest" neighbours, it must define what *nearest* means. Given two points $\mathbf{x} = (x_1, \ldots, x_n)$ and $\mathbf{z} = (z_1, \ldots, z_n)$ in $\mathbb{R}^n$, the three standard choices are:

Euclidean Distance (L2)

The straight-line distance — the most commonly used metric:

$$d_{\text{Euclidean}}(\mathbf{x}, \mathbf{z}) = \|\mathbf{x} - \mathbf{z}\|_2 = \sqrt{\sum_{i=1}^{n} (x_i - z_i)^2} \tag{72}$$

Manhattan Distance (L1)

The "city block" distance — sum of absolute differences:

$$d_{\text{Manhattan}}(\mathbf{x}, \mathbf{z}) = \|\mathbf{x} - \mathbf{z}\|_1 = \sum_{i=1}^{n} |x_i - z_i| \tag{73}$$

Minkowski Distance (General Case)

Both Euclidean and Manhattan are special cases of the Minkowski metric:

$$d_p(\mathbf{x}, \mathbf{z}) = \|\mathbf{x} - \mathbf{z}\|_p = \left(\sum_{i=1}^{n} |x_i - z_i|^p \right)^{1/p} \tag{74}$$

p	**Name**	**Geometry**
1	Manhattan	Diamond-shaped neighbourhood
2	Euclidean	Circular neighbourhood (most natural)
∞	Chebyshev	Square neighbourhood ($\max_i \lvert x_i - z_i \rvert$)

Table 18: Special cases of the Minkowski distance.

Feature Scaling is Critical

Because KNN uses raw distances, a feature measured in thousands (e.g. house price) will dominate one measured in single digits (e.g. number of bedrooms). Always **standardise** features before applying KNN:

$$x_i' = \frac{x_i - \mu_i}{\sigma_i}$$

Step 2: The Decision Rule

Let $\mathcal{N}_K(\mathbf{x})$ be the set of the K training points closest to query point $\mathbf{x}$.

KNN for Classification: Majority Vote

$$\hat{y} = \arg\max_c \sum_{\mathbf{x}_i \in \mathcal{N}_K(\mathbf{x})} \mathbf{1}[y_i = c] \tag{75}$$

Each neighbour casts one vote for its class c. The query point is assigned the class with the most votes.

KNN for Regression: Local Average

For continuous targets, the prediction is simply the **mean** of the K neighbours' target values:

$$\hat{y} = \frac{1}{K} \sum_{\mathbf{x}_i \in \mathcal{N}_K(\mathbf{x})} y_i \tag{76}$$

A more sophisticated variant uses **distance-weighted** averaging, giving closer neighbours more influence:

$$\hat{y} = \frac{\displaystyle\sum_{\mathbf{x}_i \in \mathcal{N}_K(\mathbf{x})} \frac{y_i}{d(\mathbf{x}, \mathbf{x}_i)}}{\displaystyle\sum_{\mathbf{x}_i \in \mathcal{N}_K(\mathbf{x})} \frac{1}{d(\mathbf{x}, \mathbf{x}_i)}} \tag{77}$$

Step 3: Choosing K (The Bias-Variance Tradeoff)

K is the only hyperparameter in KNN, and it has a huge effect:

K	Behaviour	Risk
$K = 1$	Decided by the single nearest point	High Variance — any outlier is law (Overfitting)
Small K	Local, jagged decision boundary	Sensitive to noise
Large K	Smooth, global boundary biased toward majority class	High Bias (Underfitting)
$K = m$	Always predicts the global majority class	Completely useless

Table 19: Effect of K on bias and variance.

Intuition

The optimal K is found via **cross-validation**: try $K = 1, 3, 5, \ldots$ on a held-out validation set and pick the K that minimises validation error. Odd values of K are preferred for binary classification to avoid tie-breaking.

Theoretical error bound. It can be shown that as $m \to \infty$ with $K \to \infty$ and $K/m \to 0$, the KNN error rate ε_{KNN} satisfies:

$$\varepsilon^* \leq \varepsilon_{\text{KNN}} \leq 2\,\varepsilon^*\,(1 - \varepsilon^*) \quad \leq \quad 2\,\varepsilon^* \tag{78}$$

where ε^* is the irreducible Bayes error rate. In other words, KNN can never do worse than *twice* the theoretically optimal error.

Step 4: The Curse of Dimensionality

KNN's biggest enemy is high-dimensional data. Consider a unit hypercube $[0, 1]^n$. To capture a fraction f of the data volume by a hypercubic neighbourhood, the edge length needed is:

$$\ell(n, f) = f^{1/n} \tag{79}$$

Dimensions n	$f = 1\%$	$f = 5\%$	$f = 10\%$
1	0.01	0.05	0.10
2	0.10	0.22	0.32
10	0.63	0.74	0.79
100	0.955	0.970	0.977

Table 20: Edge length $\ell(n, f)$ required to capture fraction f of data. In 100 dimensions, capturing just 1% of the data requires 95.5% of each axis's range — your "nearest" neighbours are no longer near.

The Curse in Practice

In high dimensions, all points become approximately equidistant from each other. The concept of "nearest neighbour" loses meaning. KNN degrades rapidly beyond ∼20 features without aggressive **dimensionality reduction** (PCA, t-SNE, UMAP).

Step 5: The Report Card

Like Decision Trees, KNN borrows its report card from the task:

- **Classification** → Accuracy, Precision, Recall, F1-Score, AUC-ROC.
- **Regression** → MSE, MAE, R^2.

Computational cost at prediction time. For a dataset of m training points with n features, predicting one query requires computing m distances, each $\mathcal{O}(n)$:

$$\text{Prediction cost} = \mathcal{O}(m \cdot n) \quad \text{(brute force)} \tag{80}$$

This can be improved to $\mathcal{O}(n \log m)$ with spatial data structures such as **KD-Trees** (works well in low dimensions) or **Ball Trees** (better in higher dimensions).

KNN vs. SVM: When to Use Which?

KNN and SVM are both non-linear classifiers that make no parametric assumptions about the data distribution, yet they work in fundamentally opposite ways.

Property	KNN	SVM
Core idea	"Show me your neighbours"	"Find the widest corridor"
Training time	$\mathcal{O}(1)$ — no training at all	$\mathcal{O}(m^2)$–$\mathcal{O}(m^3)$ QP solve
Prediction time	$\mathcal{O}(m \cdot n)$ — slow on large sets	$\mathcal{O}(s \cdot n)$, $s \ll m$ (only support vectors)
Memory	Stores entire dataset	Stores only support vectors
Parametric?	No	No
Key hyperparameter	K (number of neighbours)	C (regularisation), Kernel, γ
Non-linearity	Automatically (local voting adapts)	Via Kernel trick
Handles noise	Poorly (outlier neighbours pollute vote)	Well (soft margin absorbs outliers)
High dimensions	Very poor (curse of dimensionality)	Excellent (especially with RBF kernel)
Large datasets	Slow at inference	Slow at training
Explainability	Intuitive ("these are your neighbours")	Moderate (support vectors, margin)
Probabilistic output?	Via Platt scaling or class proportions	Via Platt scaling

Table 21: Head-to-head comparison of KNN and SVM.

When to Use KNN

- You have a **small to medium dataset** (<50,000 samples) with a **low number of features** (<20).
- The **decision boundary is irregular** and highly local (no clean mathematical shape).
- You need a **quick, zero-training baseline** to compare against more complex models.
- You are doing **recommendation** or **anomaly detection** tasks where similarity search is natural.
- Interpretability matters: you can show the user exactly which training examples drove the prediction.

When to Use SVM

- You have a **small to medium dataset** but with **many features** (text classification, genomics).
- The data is **noisy** and you need a robust, smooth boundary (soft margin handles outliers cleanly).
- You need **guaranteed global optimality** (convex QP).
- The classes are **nearly linearly separable** in some high-dimensional space reachable by a kernel.
- You can afford expensive **hyperparameter search** (grid search over C, γ, kernel type).

The Practical Rule of Thumb

KNN shines when you have *few features, moderate data, and irregular local boundaries.*
SVM shines when you have *many features, noisy data, and need a principled margin.*
For large tabular datasets, **Gradient-Boosted Trees** (XGBoost, LightGBM) tend to outperform both in practice.

Part VIII

The Story of Naive Bayes

Introduction

Every algorithm so far has been *discriminative*: it learns a decision boundary that separates classes directly from data. **Naive Bayes is different — it is generative**. Instead of drawing a dividing line, it asks a more fundamental question: *"How was this data point generated? Which class is most likely responsible for producing it?"*

Naive Bayes builds a separate probabilistic model for each class — what do spam emails look like? what do legitimate emails look like? — and then classifies a new point by asking which model explains it best.

What Is It Used For?

- **Spam filtering:** Is this email spam or not? (The canonical Naive Bayes use case, popularised in the early 2000s.)
- **Text classification:** Sentiment analysis, topic labelling, fake news detection.
- **Medical diagnosis:** Given a set of symptoms (binary features), what disease is most likely?
- **Real-time classification:** Any setting where prediction speed matters more than maximum accuracy.
- **Small data:** When you have very few labelled examples, Naive Bayes is often the only model that does not overfit.

Why Does It Work Despite the "Naive" Assumption?

The independence assumption is almost always false in the real world. The words "buy" and "now" in an email are not independent — they tend to appear together. Yet Naive Bayes often outperforms far more sophisticated models. The reason is **low variance**: because it estimates far fewer parameters, it requires far less data to produce a stable, reliable model.

Key Idea

Naive Bayes applies **Bayes' Theorem** to classify, making one critical simplifying assumption: all features are conditionally independent given the class label. This assumption is almost always wrong — yet the algorithm works surprisingly well because of its extremely low variance.

Intuition

Imagine you are a spam filter trying to classify the email: *"Congratulations! You have won a free prize!"*.
You look up: How often does the word "Congratulations" appear in spam emails? Very often — $P(\text{'Congrats'} \mid \text{Spam}) = 0.8$. How often does "prize" appear in spam? Also very often.
You multiply all these probabilities together for the spam class, then do the same for the not-spam class. Whichever product is larger wins. The multiplication is the "naive" part — you are treating each word as if it were chosen independently.

Step 1: Bayes' Theorem

For a query $\mathbf{x} = (x_1, \ldots, x_n)$ and class label y:

$$\underbrace{P(y \mid \mathbf{x})}_{\text{posterior}} = \frac{\overbrace{P(\mathbf{x} \mid y)}^{\text{likelihood}} \overbrace{P(y)}^{\text{prior}}}{\underbrace{P(\mathbf{x})}_{\text{evidence}}} \tag{81}$$

The denominator $P(\mathbf{x})$ is constant across all classes, so the **MAP classifier** (Maximum A Posteriori) reduces to:

$$\hat{y} = \arg\max_y \; P(y) \cdot P(\mathbf{x} \mid y) \tag{82}$$

Step 2: The Naive Independence Assumption

Computing $P(\mathbf{x} \mid y)$ for a high-dimensional $\mathbf{x}$ is intractable without simplification. Naive Bayes assumes:

$$P(\mathbf{x} \mid y) = \prod_{i=1}^{n} P(x_i \mid y) \tag{83}$$

So the classifier becomes:

$$\hat{y} = \arg\max_y \; P(y) \prod_{i=1}^{n} P(x_i \mid y) \tag{84}$$

In practice we work in log-space to avoid numerical underflow:

$$\hat{y} = \arg\max_y \; \log P(y) + \sum_{i=1}^{n} \log P(x_i \mid y) \tag{85}$$

Step 3: Choosing a Likelihood Model

Gaussian Naive Bayes (Continuous Features)

Assumes each feature is normally distributed within each class:

$$P(x_i \mid y = k) = \frac{1}{\sqrt{2\pi\sigma_{ki}^2}} \exp\left(-\frac{(x_i - \mu_{ki})^2}{2\sigma_{ki}^2}\right) \tag{86}$$

Parameters μ_{ki} and σ_{ki}^2 are estimated by the sample mean and variance of feature i among class-k training examples.

Bernoulli Naive Bayes (Binary Features)

Each feature $x_i \in \{0, 1\}$ (e.g. word present/absent in a document):

$$P(x_i \mid y = k) = p_{ki}^{x_i}(1 - p_{ki})^{1-x_i} \tag{87}$$

Multinomial Naive Bayes (Count Features)

For word counts $x_i \in \mathbb{Z}_{\geq 0}$ (classic text classification):

$$P(\mathbf{x} \mid y = k) \propto \prod_{i=1}^{n} \hat{p}_{ki}^{x_i} \quad \text{where} \quad \hat{p}_{ki} = \frac{\text{count}(x_i,\, y = k)}{\sum_j \text{count}(x_j,\, y = k)} \tag{88}$$

Step 4: Laplace Smoothing

If a word never appeared with class k in training, $P(x_i \mid y = k) = 0$, killing the entire product. **Laplace (additive) smoothing** adds a pseudo-count α (usually 1):

$$\hat{p}_{ki} = \frac{\text{count}(x_i, y = k) + \alpha}{\sum_j \Big[\text{count}(x_j, y = k) + \alpha\Big]} \tag{89}$$

Report Card

- Classification: Accuracy, Precision, Recall, F1, AUC-ROC.
- Training is $\mathcal{O}(mn)$ — just compute means, variances, and class proportions.
- Prediction is $\mathcal{O}(Kn)$ where K = number of classes.

Strengths and Weaknesses

Strengths: Extremely fast, works well on text, handles high dimensions, needs little data, probabilistic output.
Weaknesses: The independence assumption breaks badly when features are correlated. Cannot learn feature interactions.

Part IX

The Story of Gradient Boosting

Introduction

All the algorithms we have studied so far are *single* models. Decision Trees are powerful but overfit. Linear models are robust but underfitted on complex data. What if instead of choosing *one* model, you trained *hundreds of small, weak models* and combined their wisdom?

This is the idea behind **ensemble learning**. There are two main flavours:

- **Bagging (Random Forest):** Train many models *in parallel* on random subsets of the data, then average their predictions. Reduces variance.
- **Boosting (Gradient Boosting):** Train models *sequentially*. Each new model focuses exclusively on correcting the errors of all previous models. Reduces both bias and variance.

Gradient Boosting is the boosting approach, and in the form of **XGBoost**, **LightGBM**, and **CatBoost**, it has dominated machine learning competitions and industry applications for over a decade.

What Is It Used For?

- **Fraud detection:** Is this transaction fraudulent? (Deployed by banks and payment processors worldwide.)
- **Search ranking:** How should web pages be ordered? (Microsoft's LambdaMART, a gradient boosting variant, powers Bing's ranking.)
- **Medical risk scoring:** Predicting patient readmission, disease progression.
- **Click-through rate prediction:** Will a user click on this ad? (Core to all digital advertising platforms.)
- **Kaggle competitions:** XGBoost or LightGBM wins the vast majority of tabular data competitions.

The Core Intuition: A Team of Specialists

Intuition

Imagine you are a doctor trying to diagnose a patient. You make your best guess. A colleague reviews your notes and corrects only the cases you got wrong. A third doctor reviews both sets of notes and corrects what remains. Each expert specialises in the *residual errors* of all previous experts.
After 100 such specialists, the combined diagnosis is extraordinarily accurate — far better than any single doctor could achieve alone. This is Gradient Boosting.

Key Idea

Gradient Boosting builds an ensemble by fitting each new shallow decision tree to the **negative gradient** (pseudo-residuals) of the loss function with respect to the current ensemble's predictions. Each tree is a small correction. The final model is the sum of all these corrections.

Step 1: Forward Stagewise Additive Modelling

We want to find:

$$F^*(\mathbf{x}) = \arg\min_F \mathbb{E}_{(\mathbf{x},y)}\Big[\mathcal{L}(y,\ F(\mathbf{x}))\Big] \tag{90}$$

Instead of optimising over the entire function space at once, we build F greedily, one stage at a time:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, h_m(\mathbf{x}) \tag{91}$$

where $\eta \in (0, 1]$ is the **learning rate** (shrinkage) and h_m is the m-th **weak learner** (typically a shallow tree).

Step 2: The Pseudo-Residuals (The Gradient)

At stage m, we need to know which direction to push the predictions to reduce the loss. The answer is the **negative gradient**:

$$r_{im} = -\left[\frac{\partial\, \mathcal{L}(y_i,\ F(\mathbf{x}_i))}{\partial\, F(\mathbf{x}_i)}\right]_{F=F_{m-1}} \tag{92}$$

These r_{im} are called **pseudo-residuals**. We then fit h_m to predict these residuals:

$$h_m = \arg\min_h \sum_{i=1}^{m}\Big(r_{im} - h(\mathbf{x}_i)\Big)^2 \tag{93}$$

Example: MSE loss. For $\mathcal{L}(y, F) = \frac{1}{2}(y - F)^2$:

$$r_{im} = -\frac{\partial}{\partial F}\frac{1}{2}(y_i - F)^2\Big|_{F=F_{m-1}(\mathbf{x}_i)} = y_i - F_{m-1}(\mathbf{x}_i) \tag{94}$$

With MSE loss, the pseudo-residuals are simply the ordinary residuals.

Example: Log-loss (classification). For $\mathcal{L}(y, F) = -\Big[y \log \sigma(F) + (1 - y) \log(1 - \sigma(F))\Big]$:

$$r_{im} = y_i - \sigma\Big(F_{m-1}(\mathbf{x}_i)\Big) \tag{95}$$

Step 3: The Full Algorithm

1. Initialise $F_0(\mathbf{x}) = \arg\min_{\gamma} \sum_i \mathcal{L}(y_i, \gamma)$ (e.g. the mean of y for MSE).
2. For $m = 1, 2, \ldots, M$:
   a. Compute pseudo-residuals r_{im} for each training point.
   b. Fit a shallow tree h_m to $(\mathbf{x}_i, r_{im})$.
   c. Find the optimal step size: $\gamma_m = \arg\min_{\gamma} \sum_i \mathcal{L}(y_i, F_{m-1}(\mathbf{x}_i) + \gamma h_m(\mathbf{x}_i))$.
   d. Update: $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, \gamma_m h_m(\mathbf{x})$.
3. Output $F_M(\mathbf{x})$.

Step 4: XGBoost Regularisation

XGBoost augments the standard loss with a regularisation term over the tree structure itself:

$$\mathcal{L}^{(m)} = \sum_{i=1}^{n} \mathcal{L}(y_i, \hat{y}_i) + \underbrace{\gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2}_{\Omega(h_m)} \tag{96}$$

where T is the number of leaves, w_j are leaf weights, γ penalises tree complexity (number of leaves), and λ is an L2 penalty on the leaf scores.

Report Card

Gradient Boosting uses the standard task-appropriate report card:

- Regression: MSE, MAE, R^2.
- Classification: Accuracy, Precision, Recall, F1, AUC-ROC.

The State-of-the-Art for Tabular Data

XGBoost, LightGBM, and CatBoost are consistently the top-performing algorithms on structured/tabular ML competitions. **Key hyperparameters:** `n_estimators`, `learning_rate`, `max_depth`, `subsample`, `colsample_bytree`, λ (L2 reg), γ (leaf penalty).

Part X

The Story of Artificial Neural Networks (ANN)

Introduction

Every algorithm we have studied so far is a hand-crafted mathematical model. A human engineer decided the shape of the function (a line, an S-curve, a tree, a margin). ANNs take a fundamentally different approach: **let the data decide the shape**.

A neural network is made up of layers of simple computational units (neurons), each doing nothing more than a weighted sum followed by a non-linear squish. But stack enough of these layers together and the combined system can approximate *any* continuous function — this is the **Universal Approximation Theorem**.

A Brief History

The concept of artificial neurons dates to McCulloch & Pitts (1943). The Perceptron (Rosenblatt, 1958) was the first trainable model. Interest collapsed in the 1970s–80s (the "AI winters") when shallow networks proved limited. The **backpropagation** algorithm (Rumelhart et al., 1986) revived the field. The modern deep learning revolution began in 2012 when AlexNet won the ImageNet competition, cutting the error rate in half using a GPU-trained deep convolutional network.

What Is It Used For?

- **Image recognition:** Self-driving cars, medical imaging (detecting tumours in X-rays), facial recognition.
- **Natural Language Processing:** Translation, text summarisation, chatbots (GPT, Gemini).
- **Speech:** Voice assistants (Siri, Alexa), real-time transcription.
- **Drug discovery:** Predicting protein structures (AlphaFold), molecular property prediction.
- **Game playing:** AlphaGo, AlphaZero — superhuman performance through deep reinforcement learning.
- **Recommendation systems:** Netflix, YouTube, Spotify (what to watch/listen to next).

Types of Neural Networks

Architecture	How it works	Used for
Feedforward (MLP)	Layers connected left-to-right, no cycles	Tabular data, regression, classification
Convolutional (CNN)	Shared filters slide over spatial data	Images, audio, time-series
Recurrent (RNN/LSTM)	Hidden state carries memory across timesteps	Sequential data, text, speech
Transformer	Self-attention over all positions at once	NLP (GPT, BERT), vision, multimodal

Table 22: Major neural network architectures.

Key Idea

An ANN is a **composition of learned non-linear transformations**. Each layer learns to represent the data at a higher level of abstraction: pixels → edges → textures → objects. The power comes from depth — many layers of simple operations compound into extraordinarily rich representations.

Intuition

Imagine teaching a child to recognise a dog. You do not give them a formula. You show them thousands of photos of dogs and non-dogs. Over time, the child's brain adjusts its internal representations until it can reliably distinguish dogs from cats, horses, and chairs.
Training a neural network is the same process — but instead of a brain adjusting synaptic weights, you have gradient descent adjusting numerical parameters.

Step 1: The Neuron

A single artificial neuron computes a **weighted sum** of its inputs plus a bias, then passes the result through a non-linear **activation function** g:

$$a = g(z) = g\left(\sum_{i=1}^{n} w_i x_i + b\right) = g(\mathbf{w}^\top \mathbf{x} + b) \tag{97}$$

Common Activation Functions

Name	Formula	Range	Use
Sigmoid	$\sigma(z) = \dfrac{1}{1+e^{-z}}$	$(0,1)$	Output layer (binary)
Tanh	$\tanh(z) = \dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$(-1,1)$	Hidden layers (older)
ReLU	$\max(0,z)$	$[0,\infty)$	Hidden layers (default)
Leaky ReLU	$\max(\alpha z, z)$, $\alpha \ll 1$	$(-\infty,\infty)$	Avoids dying ReLU
Softmax	$\dfrac{e^{z_k}}{\sum_j e^{z_j}}$	$(0,1)$, sums to 1	Output layer (multi-class)

Table 23: Common activation functions.

Step 2: The Network and Forward Pass

A network with L layers applies transformations sequentially. Let $\mathbf{a}^{[0]} = \mathbf{x}$ (the input). For layer $\ell = 1, \ldots, L$:

$$\mathbf{z}^{[\ell]} = W^{[\ell]}\mathbf{a}^{[\ell-1]} + \mathbf{b}^{[\ell]} \quad \text{(linear step)} \tag{98}$$

$$\mathbf{a}^{[\ell]} = g^{[\ell]}\left(\mathbf{z}^{[\ell]}\right) \quad \text{(activation step)} \tag{99}$$

The final layer's output $\mathbf{a}^{[L]} = \hat{\mathbf{y}}$ is the prediction. This is called the **Forward Pass**.

Step 3: The Loss Function

- **Regression:** MSE $\mathcal{L} = \dfrac{1}{n}\sum_i (y_i - \hat{y}_i)^2$
- **Binary classification:** Binary Cross-Entropy (Log Loss) $\mathcal{L} = -\dfrac{1}{n}\sum_i \Big[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\Big]$
- **Multi-class:** Categorical Cross-Entropy $\mathcal{L} = -\dfrac{1}{n}\sum_i \sum_k y_{ik} \log \hat{y}_{ik}$

Step 4: Backpropagation

To update all weights, we need $\partial\mathcal{L}/\partial W^{[\ell]}$ for every layer. Backpropagation applies the **chain rule** working *backwards* from the output.

Define the **error signal** at layer ℓ as:

$$\boldsymbol{\delta}^{[\ell]} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[\ell]}} \tag{100}$$

The chain rule gives the recursion:

$$\boldsymbol{\delta}^{[L]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[L]}} \odot g'^{[L]}\left(\mathbf{z}^{[L]}\right) \tag{101}$$

$$\boldsymbol{\delta}^{[\ell]} = \left(W^{[\ell+1]\top} \boldsymbol{\delta}^{[\ell+1]}\right) \odot g'^{[\ell]}\left(\mathbf{z}^{[\ell]}\right) \tag{102}$$

where $\odot$ is the element-wise product. The gradients are then:

$$\frac{\partial \mathcal{L}}{\partial W^{[\ell]}} = \boldsymbol{\delta}^{[\ell]}\, \mathbf{a}^{[\ell-1]\top} \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[\ell]}} = \boldsymbol{\delta}^{[\ell]} \tag{103}$$

The weight update rule (gradient descent) is:

$$W^{[\ell]} \leftarrow W^{[\ell]} - \alpha \frac{\partial \mathcal{L}}{\partial W^{[\ell]}} \tag{104}$$

Step 5: The Vanishing Gradient Problem

With many layers, $\boldsymbol{\delta}^{[\ell]}$ is a product of many terms of the form $g'(z)$. For Sigmoid and Tanh, $g'(z) \in (0, 0.25]$, so:

$$\left\|\boldsymbol{\delta}^{[1]}\right\| \approx \prod_{\ell=2}^{L} \left\|W^{[\ell]\top}\right\| \cdot g'(z^{[\ell]}) \xrightarrow{L\to\infty} 0 \tag{105}$$

Gradients vanish exponentially in early layers, so they learn extremely slowly. **ReLU** ($g'(z) = 1$ for $z > 0$) was the breakthrough that made deep networks trainable.

Report Card

Same as the task (regression or classification). In addition:

- Monitor **training vs. validation loss curves** to detect overfitting.
- Use **Dropout** (randomly zeroing activations during training) as the primary regulariser.
- Use **Batch Normalisation** to stabilise and accelerate training.

Part XI

Ridge and Lasso Regression (Regularised Linear Models)

Introduction

Ordinary Least Squares (OLS) is the foundation of linear regression. But OLS has a fatal flaw: it will use *every single feature* it is given, no matter how noisy or irrelevant, and assign it a non-zero weight. When you have many features or highly correlated features, OLS produces weights that are enormous, unstable, and overfit to the specific noise in your training set.

Regularisation is the cure. You add a penalty term to the objective function that punishes large weights, forcing the model to build a simpler, more general explanation of the data.

What Are Ridge and Lasso Used For?

- **High-dimensional data:** Genomics (predicting disease from 20,000 gene expression levels), text (predicting sentiment from 100,000 word features).
- **Multicollinear features:** When features are highly correlated (e.g. house area and number of rooms), OLS weights become wildly unstable. Ridge stabilises them.
- **Feature selection (Lasso):** Automatically identifying which of 500 features actually matter, and setting the rest to exactly zero.
- **Any linear model as a baseline:** Ridge/Lasso are always the first models to try before reaching for anything more complex.

The Core Intuition: Shrinkage

Intuition

Imagine you are fitting a curve to data using a rubber band. OLS lets the curve twist and bend freely to touch every data point (overfitting). Adding regularisation is like *stretching the rubber band*: it still tries to fit the data, but it is pulled toward a simpler shape. The stronger the tension (λ), the simpler the final curve.
Ridge pulls weights toward zero but never quite reaches it. **Lasso** can snap individual weights all the way to zero, effectively removing those features from the model entirely.

OLS minimises:

$$\hat{\mathbf{w}}_{\text{OLS}} = \arg\min_{\mathbf{w}} \|\mathbf{y} - X\mathbf{w}\|_2^2 \quad \Longrightarrow \quad \hat{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y} \tag{106}$$

OLS has two failure modes:

- **Multicollinearity:** If two features are correlated, $X^\top X$ is nearly singular — weights blow up to $\pm\infty$.

- **High Variance / Overfitting:** With many features or limited data, OLS memorises training noise.

The fix: add a **regularisation term** that penalises large weights.

Step 1: Ridge Regression (L2 Penalty)

$$\boxed{\hat{\mathbf{w}}_{\text{Ridge}} = \arg\min_{\mathbf{w}} \underbrace{\|\mathbf{y} - X\mathbf{w}\|_2^2}_{\text{fit}} + \underbrace{\lambda\|\mathbf{w}\|_2^2}_{\text{penalty}}} \tag{107}$$

Closed-Form Solution

Setting the gradient to zero:

$$\begin{aligned} \nabla_{\mathbf{w}}\mathcal{L} &= -2X^\top(\mathbf{y} - X\mathbf{w}) + 2\lambda\mathbf{w} = 0 \\ \implies \quad & \boxed{\hat{\mathbf{w}}_{\text{Ridge}} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}} \end{aligned} \tag{108}$$

Adding λI to $X^\top X$ *always* makes the matrix invertible, curing the multicollinearity problem.

Geometric Interpretation

Ridge constrains $\mathbf{w}$ to lie inside an ℓ_2 ball of radius $\sim 1/\lambda$. The solution is the point where the elliptical OLS contours first touch the sphere. Because a sphere has no corners, Ridge solutions **shrink all weights towards zero** but *never* set any exactly to zero.

Step 2: Lasso Regression (L1 Penalty)

$$\boxed{\hat{\mathbf{w}}_{\text{Lasso}} = \arg\min_{\mathbf{w}} \|\mathbf{y} - X\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_1} \tag{109}$$

Lasso has **no closed-form solution** (the L1 norm is non-differentiable at zero). It is solved by **coordinate descent**: cycle through each w_j and apply the soft-thresholding operator:

$$w_j^* = \text{sign}(\rho_j) \cdot \max(|\rho_j| - \lambda/2,\ 0) \quad \text{where} \quad \rho_j = \mathbf{x}_j^\top(\mathbf{y} - X_{-j}\mathbf{w}_{-j}) \tag{110}$$

Geometric Interpretation

Lasso constrains $\mathbf{w}$ to lie inside an ℓ_1 diamond. Because the diamond has **corners at the axes**, the OLS ellipses are likely to touch a corner first, where one or more $w_j = 0$ exactly. **Lasso performs automatic feature selection.**

Step 3: Elastic Net

Elastic Net combines both penalties:

$$\hat{\mathbf{w}}_{\text{EN}} = \arg\min_{\mathbf{w}} \|\mathbf{y} - X\mathbf{w}\|_2^2 + \lambda_1\|\mathbf{w}\|_1 + \lambda_2\|\mathbf{w}\|_2^2 \tag{111}$$

It inherits Lasso's sparsity and Ridge's stability when features are correlated.

Step 4: The Effect of λ

λ	Bias	Variance
$\lambda = 0$	No regularisation (OLS)	High Variance, may overfit
Small λ	Slight shrinkage	Slightly reduced variance
Large λ	Heavy shrinkage; weights near 0	Low Variance, but High Bias (underfits)
$\lambda \to \infty$	All weights forced to exactly 0	Predicts mean always (useless)

Table 24: Effect of regularisation strength λ.

The optimal λ is chosen via **cross-validation**.

Step 5: Ridge vs. Lasso vs. Elastic Net

Property	**Ridge**	**Lasso**	**Elastic Net**
Penalty	$\lambda\|\mathbf{w}\|_2^2$	$\lambda\|\mathbf{w}\|_1$	Both
Closed form	Yes	No	No
Feature selection	No (shrinks, not zeros)	Yes (exact zeros)	Partial
Correlated features	Handles well	Picks one arbitrarily	Handles well
Best when	All features matter, multicollinearity	Many irrelevant features	High-dimensional, correlated features

Table 25: Comparison of regularised linear models.

Report Card

- Regression: MSE, MAE, R^2.
- Classification (Logistic variants): Accuracy, Precision, Recall, F1.
- Track **validation curve** as λ varies to find the bias-variance sweet spot.

Part XII

The Story of Convolutional Neural Networks (CNN)

Introduction

A plain feedforward network (MLP) treats every pixel of an image as a separate feature. A 224×224 colour image has $224 \times 224 \times 3 = 150{,}528$ input values. If the first hidden layer has just 1,000 neurons, the weight matrix alone has **150 million parameters**. Worse, the network throws away all spatial structure: it has no idea that pixel $(10, 10)$ is next to $(10, 11)$.

Key Idea

A **Convolutional Neural Network (CNN)** solves both problems with two ideas: **local connectivity** (each neuron sees only a small patch of the image) and **weight sharing** (the same filter slides over the entire image). This dramatically reduces parameters and explicitly encodes spatial structure.

What Is It Used For?

- **Image classification:** ImageNet, medical imaging (tumour detection), satellite image analysis.
- **Object detection:** YOLO, Faster R-CNN — identifying and localising multiple objects in a single image.
- **Semantic segmentation:** Labelling every pixel (self-driving cars, surgical robotics).
- **Face recognition:** DeepFace, FaceNet.
- **Video understanding:** Action recognition, video captioning.

Step 1: The Convolution Operation

A **filter** (or kernel) $\mathbf{K}$ of size $k \times k$ slides over the input feature map $\mathbf{X}$ of size $H \times W$. At each position (i, j), it computes an element-wise dot product:

$$S(i,j) = (\mathbf{X} * \mathbf{K})(i,j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \mathbf{X}(i+m,\, j+n) \cdot \mathbf{K}(m,n) \tag{112}$$

The result is a **feature map** (or activation map). With multiple filters, each learns to detect a different pattern (edges, curves, textures, shapes).

Intuition

Think of a filter as a flashlight. It shines on a small patch of the image, computes a "match score," then slides one step right and repeats. A horizontal-edge detector fires strongly wherever the brightness changes from dark to light horizontally. A vertical-edge detector does the same for vertical changes. Stack 64 such filters and you get 64 different views of the same image.

Output Size and Padding

For input size $H \times W$, filter size $k \times k$, padding p, and stride s:

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1 \qquad W_{\text{out}} = \left\lfloor \frac{W + 2p - k}{s} \right\rfloor + 1 \tag{113}$$

Padding	Effect	Output size (stride 1)
$p = 0$ (valid)	Input shrinks	$(H - k + 1) \times (W - k + 1)$
$p = \lfloor k/2 \rfloor$ (same)	Output = Input	$H \times W$

Table 26: Effect of padding on output dimensions.

Step 2: Pooling

After a convolution, a **Pooling layer** reduces spatial dimensions, cutting computation and introducing translation invariance.

Max Pooling

Over a 2×2 region with stride 2:

$$P(i, j) = \max_{0 \le m, n < 2} S(2i + m, 2j + n) \tag{114}$$

Keeps the strongest activation in each patch, discarding exact location.

Average Pooling

$$P(i, j) = \frac{1}{4} \sum_{m=0}^{1} \sum_{n=0}^{1} S(2i + m, 2j + n) \tag{115}$$

Global Average Pooling (GAP)

Modern architectures (ResNet, MobileNet) replace the final large fully-connected layers with **Global Average Pooling**: compute one average per feature map channel, reducing a $7 \times 7 \times 512$ tensor to a single 512-d vector. Drastically reduces parameters and overfitting.

Step 3: A Typical CNN Architecture

Layer	Operation	Intuition
Conv1 (64 filters, 3×3)	Convolution + ReLU	Detects edges and colour blobs
MaxPool (2×2, stride 2)	Halve spatial dims	Translation robustness
Conv2 (128 filters, 3×3)	Convolution + ReLU	Detects textures, corners
MaxPool (2×2, stride 2)	Halve again	Further compression
Flatten / GAP	Vectorise	Bridge to classifier
Dense (1024) + ReLU	Fully connected	High-level combination
Dropout (0.5)	Random zero-out	Regularisation
Dense (K) + Softmax	Output probabilities	Class scores

Table 27: A typical CNN for image classification with K classes.

Step 4: Famous Architectures

Name	Year	Top-5 error	Key innovation
AlexNet	2012	15.3%	Deep + GPU + ReLU + Dropout
VGGNet	2014	7.3%	Very deep, all 3×3 filters
GoogLeNet/Inception	2014	6.7%	Inception modules, multi-scale filters
ResNet-50	2015	3.6%	Residual (skip) connections; 152 layers
EfficientNet	2019	2.4%	Neural Architecture Search; compound scaling
ViT	2020	2.0%	Pure Transformer on image patches

Table 28: ImageNet benchmark milestones.

Residual Connections (ResNet)

The core idea of ResNet: instead of learning $H(\mathbf{x})$, learn the *residual* $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$, so the layer output is:

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$$

The skip connection creates a "highway" for gradients, solving the vanishing gradient problem for very deep networks (100+ layers).

Step 5: Transfer Learning

Training a CNN from scratch on millions of images is expensive. In practice, we use **Transfer Learning**:

1. Take a CNN pre-trained on ImageNet (ResNet, EfficientNet, etc.).
2. **Freeze** the convolutional layers (they already learned universal visual features: edges, textures, shapes).
3. **Replace** the final classification head with a new layer matching your number of classes.
4. **Fine-tune** either just the head, or all layers with a very small learning rate.

Intuition

A model trained on 1.2 million images has already learned to see. When you train it on your 1,000 medical scans, it already knows what an "edge" is. You are just teaching it a new vocabulary on top of universal grammar.

Part XIII

The Story of Recurrent Neural Networks & LSTMs

Introduction

Every network we have studied processes one input independently. But language, audio, and time-series data are fundamentally *sequential*: the meaning of a word depends on what came before it.

Key Idea

A **Recurrent Neural Network (RNN)** maintains a **hidden state** $\mathbf{h}_t$ that acts as memory — it is updated at each timestep using both the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$, allowing the network to carry information across time.

Step 1: The Vanilla RNN

At each timestep t:

$$\mathbf{h}_t = \tanh\Big(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t + \mathbf{b}_h\Big) \tag{116}$$

$$\mathbf{y}_t = W_{hy}\mathbf{h}_t + \mathbf{b}_y \tag{117}$$

The same weight matrices W_{hh}, W_{xh}, W_{hy} are reused at every step — this is **weight sharing across time**.

Backpropagation Through Time (BPTT)

Gradients are computed by unrolling the RNN through all timesteps and applying the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_T} \prod_{k=t+1}^{T} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}} \tag{118}$$

The product of Jacobians causes the **vanishing gradient problem**: for long sequences, gradients shrink exponentially and early timesteps stop learning.

Step 2: The Long Short-Term Memory (LSTM)

The LSTM (Hochreiter & Schmidhuber, 1997) solves the vanishing gradient by introducing a **cell state** $\mathbf{c}_t$ — a dedicated memory channel protected by three **gates**.

Gate	Formula	Purpose
Forget gate $\mathbf{f}_t$	$\sigma(W_f[\mathbf{h}_{t-1};\mathbf{x}_t]+\mathbf{b}_f)$	What fraction of old memory to erase
Input gate $\mathbf{i}_t$	$\sigma(W_i[\mathbf{h}_{t-1};\mathbf{x}_t]+\mathbf{b}_i)$	What fraction of new info to write
Candidate $\tilde{\mathbf{c}}_t$	$\tanh(W_c[\mathbf{h}_{t-1};\mathbf{x}_t]+\mathbf{b}_c)$	New candidate memory content
Output gate $\mathbf{o}_t$	$\sigma(W_o[\mathbf{h}_{t-1};\mathbf{x}_t]+\mathbf{b}_o)$	What fraction of memory to expose

Table 29: The four LSTM gates and their roles.

The cell state and hidden state updates:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \tag{119}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \tag{120}$$

Intuition

Think of $\mathbf{c}_t$ as a conveyor belt. The forget gate decides how much old information to wipe off. The input gate decides what new information to add. The output gate decides how much of the belt's content is readable at this moment. Because the cell state passes through only element-wise multiplications (not repeated matrix multiplications), gradients can flow across hundreds of timesteps without vanishing.

Step 3: GRU (Gated Recurrent Unit)

The GRU (Cho et al., 2014) achieves similar performance to LSTM with fewer parameters by merging the forget and input gates into a single **update gate** $\mathbf{z}_t$:

$$\mathbf{z}_t = \sigma(W_z[\mathbf{h}_{t-1};\mathbf{x}_t]) \tag{121}$$
$$\mathbf{r}_t = \sigma(W_r[\mathbf{h}_{t-1};\mathbf{x}_t]) \quad \text{(reset gate)} \tag{122}$$
$$\tilde{\mathbf{h}}_t = \tanh\Big(W[\mathbf{r}_t \odot \mathbf{h}_{t-1};\mathbf{x}_t]\Big) \tag{123}$$
$$\mathbf{h}_t = (1-\mathbf{z}_t)\odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \tag{124}$$

Property	LSTM	GRU
Gates	3 (forget, input, output)	2 (update, reset)
Parameters	$4(n_h^2 + n_x n_h)$	$3(n_h^2 + n_x n_h)$
Cell state	Separate $\mathbf{c}_t$	No separate cell
When to use	Long sequences, complex tasks	Faster training, less data

Table 30: LSTM vs. GRU comparison.

Step 4: Sequence Architectures

Mode	Structure	Application
One-to-one	Single $\mathbf{x} \to$ single $\mathbf{y}$	Vanilla MLP
One-to-many	Single $\mathbf{x} \to$ sequence	Image captioning
Many-to-one	Sequence $\to$ single $\mathbf{y}$	Sentiment analysis
Many-to-many (sync)	Sequence $\to$ same-length sequence	POS tagging
Many-to-many (Enc-Dec)	Sequence $\to$ different-length sequence	Machine translation

Table 31: RNN input/output configurations.

Part XIV

The Story of Transformers & the Attention Mechanism

Introduction

RNNs process sequences one step at a time. This is slow (no parallelism) and, even with LSTMs, struggles to relate words that are far apart in a long document.

The **Transformer** (Vaswani et al., "Attention Is All You Need," 2017) abolished recurrence entirely. Instead, it lets every token directly attend to every other token in a single parallel operation — the **Attention Mechanism**.

Key Idea

The Transformer replaces sequential processing with **self- attention**: each token simultaneously considers its relationship to all other tokens in the sequence, learning which ones to "pay attention to" when computing its own representation.

Why Transformers Won

- **Parallelism:** All tokens processed simultaneously on GPUs/TPUs. Training is orders of magnitude faster.
- **Long-range dependencies:** No information bottleneck; distance between tokens does not affect gradient flow.
- **Scalability:** Performance keeps improving as model size and data grow (GPT-4, Gemini, Claude have 100B+ parameters).

Step 1: Scaled Dot-Product Attention

For each token, we create three vectors: **Q**uery, **K**ey, and **V**alue.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{125}$$

- $Q \in \mathbb{R}^{n \times d_k}$: what this token is *looking for*.
- $K \in \mathbb{R}^{n \times d_k}$: what each token *offers*.
- $V \in \mathbb{R}^{n \times d_v}$: what each token *actually contains*.
- $\sqrt{d_k}$: scaling to prevent the dot products from growing too large and saturating the softmax.

Intuition

Imagine a library search. Your **Query** is the search term ("machine learning"). Each book has a **Key** (its metadata/title). Dot-product similarity tells you how relevant each book's Key is to your Query. The softmax produces a probability distribution over books. You then retrieve a weighted blend of the **Values** (the actual content). This is exactly what attention does for tokens.

Step 2: Multi-Head Attention

Running attention once captures one type of relationship. Transformers run it h times in parallel with different learned projections:

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V) \tag{126}$$

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,\ldots,\text{head}_h)\,W^O \tag{127}$$

Each head can specialise in a different relationship type (syntactic, semantic, coreference, etc.).

Step 3: The Full Transformer Block

One Transformer encoder block consists of:

1. **Multi-Head Self-Attention** — tokens attend to each other.
2. **Add & Norm** — residual connection + Layer Normalisation.
3. **Feed-Forward Network (FFN)** — two linear layers with a ReLU/GeLU in between, applied independently to each token.
4. **Add & Norm** — second residual.

$$\mathbf{x}' = \text{LayerNorm}\Big(\mathbf{x} + \text{MultiHead}(\mathbf{x},\mathbf{x},\mathbf{x})\Big) \qquad \mathbf{x}'' = \text{LayerNorm}\Big(\mathbf{x}' + \text{FFN}(\mathbf{x}')\Big) \tag{128}$$

Step 4: Positional Encoding

Unlike RNNs, attention has no notion of order. We inject position information by adding a positional encoding to the input embeddings:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \tag{129}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \tag{130}$$

Modern models use **learned** positional embeddings or **Rotary Position Embeddings (RoPE)**.

Step 5: Encoder-Only vs. Decoder-Only vs. Encoder-Decoder

Type	Example Models	Task
Encoder-only	BERT, RoBERTa, DistilBERT	Understanding tasks: classification, NER, QA
Decoder-only	GPT-2/3/4, LLaMA, Gemini, Claude	Text generation, chat, code generation
Encoder-Decoder	T5, BART, mT5	Translation, summarisation, question answering

Table 32: Transformer architecture variants.

The Scale Hypothesis

Unlike previous models, Transformers improve *predictably* with more data, more parameters, and more compute. This "scaling law" (Kaplan et al., 2020) means GPT-4 with $\sim$1 trillion parameters significantly outperforms GPT-3's 175 billion parameters on every benchmark — driving the modern LLM revolution.

Part XV

The Story of Generative AI: GANs, VAEs & Diffusion Models

Introduction

All algorithms so far are *discriminative*: given an input, they output a label or number. **Generative models** do the opposite: they learn the underlying distribution of the data and can *synthesise new examples* — new faces, new text, new music — that are statistically indistinguishable from real data.

Key Idea

A generative model learns $p(\mathbf{x})$ or $p(\mathbf{x}|\mathbf{z})$ from training data. Once trained, it can draw novel samples by sampling a latent code $\mathbf{z}$ from a simple prior (e.g. $\mathcal{N}(\mathbf{0}, I)$) and decoding it into data space.

Generative Adversarial Networks (GANs)

The Idea: A Counterfeiter vs. an Inspector

Intuition

A **Generator** (G) is a counterfeiter that prints fake banknotes. A **Discriminator** (D) is a bank inspector trained to spot fakes. They play a minimax game: the counterfeiter tries to fool the inspector; the inspector tries not to be fooled. Over thousands of rounds, the counterfeiter gets so good that the inspector cannot tell real from fake. *At this point, the generator produces perfect fakes.*

The GAN Objective

$$\min_G \max_D \; \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))] \tag{131}$$

- $G(\mathbf{z})$: Generator maps noise $\mathbf{z}$ to a fake image.
- $D(\mathbf{x})$: Discriminator outputs probability that $\mathbf{x}$ is real.
- D tries to maximise: assign high probability to real data, low to fakes.
- G tries to minimise: make fakes look so real that D assigns them high probability.

Training Challenges

Problem	Description
Mode collapse	Generator finds one convincing output and repeats it forever, ignoring diversity.
Training instability	The G and D loss oscillate without converging.
Gradient vanishing	If D is too good, G's gradients vanish (log saturates).

Table 33: Common GAN training challenges.

Notable GAN Variants

DCGAN — uses CNNs; stable image generation. **Wasserstein GAN (WGAN)** — replaces JS divergence with Earth Mover's distance; much more stable training. **StyleGAN** (NVIDIA) — state-of-the-art face synthesis; generates 1024×1024 photo-realistic faces. **Pix2Pix, CycleGAN** — image-to-image translation (horses to zebras, sketches to photos).

Variational Autoencoders (VAEs)

The Autoencoder Foundation

An **Autoencoder** compresses input $\mathbf{x}$ to a latent code $\mathbf{z}$ (encoder), then reconstructs $\mathbf{x}$ from $\mathbf{z}$ (decoder):

$$\mathbf{z} = E_\phi(\mathbf{x}) \qquad \hat{\mathbf{x}} = D_\theta(\mathbf{z}) \tag{132}$$

Minimises reconstruction loss $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$. The latent space is unstructured — you cannot simply sample from it.

The VAE Fix: A Probabilistic Bottleneck

A VAE (Kingma & Welling, 2013) makes the encoder output a distribution instead of a point: $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi, \boldsymbol{\sigma}^2_\phi)$.

The **Evidence Lower BOund (ELBO)** is maximised:

$$\mathcal{L}_{\text{ELBO}} = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction term}} - \underbrace{D_{\text{KL}}\big(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, p(\mathbf{z})\big)}_{\text{Regularisation term}} \tag{133}$$

The KL term forces the latent space to be smooth and close to $\mathcal{N}(\mathbf{0}, I)$, enabling meaningful interpolation and random sampling.

Intuition

A regular autoencoder is like a disorganised filing cabinet — each file has its own drawer. A VAE is like a well-organised library where similar books are shelved near each other. You can pick a random point on the shelf and find a coherent book.

Diffusion Models

The Core Idea: Learn to Denoise

Diffusion models (Ho et al., 2020) have a different generative strategy:

1. **Forward process:** Gradually add Gaussian noise to a real image over T steps until it becomes pure noise.
2. **Reverse process:** Train a neural network to predict and remove the noise, one step at a time, working backwards from pure noise to a clean image.

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}\Big(\mathbf{x}_t;\ \sqrt{1-\beta_t}\mathbf{x}_{t-1},\ \beta_t\mathbf{I}\Big) \quad \text{(forward, fixed)} \tag{134}$$

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}\Big(\mathbf{x}_{t-1};\ \boldsymbol{\mu}_\theta(\mathbf{x}_t,t),\ \boldsymbol{\Sigma}_\theta(\mathbf{x}_t,t)\Big) \quad \text{(reverse, learned)} \tag{135}$$

The training objective (a simplified form) is:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}}\Big[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t,t)\|^2\Big] \tag{136}$$

Predict the noise $\boldsymbol{\epsilon}$ that was added, then subtract it — simple but remarkably effective.

State-of-the-Art Generative Systems

DALL-E 3 (OpenAI), **Stable Diffusion** (Stability AI), **Midjourney**, and **Imagen** (Google) are all diffusion models conditioned on text embeddings. They generate photo-realistic 1024×1024 images from natural language prompts. **Sora** (OpenAI, 2024) extends this to video generation.

Part XVI

The Story of Reinforcement Learning (RL)

Introduction

All algorithms so far require labelled examples $(\mathbf{x}, y)$. But how did AlphaGo learn to play Go? There are no labelled examples — you cannot label a board position with "this move was perfect."

Reinforcement Learning takes a completely different approach: an **Agent** interacts with an **Environment**, receives **Rewards**, and learns a **Policy** to maximise total cumulative reward through trial and error.

Key Idea

RL is the computational formalisation of learning from consequences. There is no supervisor — only a reward signal that may be delayed, sparse, or noisy. The agent must explore its environment and discover, through experience, which sequences of actions lead to the most reward.

What Is It Used For?

- **Game playing:** AlphaGo/AlphaZero (Go, Chess, Shogi), OpenAI Five (Dota 2), AlphaStar (StarCraft II).
- **Robotics:** Teaching robot arms to grasp objects, locomotion control, surgical automation.
- **LLM alignment:** RLHF (Reinforcement Learning from Human Feedback) — how ChatGPT and Claude are fine-tuned to be helpful and safe.
- **Autonomous driving:** Path planning, lane changing.
- **Resource management:** Chip design (Google AlphaChip), data centre cooling.

Step 1: The MDP Framework

Formally, RL is a **Markov Decision Process (MDP)**:

$$\text{MDP} = (\mathcal{S},\ \mathcal{A},\ P,\ R,\ \gamma) \tag{137}$$

- $\mathcal{S}$: State space (everything the agent can observe).
- $\mathcal{A}$: Action space (everything the agent can do).
- $P(s'|s,a)$: Transition probability — what state s' will the environment enter after the agent takes action a in state s.
- $R(s,a)$: Reward function — immediate scalar reward.

- $\gamma \in [0, 1)$: Discount factor — how much future rewards are worth relative to immediate ones.

The agent's goal is to find a **Policy** $\pi(a|s)$ that maximises the expected cumulative discounted reward:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{138}$$

Step 2: Value Functions

The **State-Value function** $V^\pi(s)$ is the expected return starting from state s and following policy π:

$$V^\pi(s) = \mathbb{E}_\pi\Big[G_t \;\Big|\; S_t = s\Big] \tag{139}$$

The **Action-Value function** $Q^\pi(s, a)$ is the expected return starting from state s, taking action a, then following π:

$$Q^\pi(s, a) = \mathbb{E}_\pi\Big[G_t \;\Big|\; S_t = s,\ A_t = a\Big] \tag{140}$$

The optimal policy selects actions greedily from the optimal Q^*:

$$\pi^*(s) = \arg\max_a Q^*(s, a) \tag{141}$$

Step 3: Q-Learning

Q-Learning is a **model-free, off-policy** algorithm that directly learns $Q^*(s, a)$ using the Bellman optimality equation:

$$Q^*(s, a) = R(s, a) + \gamma \max_{a'} Q^*(s', a') \tag{142}$$

The update rule:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[\underbrace{r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')}_{\text{TD target}} - Q(s_t, a_t) \right] \tag{143}$$

Deep Q-Network (DQN)

DQN (DeepMind, 2015) replaces the Q-table with a deep neural network $Q(s, a; \theta)$ parameterised by weights θ. Two key stabilisation tricks:

- **Experience Replay:** Store $(s_t, a_t, r_{t+1}, s_{t+1})$ transitions in a replay buffer; sample random minibatches for training. Breaks temporal correlations.
- **Target Network:** A separate, slowly-updated copy of the Q-network to compute stable TD targets. Prevents "chasing a moving target."

DQN achieved superhuman performance on 49 Atari games using only raw pixel inputs — a landmark in deep RL.

Step 4: Policy Gradient Methods

Instead of learning a value function and deriving a policy, we directly parameterise the policy $\pi_\theta(a|s)$ and optimise it by gradient ascent on expected return.

The **Policy Gradient Theorem**:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t] \tag{144}$$

Intuition

If an action led to high reward (G_t large), increase its probability ($\nabla_\theta \log \pi_\theta$ is added). If an action led to low reward, decrease its probability. This is exactly how organisms learn from positive and negative reinforcement.

Algorithm	Key Idea	Used In
REINFORCE	Monte Carlo policy gradient	Basic tasks
Actor-Critic (A2C/A3C)	Policy (actor) + Value (critic)	Robotics
PPO (Proximal Policy Optimisation)	Clip update ratio for stability	ChatGPT, LLMs
SAC (Soft Actor-Critic)	Entropy regularisation for exploration	Robotics, locomotion
AlphaZero	MCTS + Policy/Value network	Chess, Go, Shogi

Table 34: Key RL algorithms and their applications.

Part XVII

Dimensionality Reduction: PCA, t-SNE, and UMAP

Introduction

Real datasets often have hundreds or thousands of features. Visualising such data is impossible — humans perceive in 2D or 3D. High dimensions also cause the curse of dimensionality, hurt KNN and clustering, and slow down every algorithm.

Dimensionality Reduction compresses data into fewer dimensions while preserving as much structure as possible.

Principal Component Analysis (PCA)

Key Idea

PCA finds the directions (principal components) of maximum variance in the data and projects the data onto the top k of these directions. It is a **linear** method: the lower-dimensional representation is a rotation and compression of the original feature space.

Algorithm

1. **Centre the data:** $\tilde{X} = X - \bar{X}$.
2. **Compute the covariance matrix:** $\Sigma = \frac{1}{n-1}\tilde{X}^\top \tilde{X}$.
3. **Eigendecompose:** $\Sigma = Q\Lambda Q^\top$.
4. **Sort** eigenvectors by decreasing eigenvalue.
5. **Project:** $Z = \tilde{X}Q_k$ where Q_k contains the top k eigenvectors (principal components).

In practice, we use **Singular Value Decomposition (SVD)**: $\tilde{X} = U\Sigma V^\top$; the columns of V are the principal components.

Explained Variance Ratio

The fraction of total variance explained by principal component i:

$$\text{EVR}_i = \frac{\lambda_i}{\sum_j \lambda_j} \tag{145}$$

Choose k such that cumulative EVR $\geq 95\%$ (or plot a scree plot).

Limitations of PCA

PCA only captures **linear** structure. If the data lies on a twisted, non-linear manifold (e.g. the Swiss roll), PCA fails to "unroll" it. Non-linear methods (t-SNE, UMAP) are needed.

t-SNE (t-Distributed Stochastic Neighbour Embedding)

t-SNE (van der Maaten & Hinton, 2008) is a **non-linear** visualisation method designed to reveal cluster structure.

The Algorithm

1. **High-D similarities:** Model pairwise similarities in the original space using a Gaussian:

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2/2\sigma_i^2)}{\sum_{k\neq i}\exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2/2\sigma_i^2)}$$

2. **Low-D similarities:** Model pairwise similarities in 2D using a **heavy-tailed** Student-t distribution ($\nu = 1$):

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k\neq l}(1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}$$

3. **Minimise KL divergence:** $\mathcal{L} = D_{\mathrm{KL}}(P\|Q) = \sum_{ij} p_{ij} \log \frac{p_{ij}}{q_{ij}}$ using gradient descent.

Intuition

The Student-t distribution has *heavier tails* than the Gaussian. This means t-SNE penalises placing far-apart high-D points close together in 2D very harshly, while moderately tolerating nearby points being spread out. The result: clusters compress tightly and separate clearly.

Property	Pros	Cons
t-SNE	Excellent cluster visualisation	Non-deterministic, slow $\mathcal{O}(n^2)$, cannot project new points

UMAP (Uniform Manifold Approximation and Projection)

UMAP (McInnes et al., 2018) uses topological/manifold theory to achieve similar or better results than t-SNE but much faster:

- **Speed:** $\mathcal{O}(n)$ complexity vs. t-SNE's $\mathcal{O}(n^2)$. Handles millions of points.
- **Global structure:** Preserves both local cluster structure and global distances between clusters better than t-SNE.
- **Transforms new points:** Can project unseen data, enabling it to be used as a preprocessing step.

Method	**Type**	**New points?**	**Speed**
PCA	Linear	Yes	Very fast
t-SNE	Non-linear	No	Slow
UMAP	Non-linear	Yes	Fast
Autoencoders	Non-linear (learned)	Yes	Moderate

Table 35: Dimensionality reduction method comparison.

Part XVIII

Model Evaluation: Cross-Validation & the Bias-Variance Tradeoff

Introduction

Training accuracy is a lie. A model that memorises 1,000 training examples achieves 100% training accuracy and is completely useless. We need rigorous evaluation procedures to estimate how well a model *generalises* to data it has never seen.

Step 1: The Bias-Variance Decomposition

The expected test error of any model can be decomposed as:

$$\mathbb{E}\Big[(y - \hat{f}(\mathbf{x}))^2\Big] = \underbrace{\Big(\text{Bias}[\hat{f}(\mathbf{x})]\Big)^2}_{\text{Bias}^2} + \underbrace{\text{Var}[\hat{f}(\mathbf{x})]}_{\text{Variance}} + \underbrace{\sigma_\epsilon^2}_{\text{Irreducible noise}} \tag{146}$$

Component	Meaning	Caused by
Bias^2	Error from wrong assumptions (systematic)	Model too simple (underfitting)
Variance	Error from sensitivity to training data fluctuations	Model too complex (overfitting)
Irreducible	Noise in the data itself	Cannot be reduced

Table 36: The bias-variance decomposition.

The Bias-Variance Tradeoff

Simpler models (linear regression, shallow trees) have high bias and low variance — they underfit but are consistent across datasets. **Complex models** (deep networks, deep trees) have low bias but high variance — they fit training data well but are fragile to new data. Regularisation, ensembling, and cross-validation are all strategies to find the sweet spot.

Step 2: Cross-Validation Strategies

Hold-Out Split

Partition data once: typically 70% train, 15% validation, 15% test.

- **Pros:** Simple, fast.
- **Cons:** High variance in the estimate; depends heavily on which random split was chosen.

k-Fold Cross-Validation

1. Divide data into k equal folds.
2. For each fold $i = 1, \ldots, k$: use fold i as validation, train on the other $k-1$ folds.
3. Final score: average of k validation scores.

$$\hat{\varepsilon}_{\text{CV}} = \frac{1}{k} \sum_{i=1}^{k} \varepsilon_i \tag{147}$$

Standard choice: $k = 5$ or $k = 10$, balancing bias and variance of the estimate.

Stratified k-Fold

For classification with imbalanced classes, ensure each fold contains approximately the same class proportions as the overall dataset.

Leave-One-Out CV (LOOCV)

$k = n$ (each training example is its own validation set). Gives an approximately unbiased estimate but is computationally expensive ($\mathcal{O}(n \cdot$ training time)).

Time-Series Cross-Validation

Never use future data to predict the past. Use **expanding window** or **rolling window** validation where the train set always precedes the validation set in time.

Step 3: Hyperparameter Tuning

Method	How	When to use
Grid Search	Exhaustive search over all combinations	Few hyperparameters
Random Search	Random samples from hyperparameter space	Many hyperparameters
Bayesian Optimisation	Fit a surrogate model (GP); explore where improvement is likely	Expensive evaluations
Automated ML (AutoML)	Search over architecture + hyperparameters jointly	Low human expertise

Table 37: Hyperparameter optimisation strategies.

The Test Set Is Sacred

Use the test set **exactly once**, at the very end, after all model selection and hyperparameter tuning is complete. Every time you look at the test set to make a decision, you are indirectly fitting to it — an **information leak** that produces an overly optimistic estimate of generalisation.

Step 4: Learning Curves

A **learning curve** plots training and validation error as a function of training set size:

Pattern	Meaning	Fix
Both errors high	High bias (underfitting)	More complex model
Train low, val high	High variance (overfitting)	More data, regularisation, simpler model
Both errors plateau	Near-optimal; irreducible noise dominates	Better features or data quality

Table 38: Diagnosing model behaviour from learning curves.

Part XIX

AI Ethics, Fairness, and Explainability

Introduction

A model that achieves 95% accuracy is not necessarily a *good* model. If it achieves 95% overall by performing at 80% for one demographic group and 99% for another, it is a **biased** model. If no human can understand *why* it makes a decision, it is an **opaque** model. Deploying such a model in healthcare, criminal justice, or lending can cause serious, measurable harm.

Key Idea

AI Ethics is not optional. It is a core engineering discipline. Building fair, transparent, and accountable AI systems is both a moral imperative and, increasingly, a legal requirement (EU AI Act, GDPR).

Sources of Bias

Bias Type	Description	Example
Historical bias	Training data reflects past injustice	Loan approval data biased against minorities
Representation bias	Underrepresented groups in training data	Face recognition fails on darker skin tones
Measurement bias	Proxy variables encode protected attributes	Using ZIP code as a proxy for race
Feedback loop bias	Predictions influence future training data	Predictive policing concentrates arrests
Aggregation bias	One model for diverse subpopulations	Single diabetes model for all ethnicities

Table 39: Common sources of algorithmic bias.

Fairness Metrics

Different mathematical definitions of fairness are often *mutually incompatible* (the impossibility theorem):

Criterion	Definition
Demographic Parity	$P(\hat{y}=1\|A=0) = P(\hat{y}=1\|A=1)$ Equal positive prediction rate across groups
Equalised Odds	Equal TPR *and* FPR across groups
Equal Opportunity	Equal TPR (Recall) across groups
Calibration	$P(y=1\|\hat{p}=p, A=0) = P(y=1\|\hat{p}=p, A=1)$ Predicted probabilities are equally reliable across groups

Table 40: Common fairness criteria.

The Fairness Impossibility Theorem

It is mathematically impossible to simultaneously satisfy demographic parity, equalised odds, and calibration — except in trivial cases. The appropriate fairness criterion must be chosen based on the **context and stakes** of the application.

Explainability: SHAP and LIME

"Black box" models (deep networks, gradient-boosted trees with hundreds of trees) do not inherently explain their decisions. Two widely used tools bridge this gap:

SHAP (SHapley Additive exPlanations)

SHAP values come from cooperative game theory. They answer: *"How much did each feature contribute to this specific prediction, compared to the average prediction?"*

For a prediction $f(\mathbf{x})$, the SHAP value ϕ_i for feature i is the average marginal contribution of feature i across all possible feature orderings (coalitions):

$$\phi_i = \sum_{S \subseteq \mathcal{F} \setminus \{i\}} \frac{|S|!\,(|\mathcal{F}| - |S| - 1)!}{|\mathcal{F}|!} \Big[f(S \cup \{i\}) - f(S) \Big] \tag{148}$$

SHAP satisfies desirable axioms: **efficiency** ($\sum_i \phi_i = f(\mathbf{x}) - \mathbb{E}[f]$), **symmetry**, **dummy**, and **additivity**. TreeSHAP computes exact SHAP values for tree-based models in $\mathcal{O}(TL^2)$ time.

LIME (Local Interpretable Model-Agnostic Explanations)

LIME (Ribeiro et al., 2016) explains *any* black-box model by fitting a simple interpretable model locally:

1. Perturb the input $\mathbf{x}$ to generate nearby points $\mathbf{x}' \in \mathcal{Z}$.
2. Query the black-box model for predictions on all perturbed points.
3. Fit a simple linear model to these predictions, weighting by proximity to $\mathbf{x}$:

$$\xi(\mathbf{x}) = \arg\min_{g \in \mathcal{G}} \mathcal{L}(f, g, \pi_{\mathbf{x}}) + \Omega(g)$$

4. The linear model's coefficients are the explanation.

Property	SHAP	LIME
Scope	Global + local	Local only
Consistency	Theoretically grounded (Shapley axioms)	Heuristic
Speed	Fast for trees (TreeSHAP), slow for deep models	Fast
Stability	Consistent	Can vary between runs

Table 41: SHAP vs. LIME comparison.

Responsible AI Principles

1. **Fairness:** Actively audit and mitigate demographic disparities in model performance.
2. **Transparency:** Document model limitations, training data, and intended use cases (model cards).
3. **Accountability:** Maintain human oversight for high-stakes decisions. Automate cautiously.
4. **Privacy:** Apply differential privacy, federated learning, or data anonymisation to protect individuals.
5. **Safety:** Red-team models for adversarial inputs. Use RLHF and constitutional AI to align LLMs to human values.
6. **Robustness:** Test for distribution shift, adversarial examples, and edge cases before deployment.

The Future of AI

We stand at an extraordinary inflection point. Large Language Models can write code, pass bar exams, and assist in drug discovery. Diffusion models generate photo-realistic images and videos from text. RL agents defeat world champions at games.
The most pressing frontier is not capability but alignment: ensuring that increasingly powerful AI systems reliably do what humans intend, are equitable across diverse populations, are honest about their uncertainty, and remain under meaningful human control.
This is the defining technical and ethical challenge of our era.

Part XX

Statistical Hypothesis Testing

Introduction: The Framework

Every ML model eventually asks a statistical question: *Is this result real, or could it have happened by chance?* Hypothesis testing provides the rigorous mathematical answer.

Key Idea

A **hypothesis test** starts with a **null hypothesis** H_0 ("nothing interesting is happening") and an **alternative hypothesis** H_1 ("something real is going on"). We compute a **test statistic** from the data, find its **p-value** (probability of getting a result this extreme *if* H_0 *were true*), and reject H_0 if the p-value falls below a pre-set significance threshold α (usually 0.05).

Type I and Type II Errors

	H_0 **True**	H_0 **False**
Reject H_0	**Type I Error** (α) — False Positive	**Correct** (Power $= 1-\beta$)
Fail to reject H_0	**Correct** ($1-\alpha$)	**Type II Error** (β) — False Negative

Table 42: The four outcomes of a hypothesis test.

- **Significance level** α: Maximum tolerated probability of a Type I Error (false alarm). Conventionally $\alpha = 0.05$.
- **Power** $= 1 - \beta$: Probability of correctly detecting a real effect. Aim for ≥ 0.80.
- **p-value**: *Not* the probability that H_0 is true — it is the probability of observing data at least as extreme as ours, *given that* H_0 *is true*.

One-tailed vs. Two-tailed Tests

A **two-tailed** test asks: "Is the effect non-zero in either direction?" ($H_1 : \mu \neq \mu_0$). A **one-tailed** test asks a directional question ($H_1 : \mu > \mu_0$ or $H_1 : \mu < \mu_0$). Use two-tailed tests by default unless the direction is specified in advance by theory.

Part A: Parametric Tests

The z-Test (Known Variance / Large Samples)

When to use: Population variance σ^2 is known, *or* sample size $n \geq 30$ (Central Limit Theorem kicks in).

Hypotheses: $H_0 : \mu = \mu_0$ vs. $H_1 : \mu \neq \mu_0$.

$$z = \frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}} \quad \sim \quad \mathcal{N}(0,1) \text{ under } H_0 \tag{149}$$

Reject H_0 if $|z| > z_{\alpha/2}$ (e.g. $z_{0.025} = 1.96$ for $\alpha = 0.05$).

Variant	Use case
One-sample z	Is the mean of a population equal to a reference value?
Two-sample z	Do two large independent groups have the same mean?
Proportion z	Is an observed proportion equal to a reference proportion? (e.g. click-through rate A/B test)

Table 43: z-test variants.

The t-Test (Unknown Variance / Small Samples)

When σ^2 is unknown and n is small, replacing σ with the sample standard deviation s introduces extra uncertainty. The test statistic follows a **Student's t-distribution** with heavier tails than the normal.

117.2.1 One-Sample t-Test

When: Is the mean of one group equal to a known value μ_0?

$$t = \frac{\bar{X} - \mu_0}{s/\sqrt{n}} \quad \sim \quad t_{n-1} \text{ under } H_0 \tag{150}$$

Example: Does our new drug lower blood pressure by exactly 10 mmHg on average?

117.2.2 Two-Sample Independent t-Test (Welch's)

When: Comparing means of *two independent* groups (do not assume equal variances — use Welch's correction).

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} \quad \sim \quad t_\nu \text{ under } H_0 \tag{151}$$

where the Welch-Satterthwaite degrees of freedom are:

$$\nu = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{(s_1^2/n_1)^2}{n_1 - 1} + \frac{(s_2^2/n_2)^2}{n_2 - 1}} \tag{152}$$

Example: Does training group A outperform training group B? Does model A have significantly higher accuracy than model B on *different* test sets?

117.2.3 Paired t-Test

When: Two measurements on the *same* subjects (before/after, or two models on the *same* test examples). Compute differences $d_i = X_{1i} - X_{2i}$ and run a one-sample t-test on d_i:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} \quad \sim \quad t_{n-1} \tag{153}$$

Example: Is Model A significantly better than Model B on the same 1,000 test images (matched pairs)?

t-Tests in Linear Regression

The individual coefficient p-values you see in a regression summary *are* t-tests. For coefficient $\hat{w}_j$:

$$t_j = \frac{\hat{w}_j}{\text{SE}(\hat{w}_j)} \quad \sim \quad t_{n-p-1} \text{ under } H_0 : w_j = 0$$

where $\text{SE}(\hat{w}_j) = \sqrt{\hat{\sigma}^2 (X^\top X)^{-1}_{jj}}$. A small p-value means feature j is a statistically significant predictor *given all other features in the model.*

F-Test and ANOVA

117.3.1 F-Test for Overall Regression

When: Is the regression model as a whole statistically significant (do *any* features predict y)?

$$F = \frac{\text{Explained Variance}/p}{\text{Residual Variance}/(n-p-1)} = \frac{MSR}{MSE} \quad \sim \quad F_{p,\, n-p-1} \text{ under } H_0 \tag{154}$$

Source	Sum of Squares	Degrees of Freedom
Regression (Model)	$SSR = \sum_i (\hat{y}_i - \bar{y})^2$	p (number of predictors)
Residuals (Error)	$SSE = \sum_i (y_i - \hat{y}_i)^2$	$n - p - 1$
Total	$SST = \sum_i (y_i - \bar{y})^2$	$n - 1$

Table 44: ANOVA table for linear regression.

Note that $R^2 = SSR/SST$ and $F = \frac{R^2/p}{(1-R^2)/(n-p-1)}$.

117.3.2 One-Way ANOVA

When: Comparing means across **3 or more independent groups** (running multiple t-tests inflates Type I Error).

$H_0 : \mu_1 = \mu_2 = \cdots = \mu_k$ vs. H_1: at least one $\mu_i \neq \mu_j$.

$$F = \frac{SS_{\text{Between}}/(k-1)}{SS_{\text{Within}}/(N-k)} \quad \sim \quad F_{k-1,\, N-k} \tag{155}$$

where k is the number of groups and N is total sample size.

- $SS_{\text{Between}} = \sum_i n_i(\bar{X}_i - \bar{X})^2$: variance due to group membership.
- $SS_{\text{Within}} = \sum_i \sum_j (X_{ij} - \bar{X}_i)^2$: variance within each group (residual noise).

A significant F-test tells you *some* groups differ; **post-hoc tests** (Tukey HSD, Bonferroni, Scheffé) tell you *which* pairs.

Intuition

One-way ANOVA is used in ML to compare multiple algorithms across the same dataset. E.g.: *"Are the mean accuracies of SVM, Random Forest, and XGBoost significantly different from each other?"*

117.3.3 Two-Way ANOVA

When: Two categorical independent variables (factors) and their *interaction* affect a continuous outcome.

$$y_{ijk} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \varepsilon_{ijk} \tag{156}$$

Separate F-tests for: main effect of Factor A, main effect of Factor B, and the A×B interaction term.

Example in ML: How does accuracy vary across (Algorithm) × (Dataset) combinations? Is there a significant interaction (i.e. does XGBoost only beat SVM on *some* datasets)?

Part B: Tests for Regression Assumptions

Linear regression makes four assumptions (LINE). These tests check each one:

Assumption	What it means	Test
Linearity	$\mathbb{E}[y\|\mathbf{x}]$ is linear in $\mathbf{x}$	Residual vs. fitted plot; RESET test
Independence	Residuals are uncorrelated	Durbin-Watson test
Normality	Residuals $\sim \mathcal{N}(0, \sigma^2)$	Shapiro-Wilk; Q-Q plot
Equal variance	Homoscedasticity (constant σ^2)	Levene's test; Breusch-Pagan test

Table 45: The LINE assumptions and their diagnostic tests.

Shapiro-Wilk Test (Normality)

H_0**:** The sample comes from a normal distribution.

The test statistic W compares the sample order statistics to what would be expected from a normal distribution:

$$W = \frac{\left(\sum_{i=1}^{n} a_i x_{(i)}\right)^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2} \tag{157}$$

W close to 1 $\Rightarrow$ normal. Small p-value $\Rightarrow$ reject normality.

When to use: Check residuals of a linear regression. Small samples ($n < 50$): Shapiro-Wilk; larger samples: D'Agostino-Pearson. Always supplement with a Q-Q plot.

Levene's Test (Homoscedasticity)

H_0**:** All groups have equal variance ($\sigma_1^2 = \sigma_2^2 = \cdots = \sigma_k^2$).

$$W_L = \frac{(N-k)}{(k-1)} \cdot \frac{\sum_{i=1}^{k} n_i(\bar{Z}_i - \bar{Z})^2}{\sum_{i=1}^{k}\sum_{j=1}^{n_i}(Z_{ij} - \bar{Z}_i)^2} \quad \sim \quad F_{k-1,\, N-k} \tag{158}$$

where $Z_{ij} = |X_{ij} - \bar{X}_i|$ (absolute deviations from group mean). More robust than Bartlett's test when normality may fail.

Durbin-Watson Test (Autocorrelation)

H_0**:** No autocorrelation in residuals ($\rho = 0$).

$$DW = \frac{\sum_{t=2}^{n}(e_t - e_{t-1})^2}{\sum_{t=1}^{n} e_t^2} \quad \in [0, 4] \tag{159}$$

- $DW \approx 2$: No autocorrelation (desirable).
- $DW < 2$: Positive autocorrelation (common in time-series).
- $DW > 2$: Negative autocorrelation.

Rule of thumb: Concern if $DW < 1.5$ or $DW > 2.5$.

Breusch-Pagan Test (Heteroscedasticity)

H_0**:** Variance of residuals is constant (homoscedastic).

1. Fit the original regression; obtain residuals e_i.
2. Regress e_i^2 on the original predictors $\mathbf{x}_i$.
3. Test statistic: $BP = n \cdot R^2_{\text{aux}} \sim \chi^2_p$.

A significant result means variance increases/decreases with fitted values — your standard errors are wrong and inference is invalid. **Fix:** Heteroscedasticity-consistent (HC) robust standard errors, or transform the response variable.

Part C: Non-Parametric Tests

Non-parametric tests make no distributional assumptions on the data. They work on *ranks* rather than raw values — more robust to outliers and skewed distributions, but less statistically powerful when parametric assumptions hold.

Non-Parametric Test	Parametric Equivalent	Use case
Mann-Whitney U	Two-sample t-test	2 independent groups
Wilcoxon signed-rank	Paired t-test	2 related groups
Kruskal-Wallis	One-way ANOVA	$k \geq 3$ independent groups
Friedman	Two-way ANOVA / Repeated measures	k models on same data
Spearman correlation	Pearson correlation	Monotonic (not linear) relationships

Table 46: Non-parametric tests and their parametric counterparts.

Mann-Whitney U Test (Wilcoxon Rank-Sum)

H_0**:** The two groups come from the same distribution (medians are equal).

1. Pool all $n_1 + n_2$ observations and rank them $1, 2, \ldots, n_1 + n_2$.
2. Compute $R_1 =$ sum of ranks in group 1.
3. $U_1 = n_1 n_2 + \frac{n_1(n_1+1)}{2} - R_1$; $U_2 = n_1 n_2 - U_1$.
4. Test statistic: $U = \min(U_1, U_2)$.

For large samples, U is approximately normal:

$$z = \frac{U - \mu_U}{\sigma_U} \quad \text{where} \quad \mu_U = \frac{n_1 n_2}{2}, \quad \sigma_U = \sqrt{\frac{n_1 n_2 (n_1 + n_2 + 1)}{12}} \tag{160}$$

Example in ML: Are the prediction errors of Model A stochastically smaller than Model B's, without assuming normality?

Wilcoxon Signed-Rank Test

H_0**:** Median of paired differences is zero. The non-parametric equivalent of the paired t-test.

1. Compute differences $d_i = X_{1i} - X_{2i}$. Discard zeros.
2. Rank the absolute values $|d_i|$.
3. $W^+ =$ sum of ranks where $d_i > 0$; $W^- =$ sum of ranks where $d_i < 0$.
4. Test statistic: $W = \min(W^+, W^-)$.

Example in ML: Comparing two models' error on each test example, without assuming the differences are normally distributed.

Kruskal-Wallis Test

H_0**:** All k groups come from the same distribution. The non-parametric alternative to one-way ANOVA.

$$H = \frac{12}{N(N+1)} \sum_{i=1}^{k} \frac{R_i^2}{n_i} - 3(N+1) \quad \sim \quad \chi^2_{k-1} \text{ under } H_0 \tag{161}$$

where R_i is the sum of ranks for group i and $N = \sum_i n_i$.

Post-hoc pairwise comparisons: **Dunn's test** (with Bonferroni correction).

Friedman Test

H_0**:** All k treatments (models/algorithms) perform equally across n blocks (datasets).

This is the standard test for **comparing multiple classifiers across multiple datasets** (Demar, 2006 — the gold standard in ML benchmarking):

1. Rank the k models within each dataset (block).
2. R_j = average rank of model j across all n datasets.
3. Test statistic:

$$\chi^2_F = \frac{12n}{k(k+1)} \sum_{j=1}^{k} \left(R_j - \frac{k+1}{2} \right)^2 \quad \sim \quad \chi^2_{k-1} \tag{162}$$

Post-hoc: **Nemenyi test** for pairwise comparisons.

The Demar Protocol for Comparing Classifiers

When comparing k algorithms across n datasets:

1. Use the **Friedman test** to detect any difference.
2. If significant, use the **Nemenyi test** for pairwise differences. Two classifiers differ significantly if their average rank difference exceeds the *critical difference*: $CD = q_\alpha \sqrt{k(k+1)/(6n)}$.
3. Visualise with a **critical difference diagram** showing ranked algorithms and connected non-significantly-different groups.

This is far more statistically valid than simply averaging accuracy across datasets.

Part D: Categorical & Distribution Tests

Chi-Squared (χ^2) Goodness-of-Fit Test

H_0**:** The observed frequency distribution matches an expected (theoretical) distribution.

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i} \quad \sim \quad \chi^2_{k-1} \tag{163}$$

where O_i = observed count, E_i = expected count in category i.

Example in ML: Are the predicted class probabilities from a Naive Bayes model well-calibrated against observed frequencies?

Chi-Squared Test of Independence

H_0**:** Two categorical variables are independent.

For a $r \times c$ contingency table with observed counts O_{ij} and expected counts $E_{ij} = (\text{row total}_i \times \text{col total}_j)/N$:

$$\chi^2 = \sum_{i=1}^{r}\sum_{j=1}^{c} \frac{(O_{ij} - E_{ij})^2}{E_{ij}} \quad \sim \quad \chi^2_{(r-1)(c-1)} \tag{164}$$

Example in ML:

- **Feature selection:** Is feature X statistically associated with target Y? (χ^2 feature selection in `sklearn.feature_selection.chi2`.)
- **Decision trees:** Information gain and χ^2 splits are related criteria for choosing the best feature to split on.
- **Fairness auditing:** Is the model's prediction independent of a protected attribute (e.g. gender)?

Note: For 2×2 tables with small expected counts ($E_{ij} < 5$), use **Fisher's Exact Test** instead (no χ^2 approximation needed).

Kolmogorov-Smirnov (KS) Test

H_0**:** Two distributions are identical (or a sample comes from a specific reference distribution).

The test statistic is the maximum absolute difference between the empirical CDFs:

$$D_{n,m} = \sup_x \left|F_n(x) - G_m(x)\right| \tag{165}$$

where F_n and G_m are the empirical CDFs of the two samples.

Version	Use case
One-sample KS	Does this sample follow a specific distribution (e.g. $\mathcal{N}(0,1)$)?
Two-sample KS	Are the distributions of model predictions on train vs. test the same? (Detect distribution shift / covariate shift.)

Table 47: KS test variants.

Example in ML: Detecting data drift in production — if the KS test between current user data and training data is significant, the model's assumptions may be violated and retraining is needed.

Part E: ML-Specific Tests

McNemar's Test (Comparing Two Classifiers)

When: Comparing two binary classifiers evaluated on the *same* test set. Uses a 2×2 contingency table of *disagreements*:

	Model B correct	**Model B wrong**
Model A correct	n_{11}	n_{10}
Model A wrong	n_{01}	n_{00}

Table 48: McNemar's 2×2 disagreement table.

H_0: Both models have the same error rate (P(A wrong, B right) = P(A right, B wrong)).

$$\chi^2_{McNemar} = \frac{(n_{01} - n_{10})^2}{n_{01} + n_{10}} \quad \sim \quad \chi^2_1 \tag{166}$$

For small counts ($n_{01} + n_{10} < 25$), use the exact binomial version.

Why McNemar and not a t-Test?

Paired test accuracy scores violate the independence assumption of the two-sample t-test. McNemar's test uses only the *discordant* pairs (cases where the models disagree), making it the correct and most powerful test for this scenario.

DeLong Test (Comparing Two AUC Scores)

When: Comparing the AUC-ROC of two models on the same test set.

The DeLong et al. (1988) method computes the covariance structure of the two AUC estimates using the theory of U-statistics, giving an asymptotically normal test:

$$z = \frac{\widehat{AUC}_1 - \widehat{AUC}_2}{\sqrt{\widehat{\mathrm{Var}}(\widehat{AUC}_1) + \widehat{\mathrm{Var}}(\widehat{AUC}_2) - 2\,\widehat{\mathrm{Cov}}(\widehat{AUC}_1, \widehat{AUC}_2)}} \sim \mathcal{N}(0,1) \tag{167}$$

The covariance term accounts for the fact that both AUCs are computed on the *same* set of subjects. Available in Python via `scipy` or the `pROC` package in R.

Example: Is the AUC of a deep learning model (0.91) significantly higher than a logistic regression model (0.87) on the same patient test set?

Permutation Test (Is the Model Better Than Chance?)

The most general and assumption-free test in ML.

H_0: The model's performance is no better than what would be achieved if the labels were randomly shuffled.

1. Compute observed metric S_{obs} (e.g. accuracy).
2. Repeat B times (e.g. $B = 10{,}000$):
    a. Randomly shuffle the labels y.

b. Retrain or re-evaluate the model.

c. Record the permuted metric S_b.

3. p-value $= \frac{1}{B}\sum_{b=1}^{B} \mathbf{1}[S_b \geq S_{\text{obs}}]$.

Intuition

If your model's accuracy is 75% but a model trained on randomly shuffled labels also achieves 74% most of the time, your model has learned nothing — it is exploiting class imbalance or data leakage. The permutation test catches this immediately.

Available in `sklearn.model_selection.permutation_test_score`.

Bootstrap Confidence Intervals for Model Metrics

When: You want a confidence interval for any metric (AUC, F1, RMSE) without distributional assumptions.

1. Draw B bootstrap samples of the test set (with replacement).
2. Compute the metric $\hat{\theta}_b$ on each bootstrap sample.
3. The **percentile bootstrap CI** is:

$$\left[\hat{\theta}^*_{(\alpha/2)},\ \hat{\theta}^*_{(1-\alpha/2)}\right] \tag{168}$$

i.e. the 2.5^{th} and 97.5^{th} percentiles of $\{\hat{\theta}_b\}_{b=1}^{B}$.

Example: "Our model achieves AUC = 0.89 (95% CI: 0.85–0.93)" — reported in virtually every medical AI paper.

Part F: Multiple Testing Corrections

When you run m tests simultaneously, the chance of getting at least one false positive grows rapidly:

$$P(\text{at least one false positive}) = 1 - (1-\alpha)^m \tag{169}$$

For $m = 20$ tests at $\alpha = 0.05$: $P \approx 64\%$. This is the **Multiple Comparisons Problem**.

Bonferroni Correction (FWER Control)

Control the **Family-Wise Error Rate (FWER)** — the probability of *any* false positive.

$$\alpha_{\text{adjusted}} = \frac{\alpha}{m} \tag{170}$$

Reject $H_{0,i}$ if $p_i < \alpha/m$.

Pros: Simple, very conservative, strong control.
Cons: Loses power rapidly as m grows (too many false negatives). Appropriate when any false positive is unacceptable (e.g. drug safety trials).

Benjamini-Hochberg Procedure (FDR Control)

Control the **False Discovery Rate (FDR)** — the *expected proportion* of false positives among all rejected hypotheses. Much more powerful than Bonferroni for large-scale testing.

1. Sort the m p-values: $p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(m)}$.
2. Find the largest k such that:
$$p_{(k)} \leq \frac{k}{m} \cdot q^*$$
where q^* is the desired FDR level (e.g. 0.05).
3. Reject all $H_{0,(i)}$ for $i = 1, \ldots, k$.

Example in ML: Genomics — testing 20,000 genes for association with a disease. Bonferroni would require $p < 2.5 \times 10^{-6}$ for each gene. Benjamini-Hochberg allows far more discoveries while controlling the proportion of false ones to 5%.

Method	Controls	When to use	Power
Bonferroni	FWER	Few tests; any FP is dangerous	Low
Holm-Bonferroni	FWER	Few tests; slightly more power	Moderate
Benjamini-Hochberg	FDR	Many tests; some FPs tolerable	High
Storey q-value	FDR	Genomics, large-scale GWAS	Highest

Table 49: Multiple testing correction methods compared.

Quick Reference: Which Test to Use?

Question	Parametric	Non-Parametric
Is this sample mean = μ_0?	One-sample t-test	Wilcoxon signed-rank
Do 2 independent groups differ?	Two-sample t-test (Welch)	Mann-Whitney U
Do 2 paired/matched groups differ?	Paired t-test	Wilcoxon signed-rank
Do $k \geq 3$ independent groups differ?	One-way ANOVA + Tukey HSD	Kruskal-Wallis + Dunn
Do k models differ across n datasets?	Repeated-measures ANOVA	Friedman + Nemenyi
Are 2 categorical variables associated?	—	χ^2 (or Fisher's exact)
Does data follow a distribution?	—	Shapiro-Wilk; KS test
Do 2 binary classifiers differ (same test)?	—	McNemar's test
Do 2 AUC scores differ?	DeLong test	Bootstrap CI
Is model better than chance?	—	Permutation test
Are regression residuals normal?	Shapiro-Wilk	Q-Q plot
Is variance constant (homoscedastic)?	Levene's / Breusch-Pagan	—
Are residuals autocorrelated?	Durbin-Watson	Runs test
Are multiple test results real?	Bonferroni / BH-FDR	Permutation-based FDR

Table 50: Quick reference: choosing the right statistical test.

Part XXI

The Story of K-Means Clustering

Introduction

Every algorithm in this document so far has been *supervised*: the data has labels (y_i) and the model learns to map $\mathbf{x}_i \to y_i$. But in the real world, most data is **unlabelled**. You have millions of customer records, and nobody has told you which customers are "similar."

Clustering is the task of discovering natural groupings (clusters) in unlabelled data. It is the most fundamental **unsupervised learning** problem.

Key Idea

K-Means partitions n data points into exactly K clusters such that each point belongs to the cluster with the nearest centroid. The algorithm alternates between **assigning** each point to its closest centroid and **moving** each centroid to the mean of its assigned points, repeating until convergence.

What Is It Used For?

- **Customer segmentation:** Group customers by purchasing behaviour to personalise marketing.
- **Image compression:** Replace each pixel's colour with its cluster centroid (colour quantisation).
- **Document clustering:** Group news articles by topic.
- **Anomaly detection:** Points far from any centroid are outliers.
- **Preprocessing:** Use cluster memberships as new features for downstream supervised models.
- **Vector quantisation:** Codebooks in audio compression (MP3) and computer vision (bag-of-visual-words).

Step 1: The Objective Function

K-Means minimises the **Within-Cluster Sum of Squares (WCSS)**, also called **inertia**:

$$J = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2 \tag{171}$$

where $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$ is the **centroid** (mean) of cluster k.

Intuition

Imagine placing K magnets on the floor among your data points. Each data point snaps to its nearest magnet. Then you slide each magnet to the centre of gravity of its attached points. Repeat. The magnets settle into the "natural centres" of your data. That is K-Means.

Step 2: Lloyd's Algorithm (The Standard K-Means)

1. **Initialise:** Choose K initial centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$ (randomly from the data, or via K-Means++).
2. **Assignment step (E-step):** Assign each point to its nearest centroid:

$$c_i = \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2 \qquad \forall\, i = 1, \ldots, n \tag{172}$$

3. **Update step (M-step):** Recompute each centroid as the mean of its assigned points:

$$\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i : c_i = k} \mathbf{x}_i \qquad \forall\, k = 1, \ldots, K \tag{173}$$

4. **Repeat** steps 2–3 until assignments do not change (convergence) or a maximum number of iterations is reached.

Convergence Guarantee

Each step of Lloyd's algorithm *monotonically decreases* J (or leaves it unchanged):

- The **assignment step** minimises J over all possible assignments given fixed centroids.
- The **update step** minimises J over all possible centroid positions given fixed assignments (the mean is the ℓ_2-minimiser).

Since $J \geq 0$ and strictly decreases each iteration, convergence is guaranteed. However, the algorithm converges to a **local minimum** — not necessarily the global minimum.

Complexity

Per iteration: $\mathcal{O}(nKd)$ where n = number of points, K = clusters, d = dimensions (computing nK distances).
In practice: Typically converges in 10–300 iterations. Always run multiple restarts (e.g. 10) and keep the solution with the lowest inertia.

Step 3: K-Means++ Initialisation

Random initialisation often leads to poor local minima. **K-Means++** (Arthur & Vassilvitskii, 2007) uses a smarter seeding strategy:

1. Choose the first centroid $\boldsymbol{\mu}_1$ uniformly at random from the data.
2. For each subsequent centroid $k = 2, \ldots, K$:

a. For each point $\mathbf{x}_i$, compute its squared distance to the nearest already-chosen centroid: $D(\mathbf{x}_i)^2 = \min_{j<k} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$.

b. Sample the next centroid with probability proportional to $D(\mathbf{x}_i)^2$: $P(\mathbf{x}_i) \propto D(\mathbf{x}_i)^2$.

3. Proceed with standard Lloyd's algorithm.

Intuition

K-Means++ spreads the initial centroids far apart by preferentially seeding new centroids in regions that are poorly covered. This gives an $\mathcal{O}(\log K)$ approximation guarantee on the optimal WCSS and dramatically reduces the number of restarts needed.

Step 4: Choosing K

K-Means requires specifying K in advance. Three principled methods:

The Elbow Method

Plot WCSS vs. K for $K = 1, 2, \ldots, K_{\max}$:

- WCSS always decreases as K increases (more clusters ⇒ smaller groups ⇒ smaller inertia).
- The **elbow** is the K where the rate of decrease sharply slows down — adding more clusters gives diminishing returns.
- Not always clear-cut. Use in combination with other methods.

The Silhouette Score

For each point $\mathbf{x}_i$, the silhouette coefficient measures how well it fits its own cluster vs. the next best cluster:

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \quad \in [-1, 1] \tag{174}$$

where:

- a_i = mean distance to all other points *in the same cluster* (intra-cluster cohesion).
- b_i = mean distance to all points in the *nearest neighbouring cluster* (inter-cluster separation).

s_i	Interpretation	Meaning
$\approx +1$	Well clustered	Far from neighbours, close to own cluster
≈ 0	On boundary	Between two clusters
≈ -1	Misclassified	Closer to a different cluster than its own

Table 51: Silhouette score interpretation.

The **mean silhouette score** across all points is plotted vs. K; choose K that maximises it.

Gap Statistic

Compare the WCSS of the clustering against WCSS on randomly generated reference data (with no cluster structure):

$$\text{Gap}(K) = \mathbb{E}[\log W_{\text{rand}}(K)] - \log W(K) \tag{175}$$

Choose the smallest K such that $\text{Gap}(K) \geq \text{Gap}(K+1) - s_{K+1}$ (within one standard error of the next). Most statistically rigorous of the three methods.

Method	What it measures	Limitation
Elbow	Inertia rate of change	Elbow often ambiguous
Silhouette	Cluster cohesion vs. separation	Assumes convex clusters
Gap Statistic	Vs. random reference	Computationally expensive

Table 52: Methods for choosing K.

Step 5: Evaluation Metrics

Metric	Description
Inertia (WCSS)	Lower is better. Biased toward larger K.
Silhouette	Higher is better ($\in [-1, 1]$). Model-agnostic.
Davies-Bouldin Index	Lower is better. Ratio of within-cluster to between-cluster distances.
Calinski-Harabasz Index	Higher is better. Ratio of between-cluster to within-cluster dispersion.
Adjusted Rand Index	For labelled data only. Measures agreement between clustering and true labels ($\in [-1, 1]$, 1 = perfect).

Table 53: Clustering evaluation metrics.

Step 6: Limitations & When K-Means Fails

Limitation	Why / Fix
Assumes spherical clusters	K-Means uses Euclidean distance ⇒ cannot find elongated or curved clusters. **Fix:** GMM, DBSCAN, spectral clustering.
Sensitive to outliers	Outliers pull centroids. **Fix:** K-Medoids (use medians).
Must specify K in advance	Use elbow/silhouette/gap, or DBSCAN (no K needed).
Local minima	Different restarts give different results. **Fix:** K-Means++ + multiple restarts.
Equal cluster size assumption	Assigns equal weight to all clusters. **Fix:** GMM with learnable mixing weights.
Curse of dimensionality	Distances become meaningless in high d. **Fix:** Reduce with PCA before clustering.

Table 54: K-Means limitations and fixes.

Step 7: Variants and Related Algorithms

K-Medoids (PAM)

Instead of a mean centroid, use the actual data point that minimises within-cluster distance (the **medoid**). More robust to outliers and works with non-Euclidean distances.

Mini-Batch K-Means

At each iteration, use a random mini-batch (subset) of b points instead of the full dataset for the update step. Scales to millions of points with a small accuracy tradeoff.

Gaussian Mixture Models (GMM)

The probabilistic generalisation of K-Means. Instead of hard assignments, each point has a **soft** probability of belonging to each Gaussian component:

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x};\, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{176}$$

Fitted via the **Expectation-Maximisation (EM)** algorithm. Unlike K-Means, GMMs can model clusters of arbitrary shape (via $\boldsymbol{\Sigma}_k$) and provide calibrated membership probabilities.

K-Means as a Special Case of GMM

K-Means is equivalent to a GMM where all covariance matrices are constrained to $\boldsymbol{\Sigma}_k = \sigma^2 I$ and assignments are hard (winner-takes-all). As $\sigma^2 \to 0$, the EM algorithm for a spherical GMM converges to K-Means.

DBSCAN (Density-Based Spatial Clustering)

DBSCAN (Ester et al., 1996) requires no K, discovers arbitrarily shaped clusters, and automatically labels outliers as noise:

- **Core point:** Has at least `min_samples` points within radius ε.
- **Border point:** Within ε of a core point but not itself a core point.
- **Noise point:** Neither core nor border — an outlier.

Two hyperparameters: ε (neighbourhood radius) and `min_samples` (density threshold).

Algorithm	Cluster Shape	Best For
K-Means	Spherical	Fast, large datasets, known K
K-Medoids	Spherical	Noisy data, non-Euclidean metrics
GMM	Elliptical	Soft assignments, probabilistic output
DBSCAN	Arbitrary	Irregular shapes, outlier detection
Hierarchical	Any (dendrogram)	Exploring cluster hierarchy
Spectral	Any	Graph-structured data

Table 55: Clustering algorithm comparison.

Part XXII

PCA: The Full Treatment

Introduction

Part 15 introduced PCA at a high level. This part provides the complete mathematical derivation, all the practical details, and the important variants.

Key Idea

Principal Component Analysis (PCA) finds a new orthonormal coordinate system for the data such that the axes (principal components) are ordered by the amount of variance they capture. Projecting onto the top k components compresses the data while retaining the maximum possible variance — or equivalently, minimising the squared reconstruction error.

Why Do We Need PCA?

- **Dimensionality reduction:** $d = 50{,}000$ gene expressions $\to k = 50$ PCA components, retaining 95% of variance.
- **Visualisation:** Project to 2D or 3D to inspect cluster structure.
- **Noise removal:** Low-variance components often capture noise. Discarding them de-noises the data.
- **Speed-up:** Reduce input dimension before training SVMs, KNN, or neural networks.
- **Decorrelation:** Principal components are *uncorrelated* — removes multicollinearity for regression.
- **Compression:** Images, audio, video (PCA $\approx$ SVD-based compression).

Step 1: Centring the Data

All PCA derivations assume zero-mean data. First centre:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i - \bar{\mathbf{x}} \qquad \bar{\mathbf{x}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i \tag{177}$$

Form the centred data matrix $\tilde{X} \in \mathbb{R}^{n \times d}$.

Step 2: Two Equivalent Derivations

Derivation 1: Maximum Variance

Find the unit vector $\mathbf{u}_1$ that maximises the variance of the projected data:

$$\mathbf{u}_1 = \arg\max_{\|\mathbf{u}\|=1} \operatorname{Var}\left[\tilde{X}\mathbf{u}\right] = \arg\max_{\|\mathbf{u}\|=1} \mathbf{u}^\top \Sigma\, \mathbf{u} \tag{178}$$

where $\Sigma = \frac{1}{n-1}\tilde{X}^\top\tilde{X}$ is the $d \times d$ **sample covariance matrix**.

By the **method of Lagrange multipliers** (with constraint $\mathbf{u}^\top\mathbf{u} = 1$):

$$\mathcal{L} = \mathbf{u}^\top\Sigma\mathbf{u} - \lambda(\mathbf{u}^\top\mathbf{u} - 1) \quad \Longrightarrow \quad \frac{\partial\mathcal{L}}{\partial\mathbf{u}} = 0 \quad \Longrightarrow \quad \Sigma\mathbf{u}_1 = \lambda_1\mathbf{u}_1 \tag{179}$$

The first principal component $\mathbf{u}_1$ is the eigenvector of Σ corresponding to its largest eigenvalue λ_1.

The variance captured by this component is exactly λ_1.

Each subsequent component $\mathbf{u}_j$ is the eigenvector corresponding to the j-th largest eigenvalue λ_j, subject to being orthogonal to all previous components.

Derivation 2: Minimum Reconstruction Error

Find the k-dimensional subspace U_k that minimises the mean squared reconstruction error:

$$\arg\min_{U_k} \frac{1}{n}\sum_{i=1}^{n}\left\|\tilde{\mathbf{x}}_i - U_k U_k^\top \tilde{\mathbf{x}}_i\right\|^2 \tag{180}$$

Both derivations give the same answer: the top k eigenvectors of Σ. This is a remarkable equivalence — maximising variance *is the same as* minimising reconstruction error.

Step 3: The Eigendecomposition

$$\Sigma = Q\Lambda Q^\top \qquad \text{where} \quad \Lambda = \operatorname{diag}(\lambda_1, \lambda_2, \ldots, \lambda_d), \quad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0 \tag{181}$$

and $Q = [\mathbf{u}_1 \mid \mathbf{u}_2 \mid \cdots \mid \mathbf{u}_d]$ is orthonormal ($Q^\top Q = I$).

To project to k dimensions, take $Q_k = [\mathbf{u}_1, \ldots, \mathbf{u}_k]$:

$$Z = \tilde{X}\, Q_k \in \mathbb{R}^{n\times k} \qquad \text{(the scores / principal component projections)} \tag{182}$$

Step 4: Connection to SVD (The Practical Algorithm)

Computing $\Sigma = \tilde{X}^\top\tilde{X}/(n-1)$ explicitly is numerically unstable for large d. In practice, PCA is computed via **Singular Value Decomposition**:

$$\tilde{X} = USV^\top \tag{183}$$

where:

- $U \in \mathbb{R}^{n\times n}$: left singular vectors (sample scores, up to scaling).

- $S \in \mathbb{R}^{n \times d}$: diagonal matrix of singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.
- $V \in \mathbb{R}^{d \times d}$: right singular vectors (**the principal components** = columns of V).

The relationship to eigendecomposition:

$$\Sigma = \frac{\tilde{X}^\top \tilde{X}}{n-1} = \frac{VS^\top U^\top USV^\top}{n-1} = V \cdot \frac{S^2}{n-1} \cdot V^\top \quad \Longrightarrow \quad \lambda_j = \frac{\sigma_j^2}{n-1} \tag{184}$$

SVD is numerically stable, works for rectangular matrices, and handles $d \gg n$ (more features than samples) gracefully. All major implementations (scikit-learn, NumPy) use SVD.

Truncated SVD for Large Datasets

For very large $\tilde{X}$, compute only the top-k singular values/vectors via randomised methods (Halko et al., 2011): $\mathcal{O}(ndk)$ instead of $\mathcal{O}(\min(n,d)^2 \max(n,d))$.

Step 5: How Much Variance to Keep?

Explained Variance Ratio

$$\text{EVR}_j = \frac{\lambda_j}{\sum_{i=1}^{d} \lambda_i} = \frac{\sigma_j^2}{\sum_{i=1}^{d} \sigma_i^2} \qquad \text{Cumulative EVR}(k) = \sum_{j=1}^{k} \text{EVR}_j \tag{185}$$

Common rule: Choose k so that cumulative EVR $\geq 95\%$ (or 99% for noise-sensitive applications).

The Scree Plot

Plot λ_j (or EVR) vs. component number j. Look for an "elbow" where eigenvalues drop sharply — components beyond the elbow mostly capture noise (analogous to the K-Means elbow method).

Kaiser's Rule

In standardised data (where $\sum_j \lambda_j = d$), retain only components with $\lambda_j > 1$. Rationale: a component should explain at least as much variance as a single original variable. Simple but often retains too many components.

Step 6: PCA as a Linear Autoencoder

PCA has a deep connection to neural networks. A linear autoencoder with no activation functions:

$$\mathbf{z} = W_{\text{enc}}\, \tilde{\mathbf{x}} \quad (\text{encoder: } d \to k) \tag{186}$$

$$\hat{\mathbf{x}} = W_{\text{dec}}\, \mathbf{z} \quad (\text{decoder: } k \to d) \tag{187}$$

trained to minimise $\|\tilde{\mathbf{x}} - \hat{\mathbf{x}}\|^2$ finds $W_{\text{enc}} = Q_k^\top$ and $W_{\text{dec}} = Q_k$ — **exactly the PCA projection and reconstruction.**

This means PCA is the globally optimal linear autoencoder. Adding non-linear activations gives you a **non-linear autoencoder** (or VAE), which can capture curved manifolds PCA cannot.

Step 7: Whitening (Sphering)

After PCA projection, the components z_j have different variances λ_j. **Whitening** scales them to unit variance:

$$\tilde{z}_j = \frac{z_j}{\sqrt{\lambda_j}} \quad \Longrightarrow \quad \text{Cov}(\tilde{\mathbf{z}}) = I \tag{188}$$

ZCA whitening additionally rotates back to the original space: $\tilde{\mathbf{x}} = \Sigma^{-1/2}\mathbf{x}$. This produces whitened data that still looks like the original (used in image preprocessing for deep learning and as a preprocessing step for ICA).

When to whiten: Before ICA, before feeding data to algorithms sensitive to feature scale (e.g. neural networks without BatchNorm).

Step 8: Kernel PCA

Standard PCA is **linear**. **Kernel PCA** (Schölkopf et al., 1998) extends PCA to non-linear manifolds using the kernel trick.

Replace all dot products $\mathbf{x}_i^\top \mathbf{x}_j$ with a kernel $K(\mathbf{x}_i, \mathbf{x}_j)$. The kernel matrix $\mathbf{K} \in \mathbb{R}^{n \times n}$ is eigendecomposed:

$$\mathbf{K} = V \Lambda V^\top \qquad \text{(after double-centring in feature space)} \tag{189}$$

The j-th projected coordinate for a point $\mathbf{x}$ is:

$$z_j = \sum_{i=1}^{n} v_{ij}\, K(\mathbf{x}_i, \mathbf{x}) \tag{190}$$

Kernel	Feature space	Effect
Linear	$\mathbb{R}^d$ (same)	Equivalent to standard PCA
Polynomial (d)	High-degree polynomial features	Non-linear boundaries
RBF / Gaussian	Infinite-dimensional	Very flexible non-linear PCA

Table 56: Kernel PCA variants.

Kernel PCA vs. t-SNE/UMAP

Kernel PCA produces a **deterministic**, linearly embeddable representation that can project new points. t-SNE/UMAP are better for *visualisation* but are stochastic (t-SNE) and cannot straightforwardly embed new test points (t-SNE). Use Kernel PCA as a preprocessing step; use t-SNE/UMAP for exploratory data analysis only.

Step 9: PCA in Practice

Standardise First?

- If features have **different units** (e.g. height in cm and weight in kg): **standardise** each feature to zero mean and unit variance before PCA. Otherwise, high-variance features dominate the principal components regardless of actual informativeness.
- If features are **already on the same scale** (e.g. pixel intensities in $[0, 255]$): standardisation is optional.

The PCA Pipeline

1. Standardise features (if needed).
2. Fit PCA on the **training set only** (compute $\bar{\mathbf{x}}$ and Q_k from train data).
3. Transform **both** train and test sets using the *same* fitted PCA (never fit on the test set — data leakage).
4. Choose k by cumulative EVR or cross-validation.

When NOT to Use PCA

- When **interpretability** of individual features is critical — principal components are linear combinations of all original features and are hard to interpret.
- When the data has **non-linear structure** — use Kernel PCA, t-SNE, UMAP, or an autoencoder.
- When features are **categorical** — PCA requires continuous features. Use MCA (Multiple Correspondence Analysis) instead.
- When the **signal is in low-variance directions** — e.g. in Linear Discriminant Analysis (LDA), the class-discriminating direction can have low marginal variance but high discriminative power.
- When you only have a **handful of features** — PCA adds complexity without benefit.

K-Means + PCA: The Classic Pipeline

In practice, K-Means and PCA are almost always used together:

1. **Standardise** the raw features.
2. Apply **PCA** to reduce dimensionality (e.g. 95% EVR).
3. Run **K-Means** on the PCA-reduced representation.

Why?

- Euclidean distance (which K-Means uses) is meaningless in very high dimensions (curse of dimensionality).
- PCA removes noise and correlations, making the cluster structure cleaner.

- Dramatically speeds up K-Means (d reduced from thousands to tens).
- The PCA components are uncorrelated, so the "spherical cluster" assumption of K-Means is more justified.

Spectral Clustering: Beyond the Classic Pipeline

When clusters are non-convex (e.g. concentric rings), even PCA + K-Means fails. **Spectral Clustering** constructs an affinity graph (similarity matrix) between points, computes the graph Laplacian's eigenvectors, and runs K-Means *in that eigenspace*. The eigenspace "unfolds" non-convex cluster structure that linear projections cannot capture.

Part XXIII

Advanced Gradient Descent and Optimisers

Why Plain SGD is Not Enough

Plain Stochastic Gradient Descent (SGD) updates parameters using only the gradient at the current mini-batch:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$$

This works, but it has serious problems in practice:

- **Oscillation in narrow valleys**: The loss landscape of neural networks often looks like a long, thin canyon. SGD bounces back and forth across the narrow dimension while making very slow progress along the length of the canyon toward the minimum.
- **Same learning rate for all parameters**: Some weights need large updates (those connected to rare features), others need small updates (those already near their optimum). One global η cannot handle both.
- **Saddle points**: At a saddle point, the gradient is zero but it is neither a min nor a max. SGD can get stuck because the gradient gives no signal about which direction to move.
- **Noisy updates**: Because each update uses only one mini-batch, the gradient is a noisy estimate of the true gradient. The noise causes the loss to bounce around rather than converge smoothly.

Intuition

Imagine rolling a ball down a hilly landscape toward a valley. Plain SGD gives the ball a small kick every step. The ball bounces around chaotically. The advanced optimisers below add physics: *momentum* (the ball builds up speed), *adaptive learning rates* (automatic friction per direction), or both.

SGD with Momentum

Momentum accumulates a velocity vector in the direction of persistent gradients, damping oscillations and accelerating progress.

$$v_t \leftarrow \beta\, v_{t-1} + (1-\beta)\, \nabla_\theta \mathcal{L}_t \tag{191}$$
$$\theta \leftarrow \theta - \eta\, v_t \tag{192}$$

where $\beta \in [0.8, 0.99]$ (typically 0.9) is the **momentum coefficient** and $v_0 = 0$.

Key Idea

The velocity v_t is an exponentially-weighted moving average (EWMA) of past gradients. If the gradient keeps pointing in the same direction, v_t grows large (accelerates). If the gradient oscillates, the positive and negative contributions cancel out in v_t (smooths oscillations). The effective step in any direction is amplified by a factor of up to $\frac{1}{1-\beta}$. For $\beta = 0.9$: max effective step $\approx 10\times$ the gradient.

Nesterov Accelerated Gradient (NAG)

Standard momentum computes the gradient at the current position θ, then applies the velocity. **NAG** looks *ahead*: it computes the gradient at the approximate future position $\theta - \beta v_{t-1}$:

$$v_t \leftarrow \beta\, v_{t-1} + \nabla_\theta \mathcal{L}(\theta - \beta v_{t-1}) \tag{193}$$
$$\theta \leftarrow \theta - \eta\, v_t \tag{194}$$

Intuition

Instead of computing the gradient where you *are*, compute it where you *will be* after applying momentum. If the lookahead position has a gradient pointing back, NAG applies the brakes *before* overshooting. This "corrective look-ahead" makes NAG faster to converge than standard momentum, especially near optima.

AdaGrad: Adaptive Learning Rates

AdaGrad (Duchi et al., 2011) gives each parameter its own learning rate, scaled inversely by the square root of the sum of all past squared gradients:

$$G_t \leftarrow G_{t-1} + g_t \odot g_t \qquad (G_0 = 0) \tag{195}$$
$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t \tag{196}$$

where $g_t = \nabla_\theta \mathcal{L}_t$, $\odot$ is elementwise multiplication, and $\epsilon \approx 10^{-8}$ prevents division by zero.

Key Idea

Intuition: G_t is the running sum of squared gradients. If a parameter has received many large gradient updates (i.e. it is "well-trained"), its effective learning rate shrinks. If a parameter rarely gets updated (sparse features, e.g. word embeddings for rare words), its effective rate stays large. AdaGrad is excellent for **sparse data** and NLP.

Problem with AdaGrad: G_t monotonically increases — it never forgets old gradients. Eventually, the effective learning rate shrinks to zero and training stops, even if the loss can still decrease.

RMSProp: Fixing AdaGrad's Memory Problem

RMSProp (Hinton, 2012 — unpublished lecture notes) replaces the cumulative sum in AdaGrad with an **exponentially weighted moving average** of squared gradients, so old history is gradually forgotten:

$$s_t \leftarrow \rho\, s_{t-1} + (1-\rho)\, g_t \odot g_t \tag{197}$$

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_t \tag{198}$$

where $\rho \approx 0.9$ (decay rate). Now s_t reflects a *recent* window of gradient magnitudes rather than the entire history.

- If s_t is large (recent gradients large) $\Rightarrow$ small step (we are in a steep area).
- If s_t is small (recent gradients small) $\Rightarrow$ large step (we are in a flat area — push harder).

Adam: Adaptive Moment Estimation

Adam (Kingma & Ba, 2015) combines momentum (1st moment) with RMSProp (2nd moment), and fixes the **initialisation bias** that occurs because both moments start at zero.

Full Adam Update Rule

$$m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t \qquad \text{(1st moment: mean of gradient)}$$

$$v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t \odot g_t \qquad \text{(2nd moment: variance of gradient)}$$

$$\hat{m}_t \leftarrow \frac{m_t}{1-\beta_1^t} \qquad \text{(bias-corrected 1st moment)}$$

$$\hat{v}_t \leftarrow \frac{v_t}{1-\beta_2^t} \qquad \text{(bias-corrected 2nd moment)}$$

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\, \hat{m}_t \qquad \text{(parameter update)}$$

Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\eta = 0.001$.

Why Bias Correction?

At $t = 1$: $m_1 = (1-\beta_1) g_1 = 0.1\, g_1$. This is $10\times$ *smaller* than it should be, because $m_0 = 0$ pulls the EWMA toward zero at the start. The bias correction $\hat{m}_1 = m_1/(1-0.9) = g_1$ restores the true estimate. As $t \to \infty$, $(1-\beta_1^t) \to 1$ and the correction vanishes.

Key Idea

Adam = Momentum (smooth the gradient direction) + RMSProp (per-parameter step size scaling) + Bias correction (correct for zero-initialisation). It is the **default optimiser for deep learning** because it is robust to hyperparameter choice and works well out-of-the-box for most architectures.

AdamW: Adam with Decoupled Weight Decay

Standard Adam (and SGD) regularise by adding $\lambda\|\theta\|^2$ to the loss, which produces a gradient term $\lambda\theta$ absorbed into g_t. This couples weight decay with the adaptive scaling — the decay is *divided* by $\sqrt{\hat{v}_t}$, making it effectively smaller for frequently-updated weights.

AdamW (Loshchilov & Hutter, 2019) decouples them:

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t \; - \; \eta\lambda\theta$$

The weight decay term $\eta\lambda\theta$ is applied *directly* to the parameter, independent of the gradient scale. This is equivalent to what L2 regularisation does in plain SGD, and is the **correct** way to regularise adaptive optimisers. AdamW is the standard for fine-tuning large language models.

Learning Rate Scheduling

Even with a good optimiser, a fixed learning rate is suboptimal. Schedulers adjust η during training.

Schedule	Formula / Behaviour
Step Decay	$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/k \rfloor}$. Drop by factor γ every k steps.
Exponential	$\eta_t = \eta_0 \cdot e^{-\lambda t}$. Smooth exponential decay.
Cosine Annealing	$\eta_t = \eta_{\min} + \frac{\eta_0 - \eta_{\min}}{2}\left(1 + \cos\frac{\pi t}{T}\right)$. Smooth decay to $\eta_{\min}$.
Cosine with Warm Restarts	Periodically reset η to η_0 (SGDR). Escapes local minima.
Linear Warmup	Start at $\eta = 0$, increase linearly for k steps, then decay. Standard for Transformers.
1-Cycle Policy	Warm up then cool down in one big cycle. Fast convergence (Super-Convergence).
ReduceLROnPlateau	Halve η when validation loss stops improving for p epochs (patience).

Why Warmup for Transformers?

At the start of training, the model's gradients are large and noisy. Starting with a high learning rate causes divergence. Warming up from zero gives the second-moment estimate v_t time to stabilise before the learning rate grows large, avoiding early instability. The original Transformer paper uses: $\eta_t = d_{\text{model}}^{-0.5} \cdot \min(t^{-0.5},\ t \cdot T_{\text{warmup}}^{-1.5})$

Optimiser	Best For	Key Weakness
SGD	Convex problems, well-tuned NNs	Slow, sensitive to η
SGD + Momentum	CNNs (image), generalization	Still needs tuning
AdaGrad	Sparse features, NLP	LR shrinks to 0
RMSProp	RNNs, non-stationary objectives	No bias correction
Adam	Most deep learning	May generalise worse than SGD
AdamW	LLM fine-tuning, Transformers	Slight extra memory

Part XXIV

Gradient Boosting and XGBoost: In Depth

Gradient Boosting from First Principles

Gradient Boosting builds an additive model by training weak learners (decision trees) *sequentially*, each correcting the residual error of the previous ensemble.

The Additive Model Framework

At iteration m, the ensemble prediction is:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

where h_m is the new weak learner and η is the learning rate (shrinkage). We want h_m to reduce the overall loss $\mathcal{L}(y, F(x))$.

Functional Gradient Descent

Think of it as gradient descent *in function space*, not parameter space:

$$h_m(x) \approx -\nabla_F \mathcal{L}(y, F_{m-1}(x)) = -\frac{\partial \mathcal{L}(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)} \triangleq r_i^{(m)}$$

These $r_i^{(m)}$ are the **pseudo-residuals** — the negative gradient of the loss with respect to the current prediction for each training example.

Key Idea

For **MSE loss**: $r_i = y_i - F_{m-1}(x_i)$ (actual residuals).
For **Log loss** (classification): $r_i = y_i - p_i$ (difference between true label and predicted probability).
For **MAE loss**: $r_i = \text{sign}(y_i - F_{m-1}(x_i))$ (± 1).

The Full Gradient Boosting Algorithm

1. Initialise: $F_0(x) = \arg\min_\gamma \sum_i \mathcal{L}(y_i, \gamma)$ (e.g. mean of y for MSE).
2. For $m = 1, 2, \ldots, M$:
   (a) Compute pseudo-residuals: $r_i^{(m)} = -\left[\frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$
   (b) Fit a **shallow decision tree** h_m to the pseudo-residuals $\{(x_i, r_i^{(m)})\}_{i=1}^n$.
   (c) Find optimal leaf weights γ_{jm} for each leaf j: $\gamma_{jm} = \arg\min_\gamma \sum_{x_i \in R_{jm}} \mathcal{L}(y_i, F_{m-1}(x_i) + \gamma)$
   (d) Update: $F_m(x) = F_{m-1}(x) + \eta \sum_j \gamma_{jm} \mathbf{1}[x \in R_{jm}]$
3. Output $F_M(x)$.

XGBoost: Second-Order Gradient Boosting

XGBoost (Chen & Guestrin, 2016) extends gradient boosting in several key ways. The most important is using a **second-order Taylor expansion** of the loss.

The XGBoost Objective

At step m, we add a tree f_m. Define:

$$\mathcal{L}^{(m)} = \sum_i \mathcal{L}(y_i, \hat{y}_i^{(m-1)} + f_m(x_i)) + \Omega(f_m)$$

where $\Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2$ is a regularisation term on the tree structure (T = number of leaves, w = leaf weights).

Expanding with a second-order Taylor expansion around $\hat{y}_i^{(m-1)}$:

$$\mathcal{L}^{(m)} \approx \sum_i \left[\mathcal{L}(y_i, \hat{y}_i^{(m-1)}) + g_i f_m(x_i) + \frac{1}{2} h_i f_m(x_i)^2\right] + \Omega(f_m)$$

where:

$$g_i = \frac{\partial \mathcal{L}(y_i, \hat{y}_i^{(m-1)})}{\partial \hat{y}_i^{(m-1)}} \quad \text{(first derivative = gradient)} \tag{199}$$

$$h_i = \frac{\partial^2 \mathcal{L}(y_i, \hat{y}_i^{(m-1)})}{\partial (\hat{y}_i^{(m-1)})^2} \quad \text{(second derivative = Hessian)} \tag{200}$$

Optimal Leaf Weight

Grouping training instances by which leaf j they fall into ($I_j = \{i : x_i \in j\}$):

$$\tilde{\mathcal{L}}^{(m)} = \sum_{j=1}^{T} \left[\left(\sum_{i \in I_j} g_i\right) w_j + \frac{1}{2}\left(\sum_{i \in I_j} h_i + \lambda\right) w_j^2\right] + \gamma T$$

Taking derivative w.r.t. w_j and setting to zero:

$$\boxed{w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}}$$

Substituting back gives the **structure score** of the tree:

$$\tilde{\mathcal{L}}^* = -\frac{1}{2} \sum_{j=1}^{T} \frac{\left(\sum_{i \in I_j} g_i\right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

How XGBoost Splits Nodes

For a candidate split that divides leaf j into left L and right R:

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma$$

where $G = \sum_{i \in I} g_i$, $H = \sum_{i \in I} h_i$.

Key Idea

The gain formula is the improvement in the structure score from making the split. If Gain < 0 (split doesn't improve enough to overcome the penalty γ), the split is rejected. This is how **tree pruning** is built into XGBoost — λ regularises leaf weights, γ regularises the number of leaves.

XGBoost vs Plain Gradient Boosting

Feature	XGBoost	scikit-learn GBM
Taylor expansion	2nd order (uses Hessian)	1st order only
Regularisation	λ (L2), γ (leaf) built-in	Via max_depth etc.
Missing values	Built-in sparsity-aware	No
Parallel split finding	Yes (column block)	No
Out-of-core learning	Yes	No
Monotone constraints	Yes	No

LightGBM and CatBoost

- **LightGBM** (Microsoft): Uses **leaf-wise** (best-first) tree growth rather than level-wise. Grows the leaf with the largest loss reduction at each step. Much faster for large datasets. Also uses **GOSS** (Gradient-based One-Side Sampling: keep all large-gradient samples, randomly sample small-gradient ones) and **EFB** (Exclusive Feature Bundling: bundle sparse features).
- **CatBoost** (Yandex): Handles categorical features natively using **ordered target encoding** (avoids leakage). Uses **symmetric trees** (same split at each level), which enables fast scoring.

Part XXV

Regularisation Techniques in Depth

Batch Normalisation

Batch Normalisation (Ioffe & Szegedy, 2015) addresses **internal covariate shift**: as training progresses, the distribution of each layer's inputs keeps changing because the parameters of all preceding layers change. This makes training slow (requires small learning rates and careful initialisation).

The Forward Pass

For a mini-batch $\mathcal{B} = \{x_1, \ldots, x_m\}$ of activations at one layer:

$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{(batch mean)}$$

$$\sigma^2_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{(batch variance)}$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{(normalise)}$$

$$y_i = \gamma\,\hat{x}_i + \beta \qquad \text{(scale and shift)}$$

γ (scale) and β (shift) are **learnable parameters**. Without them, normalisation would force activations to always have zero mean and unit variance, losing the expressive power the previous layers have learned (e.g. if the optimal activation for a given feature *should* have mean 5, BatchNorm would destroy that).

Key Idea

At test time: We cannot use mini-batch statistics (there might be a single sample). Instead, we use the **running mean and variance** accumulated during training (exponential moving average over mini-batch stats). These become fixed for inference.

Why BatchNorm Helps

1. **Allows much higher learning rates**: The normalisation prevents activations from exploding or vanishing, so we can use $\eta = 0.1$ instead of $\eta = 0.001$.
2. **Reduces sensitivity to initialisation**: Bad weight initialisation is "corrected" by the normalisation at each layer.
3. **Regularisation effect**: The mean/variance is estimated from the mini-batch, adding stochastic noise to the normalisation. This acts like a regulariser and often reduces the need for Dropout.

4. **Reduces internal covariate shift**: Each layer sees a more stable distribution of inputs, making the optimisation landscape smoother.

Layer Norm vs Batch Norm

Variant	Normalises over	Used in
Batch Norm	Batch dimension (per feature)	CNNs, MLPs
Layer Norm	Feature dimension (per sample)	Transformers, RNNs
Instance Norm	Spatial dims (per sample, per channel)	Style transfer
Group Norm	Groups of channels (per sample)	Object detection

Why Transformers use Layer Norm, not Batch Norm: With variable-length sequences and small effective batch sizes (due to sequence length), batch statistics are unreliable. Layer Norm normalises over the feature dimension of a single sample, which is stable regardless of batch size or sequence length.

Dropout

Dropout (Srivastava et al., 2014): During training, randomly set each neuron's activation to zero with probability p (the dropout rate). During inference, multiply all activations by $(1-p)$ to maintain the same expected output magnitude (or equivalently, use **inverted dropout**: divide by $(1-p)$ during training, do nothing at inference).

The Theory Behind Dropout

1. **Ensemble interpretation**: With n neurons, there are 2^n possible sub-networks (each neuron is either present or dropped). Dropout trains a different sub-network on each mini-batch and at inference averages their predictions (approximated by the $(1-p)$ scaling). Ensemble methods reduce variance; dropout achieves this cheaply.
2. **Co-adaptation prevention**: Neurons cannot rely on specific other neurons being present. They must learn independent, robust features. This forces the network to learn *distributed representations*.
3. **Adds noise**: Like BatchNorm, dropout injects noise into the forward pass, which acts as a regulariser.

Key Idea

Dropout rates by layer type:

- Fully-connected layers: $p = 0.5$ (classic).
- Transformer feed-forward layers: $p = 0.1$.
- Recurrent layers: use **variational dropout** (same mask across time steps).
- Convolutional layers: Dropout rarely helps; use **Spatial Dropout** (drop entire feature maps) if needed.
- **Never apply dropout during evaluation/inference**.

Early Stopping

Monitor validation loss during training. Stop when it stops improving (or starts increasing) for p consecutive epochs (**patience**).

Intuition

As training continues, a model's training loss keeps decreasing. But at some point, the model starts memorising training data (overfitting), and validation loss begins to rise. Early stopping freezes the model at the point of best generalisation. It is equivalent to L2 regularisation under certain conditions (for linear models with gradient descent).

Best practice: Save model checkpoint at each validation improvement. Restore the best checkpoint at the end. Do not use the final model weights.

Part XXVI

Word Embeddings and NLP Fundamentals

The Problem with One-Hot Encoding

In NLP, words are first represented as discrete tokens. The naive representation is **one-hot encoding**: a vector of length $|V|$ (vocabulary size) with a 1 at the word's index and 0s elsewhere.

Problems:

- **Dimensionality**: $|V| \approx 50{,}000$–$100{,}000$ for English.
- **No semantic information**: The inner product of any two one-hot vectors is 0 — "cat" and "kitten" are as different as "cat" and "car".
- **Sparsity**: Most entries are zero.

Word embeddings solve this by mapping words to dense vectors in $\mathbb{R}^d$ ($d \in [50, 1024]$) that capture semantic relationships.

Word2Vec: Learning Embeddings from Context

Word2Vec (Mikolov et al., 2013) learns embeddings by training a shallow neural network to predict words from their context (or vice versa).

Skip-Gram: Predict Context from Word

Task: Given the centre word w_c, predict the surrounding context words $w_{c-k}, \ldots, w_{c-1}, w_{c+1}, \ldots, w_{c+k}$ (window size k).

Model: Two embedding matrices:

- $E \in \mathbb{R}^{|V| \times d}$: input (centre word) embeddings.
- $E' \in \mathbb{R}^{|V| \times d}$: output (context word) embeddings.

For centre word w_c with embedding $v_c = E[w_c]$ and context word w_o with output embedding $u_o = E'[w_o]$:

$$P(w_o \mid w_c) = \frac{\exp(u_o^\top v_c)}{\sum_{w \in V} \exp(u_w^\top v_c)}$$

Objective (maximise log-likelihood over all (centre, context) pairs):

$$\mathcal{L} = \sum_{(c,o)} \log P(w_o \mid w_c)$$

Key Idea

The softmax over all $|V|$ words is computationally expensive. In practice, **Negative Sampling** is used: for each positive pair (w_c, w_o), sample K negative words w_n and maximise:

$$\log \sigma(u_o^\top v_c) + \sum_{k=1}^{K} \mathbb{E}_{w_n \sim P_n} \left[\log \sigma(-u_{w_n}^\top v_c) \right]$$

where $P_n(w) \propto f(w)^{3/4}$ (unigram distribution raised to 3/4 power, which gives rarer words slightly higher sampling probability).

CBOW: Predict Word from Context

Task: Given the context words, predict the centre word. CBOW averages the context embeddings and predicts:

$$P(w_c \mid \text{context}) = \text{softmax}\left(\bar{v}_{\text{ctx}}^\top E'\right)$$

where $\bar{v}_{\text{ctx}} = \frac{1}{2k} \sum_j E[w_j]$ for the $2k$ context words.

	Skip-Gram	**CBOW**
Predicts	Context from centre word	Centre word from context
Better for	Rare words	Common words
Training time	Slower	Faster

The Magic of Word2Vec Embeddings

Word2Vec embeddings encode semantic and syntactic relationships as linear directions in the embedding space:

$$\text{vec}(\text{“king”}) - \text{vec}(\text{“man”}) + \text{vec}(\text{“woman”}) \approx \text{vec}(\text{“queen”})$$

$$\text{vec}(\text{“Paris”}) - \text{vec}(\text{“France”}) + \text{vec}(\text{“Italy”}) \approx \text{vec}(\text{“Rome”})$$

This arises because words that appear in similar contexts get similar embeddings (the **distributional hypothesis**: “you shall know a word by the company it keeps”).

GloVe: Global Vectors for Word Representation

GloVe (Pennington et al., 2014) argues that Word2Vec is wasteful: it trains on individual context windows but never explicitly uses the global co-occurrence statistics of the corpus.

GloVe defines the **co-occurrence matrix** X where X_{ij} is the number of times word j appears in the context of word i. It then learns embeddings by fitting:

$$v_i^\top \tilde{v}_j + b_i + \tilde{b}_j \approx \log X_{ij}$$

with a weighted least-squares objective:

$$\mathcal{L} = \sum_{i,j} f(X_{ij}) \left(v_i^\top \tilde{v}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

where $f(x) = \min((x/x_{\max})^{\alpha}, 1)$ downweights very frequent pairs.

Key Idea

GloVe directly factorises the log co-occurrence matrix. The key insight is that *ratios* of co-occurrence probabilities encode meaning: $P(\text{ice} \mid \text{solid})/P(\text{steam} \mid \text{solid})$ is large — this ratio is what the embeddings learn to represent.

FastText: Subword Embeddings

FastText (Bojanowski et al., 2017) extends Word2Vec by representing each word as a bag of **character n-grams**. For example, the word "where" with $n = 3$ gives: $\langle$`wh, whe, her, ere, re`$\rangle$, $\langle$`where`$\rangle$.

The word embedding is the sum of its n-gram embeddings.

Advantages:

- Handles **out-of-vocabulary (OOV) words**: A word never seen during training can still get an embedding from its character n-grams.
- Better for morphologically rich languages (Turkish, Finnish, Arabic) where many word forms share a root.
- Captures spelling patterns ("running", "runs", "run" share subwords).

Part XXVII

Modern Deep Learning Architectures

ResNet: Residual Networks

ResNet (He et al., 2016) introduced **skip connections** (residual connections) to solve the **degradation problem**: adding more layers to a deep network made it harder to train, causing *higher* training error even without overfitting.

The Residual Block

Instead of learning a mapping $H(x)$ directly, a residual block learns the **residual** $F(x) = H(x) - x$:

$$\text{Output} = F(x, \{W_i\}) + x$$

```
Input x

   [Conv  BN  ReLU  Conv  BN]  F(x)

   x (identity shortcut)


                                  F(x) + x  ReLU  Output
```

Why does this work?

1. **Gradient highway**: During backpropagation, the gradient flows directly through the skip connection: $\frac{\partial \text{Output}}{\partial x} = \frac{\partial F}{\partial x} + 1$. The $+1$ ensures the gradient never vanishes completely, even if $\frac{\partial F}{\partial x} \approx 0$. This is the key fix for the vanishing gradient problem in very deep networks.
2. **Identity initialisation**: If the residual $F(x)$ is zero- initialised, the block is an identity function. It is easier for the network to learn small corrections $F(x)$ to the identity than to learn arbitrary mappings $H(x)$ from scratch.
3. **Ensemble interpretation**: A ResNet with n residual blocks can be seen as an ensemble of 2^n networks of different depths (each "sub-network" is defined by which skip connections to follow).

Bottleneck Blocks (ResNet-50/101/152)

For deeper networks, a **bottleneck** design reduces computation:

$$\underbrace{1\times1 \text{ Conv}}_{\text{reduce channels}} \to 3\times3 \text{ Conv} \to \underbrace{1\times1 \text{ Conv}}_{\text{restore channels}} + \text{skip}$$

The 1×1 convolutions compress and then expand the channel dimension, reducing FLOPs while maintaining expressiveness.

BERT: Bidirectional Encoder Representations from Transformers

BERT (Devlin et al., 2018) is a Transformer encoder pre-trained on two self-supervised tasks, then fine-tuned on downstream NLP tasks.

Architecture

- **BERT-Base**: 12 Transformer encoder layers, 12 attention heads, 768 hidden dimensions, 110M parameters.
- **BERT-Large**: 24 layers, 16 heads, 1024 hidden, 340M parameters.
- Input: `[CLS]` token + WordPiece tokens + `[SEP]`. Token embeddings + Position embeddings + Segment embeddings (A/B for sentence pairs).

Pre-training Tasks

1. **Masked Language Modelling (MLM)**: Randomly mask 15% of input tokens. Of those, replace 80% with `[MASK]`, 10% with a random word, 10% unchanged. Train the model to predict the original token. This forces **bidirectional context**: the model must use both left and right context to predict the masked word.
2. **Next Sentence Prediction (NSP)**: Given two sentences A and B, predict whether B actually follows A (50% yes, 50% random). Helps with tasks requiring cross-sentence understanding (QA, NLI).

Fine-Tuning BERT

Add a task-specific head on top of the `[CLS]` token representation and fine-tune all parameters jointly:

- **Classification**: Linear layer on `[CLS]`.
- **Token classification (NER)**: Linear on each token.
- **QA**: Predict start and end span indices.

Key Idea

Why BERT is powerful: Pre-training on 3.3×10^9 words teaches the model about language structure, facts, and reasoning. Fine-tuning needs far less labelled data because these capabilities are already encoded in the weights. This is **transfer learning** for NLP.

GPT: Generative Pre-Training

GPT (Radford et al., 2018) and its successors (GPT-2, GPT-3, GPT-4) use a **Transformer decoder** pre-trained as a **causal language model**: predict the next token given all previous tokens.

Causal (Masked) Self-Attention

Unlike BERT (bidirectional), GPT uses a **causal mask**: each token can only attend to tokens at the same or earlier positions. This is enforced by setting the attention weights $e_{ij} = -\infty$ for $j > i$ before the softmax:

$$\text{Attention}(Q, K, V)_{ij} = \text{softmax}\left(\frac{q_i k_j^\top}{\sqrt{d_k}} + M_{ij}\right) v_j, \quad M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}$$

GPT Architecture

Input $\to$ Token Emb + Pos Emb $\to$ $N \times$ [LayerNorm $\to$ CausalSelfAttention $\to$ Residual $\to$ LayerNorm $\to$ FFN $\to$ Residual] $\to$ LayerNorm $\to$ LM Head (linear + softmax)

The **FFN** (Feed-Forward Network) inside each block is:

$$\text{FFN}(x) = W_2\,\text{GELU}(W_1 x + b_1) + b_2$$

with $d_{\text{ff}} = 4 \times d_{\text{model}}$ (4× expansion).

BERT vs GPT: Key Differences

Property	BERT	GPT
Transformer component	Encoder (bidirectional)	Decoder (causal/unidirectional)
Pre-training task	MLM + NSP	Causal language modelling
Strong at	Classification, NER, QA	Generation, summarisation
Context	Sees whole sequence	Only sees past tokens

Part XXVIII

Feature Engineering and Data Preprocessing

Why Feature Engineering Still Matters

Even with deep learning, feature engineering is critical for:

- Tabular data (tree-based models and linear models dominate).
- Small datasets (deep learning needs lots of data).
- Interpretability requirements.
- Inference speed constraints.

Key Idea

"Coming up with features is difficult, time-consuming, and requires expert knowledge. Applied machine learning is basically feature engineering." — Andrew Ng

Handling Missing Values

Strategy	When to Use
Mean/Median imputation	Numeric columns with MCAR (Missing Completely At Random). Median for skewed distributions. Adds no new information.
Mode imputation	Categorical columns. Dangerous if the mode dominates.
KNN imputation	Use nearest neighbours to impute. Preserves correlations. Expensive for large datasets.
Iterative imputation (MICE)	Model each feature as a function of the others. Best quality. Use for MAR data.
Indicator column	Add a binary column `is_missing`. Lets the model learn from the missingness pattern itself. Always do this alongside numeric imputation.
Drop column	If >70% missing and not informative.
Tree models	XGBoost/LightGBM handle missing values natively. No imputation needed.

Types of Missingness:

- **MCAR**: Missing Completely At Random. Unrelated to any values. Random equipment failure. Imputation is safe.

- **MAR**: Missing At Random. Missingness depends on *other* observed variables. A survey question about income is more likely to be skipped by high earners, but we can model this from other demographics.
- **MNAR**: Missing Not At Random. Missingness depends on the missing value itself. Truly problematic; imputation introduces bias.

Encoding Categorical Features

Encoding	When / How
One-Hot Encoding	Low-cardinality nominal features ($\|C\| < 15$). Creates $\|C\| - 1$ binary columns. Avoid with tree models (inefficient but OK).
Label Encoding	Ordinal features (small, medium, large) or tree-based models. Never for linear models with nominal features (introduces false order).
Target Encoding	Mean of target per category. Powerful for high-cardinality. Risk: target leakage. Use cross-fit or smoothed version: $\hat{\mu}_c = \frac{n_c \bar{y}_c + m\bar{y}}{n_c + m}$ where m is smoothing factor.
Count/Frequency Encoding	Replace category with its count or frequency. Robust, no leakage. Useful for high-cardinality.
Binary Encoding	Encode label as binary digits, convert to columns. Reduces from $\|C\|$ to $\lceil \log_2 \|C\| \rceil$ columns.
Hashing Trick	Map categories to fixed number of buckets via hash. Fast, handles OOV. Some collision risk. Used in large-scale systems.
Embeddings	Train a neural network embedding layer. Learns semantic relationships. Best for very high cardinality ($\|C\| > 1000$).

Feature Scaling

Method	Formula	Use When
StandardScaler (Z-score)	$x' = (x - \mu)/\sigma$	Gaussian-distributed; for linear models, SVM, PCA, neural nets
MinMaxScaler	$x' = (x - \min)/(\max - \min)$	Bounded features; image pixels [0,1]
RobustScaler	$x' = (x - Q_{50})/(Q_{75} - Q_{25})$	Outliers present
MaxAbsScaler	$x' = x/\lvert x\rvert_{\max}$	Sparse data; preserves sparsity
Log transform	$x' = \log(1 + x)$	Right-skewed distributions; make more Gaussian
Box-Cox	Generalised power transform to Gaussianise	Strictly positive features

Key Idea

Do NOT scale before tree-based models (Random Forest, XGBoost, LightGBM). Trees only care about the ordering of feature values, not their magnitude. Scaling has zero effect on their performance but wastes time.

Handling Class Imbalance

In binary classification, when one class vastly outnumbers the other (e.g. fraud detection: 0.1% positive), standard training fails — a model that always predicts "not fraud" achieves 99.9% accuracy but is useless.

Data-Level Methods

- **Random Oversampling**: Duplicate minority class examples. Simple but causes overfitting (exact copies).
- **SMOTE** (Synthetic Minority Oversampling Technique): Synthesise new minority samples by interpolating between existing ones. For each minority sample x_i, pick a random neighbour x_{nn} and create: $x_{\text{new}} = x_i + \lambda(x_{nn} - x_i)$, $\lambda \sim U[0, 1]$.
- **Random Undersampling**: Remove majority class examples. Fast but discards potentially useful data.
- **Tomek Links**: Remove majority samples that are nearest neighbours of minority samples — cleans the boundary.

Algorithm-Level Methods

- **Class weights**: Set `class_weight={0:1, 1:99}` in scikit-learn. The loss for minority class errors is penalised 99× more. XGBoost: `scale_pos_weight = #neg / #pos`.
- **Threshold moving**: After training a probability model, lower the decision threshold (e.g. from 0.5 to 0.2) to increase recall for the minority class.
- **Focal Loss** (Lin et al., 2017 for RetinaNet): Modifies cross-entropy to downweight easy (well-classified) examples: $\text{FL}(p_t) = -(1-p_t)^\gamma \log(p_t)$. When $\gamma = 2$, easy examples contribute 100× less than hard ones.

Feature Selection

Method	How it works
Filter methods	Rank features by a univariate statistic, independent of the model. Pearson correlation, mutual information, χ^2 test, ANOVA F-score. Fast.
Wrapper methods	Use model performance to evaluate feature subsets. Forward selection (greedily add), backward elimination (greedily remove), RFE (Recursive Feature Elimination). Slow but accounts for interactions.
Embedded methods	Feature selection is part of training. LASSO (L1) drives coefficients to zero. Tree-based feature importance. Fast, accounts for interactions.
Permutation importance	Randomly shuffle a feature; measure performance drop. Model-agnostic. More reliable than impurity-based importance.
SHAP values	Shapley values from game theory. Consistent, model-agnostic attribution. Gold standard for feature importance.

Feature Creation and Transformation

- **Interaction features**: $x_1 \cdot x_2$, x_1/x_2. Critical for capturing non-linear relationships with linear models.
- **Polynomial features**: x^2, x^3. Explicitly model non-linear relationships.
- **Binning**: Convert continuous to categorical (age groups). Helpful for noisy features or when the relationship is step-function-like.
- **Date/time decomposition**: Extract year, month, day-of-week, hour, is_weekend, days_since_event from timestamp features.
- **Aggregation features**: For each entity, aggregate statistics (mean, std, count, max, min) over a time window or group. Core of most Kaggle-winning pipelines for tabular data.
- **Lag features**: For time series, shift the target variable to create features: $y_{t-1}, y_{t-2}, \ldots$ (autoregressive features).

- **Target encoding**: Replace category with mean target value (see encoding section above).
- **TF-IDF**: Convert text to weighted term-frequency vectors. $\text{TF-IDF}(t, d) = tf(t, d) \cdot \log(N/df(t))$.

Part XXIX

Recommendation Systems

The Recommendation Problem

Given a set of m users and n items, predict which items each user will like (or the rating they would give). The interaction matrix $R \in \mathbb{R}^{m \times n}$ is extremely sparse ($> 99\%$ empty for Netflix-scale systems).

Approach	Data Used	Cold Start?
Collaborative Filtering	User-item interactions only	Fails for new users/items
Content-Based Filtering	Item features, user features	Handles new items
Hybrid	Both	Best of both worlds

Collaborative Filtering

Memory-Based: User-User and Item-Item

User-User CF: To recommend items for user u:

1. Find the k most similar users to u (using cosine or Pearson similarity on their rating vectors, ignoring unrated items).
2. Recommend items these similar users liked but u hasn't seen.
3. Predicted rating: $\hat{r}_{ui} = \bar{r}_u + \dfrac{\sum_{v \in N_k(u)} \text{sim}(u,v)(r_{vi} - \bar{r}_v)}{\sum_{v \in N_k(u)} |\text{sim}(u,v)|}$

Item-Item CF (Amazon's approach): Compute similarity between items and recommend items similar to what the user has already interacted with. More stable than user-user (item similarities change slower than user tastes).

Model-Based: Matrix Factorisation

The key insight: even though R has $m \times n$ entries, the "true" number of factors driving preferences is small ($k \ll \min(m, n)$). Decompose:

$$R \approx PQ^\top$$

where $P \in \mathbb{R}^{m \times k}$ (user factors) and $Q \in \mathbb{R}^{n \times k}$ (item factors). The predicted rating:

$$\hat{r}_{ui} = p_u^\top q_i + b_u + b_i + \mu$$

where b_u, b_i are user and item biases and μ is the global mean rating.

Training objective (minimise over observed ratings):

$$\min_{P,Q,b} \sum_{(u,i) \in \Omega} (r_{ui} - \hat{r}_{ui})^2 + \lambda \left(\|P\|_F^2 + \|Q\|_F^2 + \|b_u\|^2 + \|b_i\|^2 \right)$$

Solved with **SGD** or **ALS** (Alternating Least Squares — fix P, solve for Q in closed form, then fix Q, solve for P; repeat).

Neural Collaborative Filtering

Replace the dot product $p_u^\top q_i$ with a neural network that can model non-linear interactions:

$$\hat{r}_{ui} = f(p_u \odot q_i;\, \phi)$$

where $\odot$ is elementwise product (element-wise interaction), fed into MLP layers.

Modern approach: **Two-tower model**. User tower encodes user features to embedding e_u. Item tower encodes item features to embedding e_i. Score $= e_u^\top e_i$. Trained on positive/negative pairs. At inference, use approximate nearest neighbour search (FAISS, ScaNN) to find top-k items efficiently for any user.

The Cold-Start Problem

- **New user**: No interaction history. Solutions: ask for explicit preferences (onboarding), use user features (age, location, device), or recommend globally popular items.
- **New item**: No ratings. Solutions: use item content features (genre, description, metadata), or use a content-based model for new items until enough interactions accumulate.
- **Exploration vs Exploitation** (Bandit problem): You cannot only recommend what you know the user likes (exploitation) — you must sometimes recommend unknown items to gather signal (exploration). Algorithms: ϵ-greedy, UCB, Thompson Sampling.

Part XXX

MLOps: From Notebook to Production

The ML Production Gap

Only 20% of ML project time is on model development. The other 80% is on: building pipelines, serving infrastructure, monitoring, data validation, and retraining systems. A model that works in a notebook but fails in production is worthless.

Key Idea

Training-Serving Skew: The model was trained on data with different statistical properties than it encounters in production. The #1 cause of production ML failures. Sources: different preprocessing pipelines, different data sources, data collected at a different time.

ML Training Pipeline

A production training pipeline is not just `model.fit()`. It includes:

1. **Data ingestion**: Read from data warehouse, feature store, data lake. Validate schema and statistics (Great Expectations, TFDV).
2. **Feature engineering**: Apply the same transformations as training. Use a **feature store** (Feast, Tecton) to ensure train-serve consistency.
3. **Training**: With versioned code (Git), versioned data (DVC), and experiment tracking (MLflow, Weights & Biases).
4. **Evaluation**: On a held-out test set + business metrics. Compare against the current production model (**champion vs challenger**).
5. **Model registry**: Store the trained model with metadata, metrics, and lineage (MLflow Model Registry, SageMaker Model Registry).
6. **Deployment**: Promote to staging, run integration tests, then promote to production.

Model Serving Patterns

Pattern	Latency	Use Case
Batch inference	Hours	Monthly churn prediction, nightly recommendations
Real-time (REST)	50–200ms	Fraud detection, search ranking
Streaming (Kafka)	10–50ms	Real-time personalisation
Edge (on-device)	<5ms	Mobile keyboards, face unlock

Optimisation techniques for fast inference:

- **Quantisation**: Reduce weight precision from FP32 to INT8 or FP16. 4× smaller model, 2–4× faster inference, small accuracy loss.
- **Pruning**: Remove redundant weights (set to zero) or entire neurons. Reduces model size with structured sparsity.
- **Knowledge Distillation**: Train a small "student" model to mimic the output distribution (soft labels) of a large "teacher" model. The student learns from the teacher's confidence scores (temperature-scaled softmax), not just hard labels.
- **ONNX**: Export model to a standard format for optimised cross-platform inference (TensorRT, OpenVINO, CoreML).

Model Monitoring and Data Drift

Once deployed, a model degrades over time because the world changes.

Drift Type	Description
Data drift (covariate shift)	The distribution of input features $P(X)$ changes. Same relationship $P(Y\|X)$ but different inputs. E.g., a credit scoring model trained on 2020 data encounters 2024 spending patterns.
Concept drift	The relationship $P(Y\|X)$ changes. A model predicting clicks becomes stale as user behaviour evolves.
Label drift	The distribution of targets $P(Y)$ changes.
Prediction drift	Model output distribution changes. Often the first detectable symptom.

Detecting drift:

- **Population Stability Index (PSI)**: $\text{PSI} = \sum_i (A_i - B_i) \ln(A_i/B_i)$. PSI < 0.1: stable. PSI 0.1–0.2: slight change. PSI > 0.2: significant drift.
- **Kolmogorov-Smirnov test**: Non-parametric test for difference in distributions. For each feature, compare training vs. production distribution.
- **Jensen-Shannon divergence**: Symmetric version of KL divergence.
- **Monitor performance metrics**: Requires ground truth labels (often delayed). Track accuracy, AUC, F1 over time.

A/B Testing for ML Models

Before full deployment, test the new model on a fraction of live traffic.

1. **Split traffic**: Route $X\%$ of users/requests to the new model (treatment), $100 - X\%$ to the old model (control).

2. **Define metrics**: Online metric aligned with business goal (CTR, conversion rate, revenue, engagement time). Guard metrics (latency, error rate) ensure no regression.
3. **Statistical significance**: Run until you achieve 95% confidence. Use a two-sample t-test or z-test for proportions. Minimum detectable effect (MDE) determines sample size needed.
4. **Avoid peeking**: Do not evaluate results until the planned sample size is reached (stops p-hacking).

Part XXXI

ML Interview Corner: Common Questions with Deep Answers

Fundamentals

Bias-Variance Tradeoff The Full Picture

Q: Explain the bias-variance decomposition and what it means for model selection.

For any model $\hat{f}$, the expected MSE on a test point x decomposes as:

$$\mathbb{E}[(y-\hat{f}(x))^2] = \underbrace{\left(\mathbb{E}[\hat{f}(x)] - f(x)\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)]\right)^2\right]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}$$

- **Bias**: How wrong is the model *on average*? High bias means the model misses systematic patterns (underfitting). Example: fitting a linear model to quadratic data.
- **Variance**: How sensitive is the model to the training data? High variance means the model overfits to the specific training set, changing drastically with new data. Example: a depth-20 decision tree.
- **The tradeoff**: Increasing model complexity reduces bias but increases variance. The sweet spot minimises total error.
- **Ensemble methods**: Random Forest reduces variance (via averaging); Boosting reduces bias (by correcting residuals). This is why ensembles are powerful.

Cross-Validation for Time Series

Q: Can you use standard K-fold CV for time series? Why not?

No. Standard K-fold randomly assigns samples to folds. This causes **data leakage**: future data (fold k) gets used to train, and past data (fold $j < k$) gets used to validate. The model "sees the future", leading to optimistically biased performance estimates.

Correct approach — Time Series Split (walk-forward validation):

1. Fold 1: Train on $[1,\, k]$, validate on $[k+1,\, k+h]$.
2. Fold 2: Train on $[1,\, k+h]$, validate on $[k+h+1,\, k+2h]$.
3. ... always train on the past, validate on the future.

This mirrors how the model will actually be deployed: always predicting the future from past data.

Precision vs Recall When to Optimise Which?

- **Optimise Recall** (minimise false negatives) when missing a positive is very costly: cancer screening (missing a tumour is worse than a false alarm), fraud detection (missing fraud is

worse than blocking a legitimate transaction), email spam (missing spam is fine, but blocking a legitimate email is bad).

- **Optimise Precision** (minimise false positives) when acting on a false positive is costly: content recommendation (showing irrelevant ads wastes budget), search engines (returning irrelevant results degrades experience).
- **Use F1** when you need a single metric balancing both.
- **Use Fβ** ($\beta > 1$ weights recall more, $\beta < 1$ weights precision more).

Tree-Based Models

Why Random Forests Reduce Variance

Q: Mathematically explain why averaging decision trees in a Random Forest reduces variance.

A single tree $\hat{f}(x)$ has variance σ^2.

If we average B **independent** trees, the variance of the average is:

$$\text{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x)\right) = \frac{\sigma^2}{B}$$

Variance shrinks to zero as $B \to \infty$.

But trees trained on bootstrap samples of the same data are **not independent** — they are correlated ($\text{Corr}(\hat{f}_i, \hat{f}_j) = \rho$). For B trees with pairwise correlation ρ:

$$\text{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

As $B \to \infty$, this converges to $\rho\sigma^2$, not zero.

Random feature subsampling (at each split, consider only $\sqrt{p}$ features) *decorrelates* the trees (reduces ρ), which directly reduces the variance of the ensemble.

XGBoost vs LightGBM vs CatBoost

Q: When would you choose each?

- **XGBoost**: When you need a well-understood, robust baseline. Good explainability tools. Tends to generalise well with careful tuning.
- **LightGBM**: When you have a very large dataset ($> 10^6$ rows) and need fast training. The GOSS sampling and leaf-wise growth make it significantly faster than XGBoost.
- **CatBoost**: When you have many high-cardinality categorical features and don't want to manually encode them. Its ordered target encoding avoids leakage without cross-fold tricks.

Deep Learning

Explain the Vanishing Gradient Problem and its Solutions

In a deep network, gradients are propagated backward via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{\partial \mathcal{L}}{\partial a^{[L]}} \cdot \prod_{l=2}^{L} \frac{\partial a^{[l]}}{\partial a^{[l-1]}}$$

Each factor is $W^{[l]} \cdot \sigma'(z^{[l]})$. For sigmoid/tanh: $\sigma'_{\max} = 0.25$ (sigmoid), $\tanh'_{\max} = 1.0$. With 50 sigmoid layers: gradient shrinks by $0.25^{50} \approx 10^{-30}$. Early layers learn essentially nothing.

Solutions:

1. **ReLU activations**: $\sigma'(z) = 1$ for $z > 0$. No saturation in the positive region. But can cause **dying ReLU** (neurons stuck at 0 permanently). Use **Leaky ReLU** (αz for $z < 0$) or **GELU** (smooth approximation, used in Transformers).
2. **Residual connections**: Add identity skip connections (ResNet). Gradient flows unimpeded through the skip path.
3. **Batch Normalisation**: Normalises activations at each layer, preventing gradient explosion/-vanishing.
4. **Careful weight initialisation**:
   - **Xavier/Glorot**: $W \sim \mathcal{N}(0, \frac{2}{n_{in}+n_{out}})$. For sigmoid/tanh.
   - **He initialisation**: $W \sim \mathcal{N}(0, \frac{2}{n_{in}})$. For ReLU (accounts for half the neurons being dead).
5. **Gradient clipping**: Cap gradient norm to a maximum value. Critical for RNNs trained with BPTT.

Attention is All You Need — Core Idea

Q: Why is attention better than RNNs for long sequences?

- **RNN limitation**: Information from position 1 must flow through $n-1$ hidden state transitions to affect position n. Each transition can corrupt the signal. Maximum path length: $O(n)$.
- **Attention**: Every position attends directly to every other position. Path length: $O(1)$. The relationship between position 1 and position n is computed in one step.
- **Parallelism**: RNNs are inherently sequential (step t depends on step $t-1$). Attention operates on all positions simultaneously.
- **Trade-off**: Attention is $O(n^2)$ in memory/compute (every pair). For very long sequences ($n > 10^4$), this becomes expensive. Solutions: sparse attention (Longformer), linear attention (Performer).

ML Systems Design Questions

How Would You Design a Movie Recommendation System?

Step 1: Clarify requirements

- Scale: 10M users, 100K movies. 1M daily active users.
- Latency: $<$ 100ms for homepage recommendations.
- Freshness: New movies should appear in recommendations within hours.

Step 2: Two-stage architecture (industry standard)

- **Retrieval (Recall stage)**: From 100K movies, quickly retrieve a candidate set of ∼1000 for each user. Use approximate nearest neighbour search on user/item embeddings (FAISS). Fast (<10ms).
- **Ranking stage**: Score the ∼1000 candidates with a rich feature model (user history, item features, context). Return top 20. Can use a deep neural net. Slower (∼50ms).

Step 3: Feature engineering

- User features: watch history, ratings, demographics, time of day.
- Item features: genre, director, cast, release year, average rating, popularity.
- Context features: device, current time, session length.
- Cross features: user's preferred genre × item genre match.

Step 4: Handling cold start

- New users: Recommend globally trending + content-based for explicit preferences.
- New movies: Use content features (genre, synopsis embeddings) until interactions accumulate.

Step 5: Training and updating

- Implicit feedback: watch time, completion rate, re-watches (not just clicks).
- Retrain daily/weekly. Monitor for popularity bias and filter bubbles.
- A/B test new models before full rollout.

How Do You Handle a Skewed Dataset in Classification?

Full answer: (1) Understand the degree of imbalance and the business cost of FP vs FN. (2) Choose the right metric: not accuracy. Use ROC-AUC for overall discrimination, PR-AUC for imbalanced positive class, Fβ if FN cost $\neq$ FP cost. (3) Data level: SMOTE for minority class oversampling, or clean undersampling. (4) Algorithm level: class weights in the loss function (most effective, try this first). (5) Threshold optimisation: find the threshold on the validation set that optimises your business metric (not default 0.5). (6) Ensemble: combine multiple models; their errors may be uncorrelated. (7) Anomaly detection: if positive class is very rare ($<$ 0.01%), frame as anomaly detection instead.

Explain L1 vs L2 Regularisation — Deep Answer

L2 (Ridge): Adds $\lambda\|\theta\|_2^2$ to the loss. Gradient: $2\lambda\theta$. Penalty is proportional to current weight, so large weights are penalised more. *All weights shrink toward zero*, but rarely reach exactly zero. Corresponds to a Gaussian prior on weights: $\theta \sim \mathcal{N}(0, 1/(2\lambda))$.

L1 (Lasso): Adds $\lambda\|\theta\|_1$ to the loss. Subgradient: $\lambda \cdot \text{sign}(\theta)$. The penalty is constant magnitude (λ) regardless of the weight value. This means:

- Small weights experience the same shrinkage force as large ones.
- Weights that are almost zero get pushed all the way to exactly zero.
- This produces **sparse solutions**: many weights are exactly zero (automatic feature selection). Corresponds to a Laplace prior on weights.

Geometric intuition: The constraint set for L1 is a diamond (in 2D), for L2 a circle. The loss ellipses typically first touch the diamond at a corner (where one weight is zero), creating sparsity. They touch the circle off-corner, rarely producing zeros.

When Does a Neural Network Fail to Learn?

Common failure modes and diagnostics:

- **Loss is NaN or explodes**: Learning rate too high, or no gradient clipping for RNNs. Fix: lower η, add gradient clipping.
- **Loss decreases then plateaus high**: Underfitting. Fix: more capacity (deeper/wider), train longer, better features.
- **Train loss low, val loss high**: Overfitting. Fix: dropout, regularisation, data augmentation, early stopping, more data.
- **Training loss oscillates**: Learning rate too high. Fix: reduce η or use warmup.
- **Loss barely decreases**: LR too small, or bad initialisation, or vanishing gradients. Fix: increase η, add BatchNorm, use He init.
- **Dying ReLU**: Many neurons always output zero. Fix: Leaky ReLU, smaller learning rate, better initialisation.
- **Class imbalance**: Loss looks OK but model always predicts majority. Fix: class weights, check per-class metrics not just overall accuracy.

References

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[2] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.

[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, 1998.

[5] A. Vaswani et al., "Attention Is All You Need," *NeurIPS*, 2017.

[6] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 9(8):1735–1780, 1997.

[7] I. Goodfellow et al., "Generative Adversarial Networks," *NeurIPS*, 2014.

[8] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," *ICLR*, 2014.

[9] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," *NeurIPS*, 2020.

[10] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, 518:529–533, 2015.

[11] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[12] L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," *Journal of Machine Learning Research*, 2008.

[13] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction," *arXiv:1802.03426*, 2018.

[14] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," *NeurIPS*, 2017.

[15] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why Should I Trust You?': Explaining the Predictions of Any Classifier," *KDD*, 2016.

[16] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016.

[17] J. Kaplan et al., "Scaling Laws for Neural Language Models," *arXiv:2001.08361*, 2020.

[18] W. S. Gosset ("Student"), "The Probable Error of a Mean," *Biometrika*, 6(1):1–25, 1908.

[19] B. L. Welch, "The generalization of 'Student's' problem when several different population variances are involved," *Biometrika*, 34(1–2):28–35, 1947.

[20] S. S. Shapiro and M. B. Wilk, "An analysis of variance test for normality (complete samples)," *Biometrika*, 52(3–4):591–611, 1965.

[21] H. Levene, "Robust tests for equality of variances," in *Contributions to Probability and Statistics*, Stanford University Press, 1960.

[22] J. Durbin and G. S. Watson, "Testing for Serial Correlation in Least Squares Regression," *Biometrika*, 37(3–4):409–428, 1950.

[23] T. S. Breusch and A. R. Pagan, "A Simple Test for Heteroscedasticity and Random Coefficient Variation," *Econometrica*, 47(5):1287–1294, 1979.

[24] H. B. Mann and D. R. Whitney, "On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other," *Annals of Mathematical Statistics*, 18(1):50–60, 1947.

[25] F. Wilcoxon, "Individual Comparisons by Ranking Methods," *Biometrics Bulletin*, 1(6):80–83, 1945.

[26] W. H. Kruskal and W. A. Wallis, "Use of Ranks in One-Criterion Variance Analysis," *Journal of the American Statistical Association*, 47(260):583–621, 1952.

[27] M. Friedman, "The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance," *Journal of the American Statistical Association*, 32(200):675–701, 1937.

[28] J. Demšar, "Statistical Comparisons of Classifiers over Multiple Data Sets," *Journal of Machine Learning Research*, 7:1–30, 2006.

[29] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages," *Psychometrika*, 12(2):153–157, 1947.

[30] E. R. DeLong, D. M. DeLong, and D. L. Clarke-Pearson, "Comparing the Areas under Two or More Correlated Receiver Operating Characteristic Curves: A Nonparametric Approach," *Biometrics*, 44(3):837–845, 1988.

[31] Y. Benjamini and Y. Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing," *Journal of the Royal Statistical Society: Series B*, 57(1):289–300, 1995.

[32] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *Annals of Statistics*, 7(1):1–26, 1979.